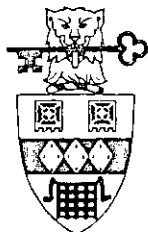


VECTOR

100+ Pages of the Best in APL

	Page
* 16-page Educational Supplement	17
* Metapraxis Design Award	35
* APL/SQL Integration	75
* Function Point Analysis	79
* Jizba on Generic Locals	103
* Sullivan on an Unknown APL2 AP	113



*The Journal of the
British APL Association*

ISSN 0955-1433

Vol.5 No.3 January 1989

A Specialist Group of the British Computer Society

Contributions

All contributions to VECTOR may be sent to the Editor at the address on the inside back cover. Letters and articles are welcome on any topic of interest to the APL community. These do not need to be limited to APL themes, nor must they be supportive of the language. Articles should be accompanied by as much visual material as possible (ideally with a photograph of the author and a brief biographical note). Unless otherwise specified, each item will be considered for publication as a personal statement by the author.

Please supply as much material as possible in machine-readable form, ideally on an IBM PC compatible (DSDD) diskette. APL code should be kept separate from the text, and can be accepted as camera-ready copy, or as APL*PLUS/PC workspaces or IBM .AIO files.

Except where indicated, items in VECTOR may be freely reprinted with appropriate acknowledgement. Please inform the editor of your intention to re-use material from VECTOR.

Membership Rates 1989-90

Category	Fee	Vectors	Passes
Non-voting Student	5	1	1
UK Private	10	1	1
Overseas Private	18	1	1
(Supplement for Airmail)	8		
UK Corporate Membership	85	10+	5
Overseas Corporate	140	10+	
Sustaining	360	50+	5

The membership year runs from 1st May to 30th April. Applications for membership should be made to the Treasurer on the form on the inside back page of VECTOR. Passes are required for entry to some association events, and for voting at the Annual General Meeting. Applications for student membership will be accepted on a recommendation from the course supervisor. Overseas membership rates cover VECTOR surface mail, and may be paid in sterling, or by Visa or Mastercard at the prevailing exchange rate.

Corporate membership is offered to organisations where APL is in professional use. Corporate members receive 10 copies of VECTOR, and are offered group attendance at association meetings. A contact person must be identified for all communications.

Sustaining membership is offered to companies trading in APL products; this is seen as a method of promoting the growth of APL interest and activity. As well as receiving public acknowledgement for their sponsorship, sustaining members receive bulk copies of VECTOR, and are offered news listings in each issue.

Advertising

Advertisements in VECTOR should be submitted in typeset camera-ready A5 portrait format with a 20mm blank border. Illustrations should be black-and-white photographs or line drawings. Rates are 250 per full page, 100 for half-page or less (subject to layout constraints).

Deadlines for bookings and copy are given under the Quick-reference Diary

Advertisements should be booked with, and sent to, Cathy Dargue, whose address is given with the Index to Advertisers.

CONTENTS

Guest Editorial:	David Preedy	Page 3
APL NEWS		
Quick Reference Diary		7
General Correspondence		9
News from Sustaining Members	Cathy Dargue	13
The Education VECTOR	Alan Sykes	17
British APL Association News	Graham Parkhouse	33
1989 Design Award to Metapraxix	David Preedy	35
APL People: Survey Results	Jill Moss	36
REVIEWS		
APL Product Guide	Cathy Dargue	39
New Product Announcements		40
Product Catalogue		41
APL Booklist	Ed Shaw	50
Book Reviews ...		
APL is Easy!/APL2 at a Glance	Bill Ross	51
RECENT MEETINGS		
Nov 18th: APL at the Leading Edge		56
Expert Systems with Existing Software	Allan Prys Williams	57
Integrated Information: APL and SQL	Brian Lochran	75
January 13th: Function-point Analysis		79
Introduction to FPA	Romilly Cocking	79
FPA in IBM	Alan Williams	80
Case Study at British Airways	Maurice Jordan	83
February 17th: APL: Future Plans	Adrian Smith	84
GENERAL ARTICLES		
Assurance Quotations	Harry Phillips	88
Case Study: How Expensive is APL?	F.H.D van Batenburg	91
Generic Local Objects	Zdenek Jizba	103
TECHNICAL SECTION		
Technical Editorial:	Jonathan Barman	108
Technical Correspondence		110
Hackers Corner: An Undocumented AP	John Sullivan	112
A Good Character Scatterplot	Alan Sykes	116
British APL Association Software Library		
Software Submission Details	David Ziemann	124
Software Order Form	David Ziemann	126
Index to Advertisers		127



APL DEVELOPMENT PROGRAMMERS

British Airways use APL extensively to develop decision support systems for the airline. We are urgently seeking development programmers to work on a number of projects at Heathrow, providing the opportunity to get involved in both rewarding and interesting developments.

Applicants should have at least one year's experience of development using APL and preferably more. Comprehensive training will be provided to candidates where required.

In return, you will receive a competitive salary as well as an excellent benefits package which includes holiday bonus, profit sharing and favourable holiday travel opportunities. We offer a performance related pay scheme that will fully reward good performers.

For further details about the above vacancies, please telephone Nick Jones on 01-751 8150.

Alternatively, send a full c.v. to: Recruitment & Selection, Ref: TN/1143/NJ/VECTOR, British Airways Plc, "Meadowbank", PO Box 59, Hounslow TW5 9QX.



**BRITISH
AIRWAYS**

Guest editorial: Plus ca change ...

by David Preedy (Development Director), Metapraxis Ltd.

Having taken a break from writing VECTOR editorials, I was very pleased to find out that I had not disappeared completely from the records of the BAA (only from the membership and mailing lists!) and to be asked to put finger to keyboard yet again.

Standing back from the internals of Committee membership (and also being relieved of the corresponding responsibilities) gives me a better opportunity to consider how the APL community is faring as we move towards the 1990s and to review the past four "VECTOR years".

The original VECTOR working party saw itself as a major component of a two-fold strategy. One target was to improve communications, expertise and standards within the global APL family; the other was to promote an awareness of APL and its strengths to a much wider community.

Five years later, a brief perusal of recent copies of VECTOR shows that whatever progress has been made on the internal objectives, progress on raising APL's profile in the outside world has been negligible.

"... we are spending over 90% of our efforts providing services to members and almost none on persuading non-APLers to try using APL." - Anthony Camacho, VECTOR 4.1

"What we have not yet done is to carry the disease to those not already infected." - Adrian Smith, VECTOR 4.2

"I too am concerned about the thin appreciation of APL." - Richard L Connor, VECTOR 4.2

"In the whole of my professional data processing career I have never come across a community so dedicated to contemplating its own navel." - Barrie Webster, VECTOR 4.2

"no whisper of APL was heard" - At the 1985 symposium of the International Council for Mathematical Instruction on the influence of computer in Mathematics and its teaching, Norman Thompson, VECTOR 4.3

"APLers seem to expect everyone else in the computer world to share their enthusiasm for the aesthetic qualities of the language; in the real world people just want to see results, or to have effective and straightforward end-user tools made available to them" - Mark Cranny, VECTOR 5.1

To overcome this problem we have to establish APL's credibility by demonstrating its effectiveness in addressing real problems. There are three essential steps in such an exercise.

First APL has to be used effectively. We all know of the successes within our own organisations, and events like the APLications conference at Canterbury last year spread the gospel to other APLers. If we cannot establish that APL is providing effective solutions, then we might as well give up now.

Second, we have to persuade developers to publicise their successes and mention APL's contribution. This can be harder and may face several obstacles:

- secrecy, especially if the development is giving a company a significant competitive advantage; this means that the best developments are often kept under wraps;
- lack of time, needed to stand back and describe what has been achieved; many employers will be reluctant to commit resources to such activity;
- lack of writing skills, needed to describe the benefits of the project giving an appropriately short but incisive comment on why APL provided the best approach;
- defensiveness; this can be a fundamental drawback for many people who have used APL to develop a successful commercial system; for the users of the system, the fact that APL was used is normally irrelevant; for their DP advisers it merely serves to raise eye-brows; the developer wants to publicise his product, but not necessarily its use of APL.

Third, we have to ensure that the material that is produced gets to the right people outside the APL community. We have to think less about how it boosts our own collective ego, and more about what other parties get out of it. If we want an article in the computer press we have to make it newsworthy, topical and of general interest. If we want to persuade colleges to include APL in their courses, we must show them employers looking for APL expertise. If we want APL to be used in schools, we must provide a cheap interpreter that works on their computer and provide the back-up material to help the teacher. If we want system developers to use APL as a tool, we must be able to point to an active job market in APL programmers, a high reputation amongst prospective users and their advisers, and future guaranteed compatibility with the next generations of hardware and operating system.

APL's future will be determined to a large extent by the personal preferences and prejudices of the people who choose to make a career in system development. The computing techniques of the next decade are being decided in colleges now; the techniques of the next century are being shaped in the schools. In neither of these areas is APL being used significantly. Over the past four years our company has trained fourteen graduates in APL; all of them had some experience of computing - at home, at school or at college; we have yet to recruit any graduate who already knows APL.

Some actions are being taken to address this problem, notably the I-APL initiative, but the BAA should also be doing much more. The following are some suggestions; I am sure that other readers will be able to provide other, doubtless better, ideas.

There should be an APL careers profile, circulated not only to all university and college careers offices, but also to all mathematics and engineering departments and to all lecturers in the computing departments; this should list the major companies using APL, with details of career opportunities and contact points. (Incidentally this could probably be made profitable, but this is definitely not the aim.)

The BAA should sponsor well produced case-studies, some for publication in the computer press, others for use in the academic community. Each case-study must be carefully targeted so that its message is relevant to its specific audience. In this context it is notable that VECTOR started off with a case-study in each issue; the stream has dried up and in total VECTOR has covered only six case-studies, the last nearly three years ago.

Competitions should be promoted at three levels: the best school project undertaken with APL; the most innovative use of APL at university or college; and the best case-study from a business application.

The tragedy is that the BAA has for some years been able to afford to do something about this. Five years ago we were broke; after APL86 we had accumulated a war-chest of £58,000 and this is continuing to grow - up to £70,000 in 1987/88. If we can't find a good way to spend this money, we should give it back to the members - but please let me back onto the membership list first!

Dates for Future Issues of VECTOR

	Vol.5 No.4	Vol.6 No.1	Vol.6 No.2
Copy date	1st March 89	1st June 89	1st Sept 89
Ad booking	10th March 89	12th June 89	10th Sept 89
Ad Copy	25th March 89	25th June 89	21st Sept 89
Distribution	April 89	July 89	October 89

FREELANCE CONSULTANTS

Maddox Ford is a software house specialising in the development of business information systems.

These applications are implemented using a range of DP products including relational databases, 4GLs and when appropriate, APL.

Our policy is to use project teams which blend our in-house skills with complementary freelance expertise.

We aim to develop a long-term association with a number of independent DP professionals who will play a strategic part in our expansion.

If you are interested in combining the benefits of working closely with a successful consultancy whilst retaining your independence, contact:

Brian Drury on (01) 202 3094

**MADDOX
FORD**

1 Endersleigh Court, London, NW4 4SE

Quick Reference Diary

Date	Venue	Event
17 March	London	BAA Meeting
21 April	London	BAA Annual General Meeting The new committee is revealed. An event not to be missed!

All British APL Association meetings are to be held at the Royal Over-Seas League, Park Place, near Green Park Tube station. Meetings begin at 2.00pm and close at 5.30pm.

AAAA	PPPPP	LL	999999	000000
AA AA	PP PP	LL	99 99	00 00
AA AA	PP PP	LL	99 99	00 00
AAAAAA	PPPPP	LL	999999	00 00
AA AA	PP	LL	99	00 00
AA AA	PP	LL	99	00 00
AA AA	PP	LLLLLL	99	000000

The preparations for APL90 are progressing well and we hope to see you all on August 13-17 1990 in Denmark. Now may be a good time to start thinking about the paper(s) you will submit: the official call for papers will be issued shortly after the summer holidays.

In order to provide the optimal setting for the conference, it has been finally placed in Elsinore (the city of Hamlet) just north of Copenhagen. This choice offers very attractive surroundings, improved conference facilities and a smaller conference fee. Should you decide to bring the family, the conference hotels offer excellent family accommodation.

Please direct any enquiries to:

APL90
c/o Kim S. Andreasen
Snerlevej 3
DK-3000 Helsingør
DENMARK

dyalog APL

C INTERFACE

```
#include "apl.h"
#include "support.h"

main()
{
    ARRAY *rslt, *larg, *rarg;
    *getarray(), *fre
    *idn_d(), *d_idn
    fncode;

    startup(2);
    define("IDN_TO_DATE", RSU
    define("DATE_TO_IDN", RSU

    larg = rarg = rslt = 0;
    setjmp(errbuff);

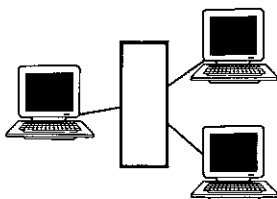
    while(1)
    {
        larg = freearray;
        rarg = freearray;
    }
}
```

UNIX INTERFACE

CHOICE OF HARDWARE

IBM Ps/2 Model 80
Compaq 386
Altos 3068
Apollo
AT&T 3B2
Bull SPS7, SPS9
Dec Vax
Hewlett-Packard HP9000
IBM 6150
Masscomp
NCR Tower 32
Pyramid
Sun
Unisys Series 5000

MULTI-USER



FILE SYSTEMS

SQL INTERFACE

EDITORS

```
[0] Z←DISPLAY Z;
[1] JIO←0 ⚡ SZ←x
[2] Z←' '(UFMT
[3] NSS:→(1 ⚡DZ
[4] NOR:Z←(1⚡pZ)p
[5] →(1=DZ)/SMP
[6] Z←' ',(QTV←1
[7] SMP:QB←(2i0 '
[8] FMT:Z←DFMT Z
[9] BOX:R C←'1+pZ
[10] C←'1+1/2 S2
[11]
```

FULL-SCREEN INPUT

Share Name

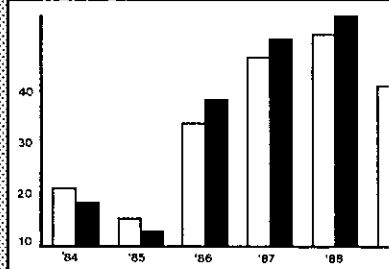
Number of Shares

Unit Cost

Buyer

EMPNO	ENAME	JOB	MGR
7329	SMITH	CLERK	7902
7499	ALLEN	SALESMAN	7698
7521	WARD	SALESMAN	7698
7566	JONES	MANAGER	7839
7654	MARTIN	SALESMAN	7698
7698	BLAKE	MANAGER	7830
7782	CLARK	MANAGER	7839
7788	SCOTT	ANALYST	7566
7839	KING	PRESIDENT	
7844	TURNER	SALESMAN	7698
7876	ADAMS	CLERK	7788

GRAPHICS INTERFACE



DEVICE SUPPORT

NESTED ARRAYS

DISPLAY REPORT

	North	South	East
Cakes	14	76	46
Buns	22	5	68

...gives you all this – and APL too!

For further information, contact
Dyadic Systems Limited
 Park House, The High Street, Alton, Hampshire, GU34 1EN
 Telephone: (0420) 87024. Telex: 858811

APL Training Courses

Dates shown cover the period from April to December 1989. For confirmation of dates and/or further details please contact the vendor directly.

Level	Company	Days	Dates
Beginners	Cocking & Drury	3	7 March, 21 March, 18 April,
			16 May, 6 June, 20 June
	MicroAPL	1	9 March, 4 May, 29 June,
			24 Aug, 19 Oct
	Mercia	3	7 March, 16 May, 17 July
Introductory	MicroAPL	1	30 Mar, 25 May, 20 July,
			14 Sept, 9 Nov
Intermediate	MicroAPL	1	16 Mar, 11 May, 6 July,
			31 Aug, 26 Oct
	Mercia	2	25 April, 6 June
Advanced	MicroAPL	1	23 March, 18 May, 13 July,
			31 Aug, 14 Sept, 26 Oct
System Design	Cocking & Drury	4	22 May
	Mercia	3	12 April, 28 June, 3 August
Statgraphics	Cocking & Drury	2	14 March, 11 April, 9 May, 13 June
	Mercia	1	15 March, 10 May, 27 June,
			2 August

General Correspondence

The VECTOR working group welcomes correspondence on any topic affecting the APL community. All letters should be addressed to the Editor, and should indicate whether they are for the general or technical section. (Letters containing APL code will normally be classed as 'technical'.) The editor reserves the right to edit any letter unless the writer states that it is to be published in full or not at all.

The State of the APL Association

From: Anthony Camacho

6 February 1989

I am concerned about the APL Association. I am also trying to help in various ways but I think more is needed than I can do. I know I am not alone in asking what has gone wrong. I have protested privately to Treasurer, Chairman and Secretary more than once. Although I have received apologies on occasions when I have personally been let down there is still no explanation or improvement. Things cannot be put right if officers neither discharge their obligations nor ask for help when things get too much for them.

The symptoms are that:

1. meetings have been arranged too late - it is a long time since Vector has been able to list the subjects of future meetings.
2. publicity has not been sent out in time for several meetings - one was cancelled because speakers withdrew when no notice had been sent out by the Monday before the meeting.
3. no advantage is taken of the free listings of events in the computer magazines or of many other opportunities of publicising APL and the Association.
4. many members who have paid their subscription have not received Vector or notices of meetings - it is some time since I was an officer of the Association, but people ring me up in the hope I can do something.
5. Vector is always late - so late that members who don't receive it assume it is lateness rather than some error in the mailing system.
6. inadequate use has been made of the hired clerical assistance (Alison Chatterton) or of the computer with which the Association provided her.
7. the Association's membership is being allowed to shrink with no visible effort to recruit new members or to persuade old members to renew their subscriptions.

The main benefit the Association provides is Vector, the other significant benefit for some members is the meetings. Surely we should do our best to see that every member gets the benefits.

The last mailing of Vector was made with an incomplete set of labels and so between 150 and 200 members (fully paid up) have not yet received Vector 5.1. As nobody knows who was left out, the Vector working group has to write to everyone and ask: how embarrassing!

The membership system in use at present is not in APL and the Association does not own the package it runs on. Surely we should use APL to maintain our membership list. The current package is inadequate and for example it took two months to get it adjusted to produce labels on the APL Association's printer.

I-APL has a membership maintenance workspace (written by Sylvia Camacho in APL*PLUS/PC) which we are donating to the Association and amending to suit its needs. We are also going to convert the membership list into a component file which can easily be maintained and used. Alison Chatterton (the Association's administrative assistant) used to work full time with APL and will find the workspace much easier to manage than a package. We have undertaken to help to produce a set of labels at any time, given two days notice.

The Activities officer's predecessor should have handed over a year's planned meetings, but did not. This explains the lateness of arrangements of the early meetings this year, but only the early ones. Publicity is still absent; notices are still late. Two of the meetings have been splendid and of great interest to professional users of APL, but very poorly attended. This, I guess, is the consequence of the failure to advertise; worse meetings had much bigger audiences two years ago. How far ahead are the meetings planned now? Today is 6 February and I have not yet received a notice of the 17 February meeting.

I am told that 164 of those who were members in 1987-88 have so far not renewed their memberships. I believe no reminders have been sent out. Is this true? A previous Treasurer once used to send out a membership subscription reminder in the form of an empty Vector - just a photocopy of the cover so that the recipients could see some of the things they were missing. This reduced the drop-out rate greatly.

The recruitment officer a year or more ago promised to prepare a list of members who would be ready to help teachers who wanted to try out APL. Now that there is an APL that anyone can afford there are quite a number of teachers who would like to experiment. There is no list. The I-APL name and address list of 1400 people is waiting for the APL Association to exploit. What has been done? Nothing!

Somewhere the duties of the officers are written down and the Chairman is supposed to plan the diary and check that each member of the committee is doing/has done the necessary work in good time to achieve the Association's aims. Plainly this is not happening at present.

I suggest that members who are able and willing to help with any of these things and feel as I do that help is plainly needed should write to or telephone the Chairman and offer.

Yours sincerely,

Anthony Camacho
2, Blenheim Road,
St Albans, Herts,
AL1 4NR

MATERIALS MANAGEMENT CONSULTANT

MERCIA SOFTWARE are a rapidly expanding company providing highly sophisticated and specialised PC based software to industry and commerce.

Due to the success of our LOGOL system for finished goods control, we need a numerate graduate to be involved with the installation, support and technical aspects of LOGOL.

You should have a degree in Stats, Maths or OR, and have experience in fields such as Sales Forecasting, Inventory Management, Scheduling or Distribution. Programming skills are vital, preferably in APL.

To apply please send a comprehensive C.V. to **Gareth Brentnall** at the address below.

MERCIA 
Software Limited

Aston Science Park,
Love Lane,
Birmingham B7 4BJ
Tel: 021-359 5096

News from Sustaining Members

Compiled by Cathy Dargue

APL People Ltd

APL People are pleased to welcome Karen Oppen who joined the company a few months ago in the Bath office. Karen is assisting Jill with the permanent recruitment side of the business (some of you will already have spoken to her on the phone). Now you can put a face to the voice! Call Karen or Jill to find out more about our service to APLers!



New Recruit Karen Oppen,
pictured with Jill Moss of APL People

HMW Computing Ltd

HMW Computing Limited continue to work on "4XTRA" our front end Foreign Exchange Dealing and Position Keeping System adding facilities to enable overall management analysis across all dealing positions both graphical and numerically. We have also improved file access times by moving away from APL component files. If you would like to see a demonstration of "4XTRA" we now have regular demonstrations taking place every Tuesday. Please call us for a booking.

HMW Employment Limited is gaining acknowledgement in the APL field and potential employers will be pleased to hear that we have numerous skilled people looking for new careers as well as a pool of well qualified contractors ready to start work now.

To find out what HMW can offer, please call Stan Wilkinson on 01-353-4212 or write to us at Hamilton House. If you would prefer to phone out of office hours Stan's home number is 01-286-7068.

MicroAPL Ltd

Version 7.10 of APL.68000 for the Apple Macintosh has been well received. The beta test finished at the end of November 1988 and shipments of the upgrade started towards the end of December. We have been encouraged by the response to the software and also by the rapidly growing numbers of Mac users in the corporate sector. The USA continues to be the strongest market for the Mac version of APL.68000, although the level of UK sales for the Mac version is growing fast. Any users who have not yet received upgrade notices should contact MicroAPL.

APL.68000 for the Apple Mac now offers an APL multi-tasking environment. An APLuser can run workspaces, EDIT tasks and a terminal emulator task at the same time. Each copy of APL.68000 supports a number of tasks, and multiple copies of APL.68000 may be run under Multifinder. Richard Nabavi discussed the way in which the feature was achieved in Vector 5.1.

Upgrades for the Commodore Amiga and Atari ST versions of the APL.68000 are being planned. These will incorporate many, if not all, of the features of the Mac version. We hope to announce the first of these in the next issue of Vector. If any users have upgrade suggestions, you should have sent them in by now!

MicroAPL runs regular courses covering many topics from the elementary use of APL through to advanced APL Systems Programming. Naturally we specialise in APL.68000, but we also run courses on APL*PLUS and can arrange training for most mainframe and mini APLs. Each course may be tailored to suit individual needs if required. In addition our subsidiary, Sahara Software runs complementary courses covering the Mirage Operating System, Word Processing and general 68000 Assembly Language Programming.

For details of dates, see the Course Diary on page 9. For further information or bookings please call David Eastwood on 01-922-8866.

Cocking and Drury Ltd

Demand for the second-generation APL*PLUS Mainframe system continues, and a recent flurry of installs has kept Dave Phillips' team very busy indeed. The popularity of APL*PLUS Mainframe is particularly due to its financial benefits in terms of machine and staff productivity. Small migration costs, plus provision of a second-generation interpreter without the need for conversion, lead customers to move quickly. Coupled with performance improvements through Fast-functions, faster algorithms and the Compiler, plus the storage, access speed and organisational benefits brought by Sharefile, all user departments get immediate value.

Meanwhile, the APL*PLUS II system for 80386 machines produces a steady stream of enquiries. Our customers like us selling it properly supported under licence; it gives them confidence for the future. The performance of this system is excellent; users are reporting that it provides faster response than their mainframe APL sessions.

The APL*PLUS II 'run-time' system allows easy distribution of PC applications to this high performance environment. A one-price site licence agreement is now also available for the APL*PLUS II system.

Dyadic Systems Ltd

Screen Manager. Dyadic's new screen manager (which went out on Beta test before Christmas) is nearing final release. Although the basic design is unchanged, feedback from users has resulted in numerous improvements being made. Field types include character, numeric and date. Field behaviour includes autotab, exit if modified, exit if invalid number, exit on entry to field, pocket-calculator input, and everything else you need to control user interaction in a sophisticated and efficient manner.

Oracle Interface. Dyalog has recently supplemented the IBM 'vendor-logo' Oracle interface with an interface to the full Oracle product on all systems that support Dyalog APL.

New APL Development Environment. Dyalog is in the early stages of designing a new environment in which the APL session, the editor, and the stack are all integrated in a seamless fashion. It will fully exploit the workstation environment of a bit-mapped colour screen but will remain fully compatible with existing APL conventions.

Dyalog APL for AIX and MS-DOS. Dyalog APL will shortly be available for AIX on PS/2s and mainframes in addition to the IBM 6150. Dyadic are currently working on an implementation of Dyalog APL that runs under MS-DOS on 386-based PCs. Performance of the DOS version is similar to that of Dyalog APL under Xenix. Under DOS, the component file system is slower than it is under Unix, but is highly competitive with other DOS APLs. Like Dyalog APL under Xenix, graphics will be supported by an interface to GSS*CGI.

A single source for APL*PLUS

APL*PLUS
PC



for PCs
and the PS2

APL*PLUS
UNIX



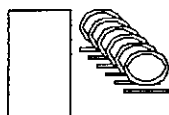
for UNIX
systems

APL*PLUS
PRO



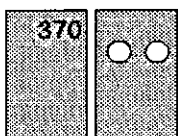
for the PS2
model 80

APL*PLUS
VMS

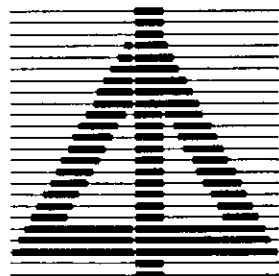


for the
VAX range

APL*PLUS
VM & MVS



for IBM
mainframes



COCKING & DRURY LTD.

THE APL PROFESSIONALS

180 Tottenham Court Road,
London W1P 9LE
Tel: 01 - 436 9481

**THE
EDUCATION
VECTOR**

January 1989

Edited by Alan Sykes

Reprinted from VECTOR Vol.5 No.3. VECTOR is the Quarterly Journal of the British APL Association; for more information about the British APL Association, contact the Secretary: Graham Parkhouse, Dept. of Mech. Eng., University of Surrey, GUILDFORD.

Contents

Editorial	Alan Sykes	18
Puzzle Corner	Alan Sykes	19
I-APL at ICME-6 in Budapest	Neil Bibby and John Durran	20
The Latest I-APL News	Anthony Camacho	22
Counting in APL (Part Two)	Alan Sykes	26

Dr Alan Sykes,
Dept of Management Science and Statistics,
University College of Swansea,
Singleton Park, SWANSEA SA2 8PP

The Education Vector

by Alan Sykes

Prior to the writing of this editorial, there has been no time for feedback from the first issue of Education Vector in its new form, so in the faint hope that I got it about right the first time round, this second issue is very much in the same mould.

Last year the British APL Association supported two APL-enthusiastic educationalists John Durran and Neil Bibby to attend the Sixth International Congress on Mathematical Education held in Budapest. Neil's report on their experiences there makes interesting reading. (I wonder whether my attempts at writing APL appear elitist!)

Neil mentions in his letter the need to talk about and demonstrate APL at the annual teachers' conferences in Britain. I hope to do just that this Easter by speaking at the Mathematical Association's 1989 conference to be held in Sheffield. This conference (which I helped to organise when it was at Swansea some six years ago) attracts some 300 of our (mainly) secondary school teachers from all over Britain, and I look forward to the opportunity of demonstrating live, some of the delights of using APL in Mathematics and Statistics. Also in my native South Yorkshire, Richard Webber has contacted us to say that he is prepared and keen to offer courses for teachers in APL in the locality. Perhaps as a result of my visit and talk, enough interest will be generated locally to make such a course viable.

Recently, the British APL Association has been asked to consider funding a project to construct an APL Statistics library. Preparatory work has commenced with the planning of a conference for academic statisticians in Britain, a growing number of whom use APL for teaching and research. I hope that the Royal Statistical Society will see fit to give its support to this venture and I look forward to an interesting interchange of ideas on programming statistical algorithms in APL.

Let me finish this second editorial by extending an invitation to you, the reader, to share your thoughts on APL and education (in its widest sense) with me and other readers by contributing to future issues. Please send all contributions to my address at the University College of Swansea (see front cover).

And finally (!) may I draw your attention to APL Puzzle Corner where I invite relative newcomers to APL to try to solve a simple problem using APL. Whatever your solution (it does not, I repeat NOT have to be a one-liner!) please send your solution direct to me. There is a £5 book-token prize for the most worthy solution received within four weeks of publication. Please indicate briefly in your submission the extent of your experience of APL to date.

Puzzle Corner

Professor A.P.Ellis looked anxiously at his watch and then at his APL terminal. There in front of him was the 50 by 8 matrix of the end-of-term marks, which had to be converted from percentages to grades in the next few minutes so that he could stick the results up on the notice board.

Muttering into his beard to the effect that he couldn't see why he should not give the students the actual percentages instead of grades A (70 or over), B (60 to 69), C (50 to 59) D (40 to 49), E (35 to 39) and F (34 or under) he suddenly realised how easy it was to do the whole operation of converting the number matrix of percentages to a character matrix of grades in APL.

Swiftly he typed at the keyboard, and before you could say 'quick basic' the required character matrix appeared on the screen. How do you think he did it?

Humble-pie Corner

Typesetting APL code is always a challenge; a couple of small glitches crept in to the last Education Vector, namely:

```
PAGE+2 3 4 20p1480
```

... half way down page 21. Also several places where there should have been a new-line to distinguish the bit you typed in from APL's response. An example would be:

```
      +/(16)°. =NOS
2 3 1 2 1 1
```

... on page 28. At least one sharp-eyed reader (aged 5) spotted that you actually get:

```
1 5
3 3
5 2
```

... as the mid-interval frequency table, not 6 3 2 as shown. At some point the vector NOS must have gained an extra element, as there were definitely only 10 numbers in the initial list!

Report on the Promotion of I-APL at ICME-6 Budapest 27th July - 3rd August 1988

by Neil Bibby and John Durran

The Sixth International Congress on Mathematical Education attracted some 2500 delegates. The committee of the British APL Association was right to have judged this an ideal opportunity to promote APL in general, and I-APL in particular, in the world of mathematics education, and we believe that the following report justifies our claim to some success in having done so.

During our time in Budapest we were able to give both practical demonstrations of APL and lectures on its structure and syntax. For the practical demonstrations, we had first to track down a suitable machine, and were initially rather frustrated in our efforts. However, the problem was solved by one of the Technical University's extremely helpful computer staff, whom one of us (NB) came across admittedly more by serendipity than by intention. In so doing, we met some Hungarian students of about sixth-form age who, despite their minimal English picked up a few APL idioms in a very short time. This was one more confirmation of the power of APL symbolism.

We thus not only secured the use of a PC-clone but also the assistance in our demonstrations of some new APL enthusiasts! These demonstrations were carried out on the first-floor gallery of the main building of the University, where many casual passers-by would stop, chat and try for themselves. Our demonstration was flagged by a poster which included Cocking and Drury's APL/BASIC comparison for the sum $1 + 2 + \dots + 10$ (by kind permission of Romilly Cocking) - extremely unfair to BASIC, of course, but then it's difficult not to be! The poster also advertised I-APL, and even when the order forms we had available had run out, many people were interestedly noting down the address.

Two of our most memorable demos were to two 'high-priests' of the British LOGO establishment, Trevor Fletcher and Celia Hoyles, of whom Trevor seemed the more enthusiastic. To complement these demonstrations, John Durran gave lectures on the structure of the language to some rather small, but nevertheless intensely interested audiences.

We were asked to include in this report some advice on how APL can best be promoted in the world of education. The following comments are those of one of us (NB).

- (a) The first issue I wish to emphasise is the problem of the elitist attitude of many APLers. If the language is to secure any degree of popularity in education, APLers will have to suppress their 'powerful one-liners' view of the world, adopt a much more sympathetic attitude to novices, and avoid dismissal of their initial forays into the language as trivial, for instance.
- (b) There is also an image problem: APL is often seen as a crank's language. It's often too easy for people to write-off APL because of the very style of its adherents. There is a fine line between perceived enthusiasm and perceived eccentricity.
- (c) As far as schools' use of APL is concerned I would recommend that I-APL users' groups be set up within the ASE, MA and ATM. As an initial step towards this, there ought regularly to be APL sessions, on the lines of those we held in Budapest, at the annual conferences of each of these three associations. (At the moment most teachers have never even heard of the language.) These sessions should be complemented by regular items in the various journals of these bodies.
- (d) There are various ways the I-APL product can be improved. The most urgent of these would be the supply of keyboard stickers for the character set, whether or not the keyboard layout is aligned with the standard. The second is that the literature accompanying the BBC version should be much more friendly than that which comes with the Clone version - apart from the generally very good but curiously laid-out Helzer book, the latter is not calculated to win adherents, and care should be taken not to frighten off newcomers by appearing elitist.

We hope that we have adequately fulfilled our brief; our impression from Budapest was that there is a lot of potential interest in the language which, suitably nurtured, should ensure a much more secure place for APL in the world of mathematics education than it has hitherto had. It was a pleasure to have been able to act on behalf of the BAA in this way, and we are both grateful for the financial support given to us to facilitate our trip.

Neil Bibby & John Durran
October 1988

I-APL NEWS

by Anthony Camacho

Version 1.1 of the I-APL interpreter is ready. Below is an order form.

I-APL is ready for the BBC Master and for the model B with 32 of sideways RAM.

I-APL is ready for RML Nimbus (PC, AX, VX and others) and Archimedes.

I-APL is available again for PC clones. The new version will run on many near-clones which would not work with version 1.0.

We said we would have version 1.1 ready by November 1988. We are very sorry for the delay. It is not Paul Chapman's fault. We felt it important to get the BBC version launched and couldn't print a combined manual until we were sure how the BBC version would work.

Everyone on the mailing list will be sent an order form and this newsletter.

BBC B and Master

You need 32K of sideways RAM to run I-APL on a BBC B. Type *APL <return> and an autoloader program will search for sideways RAM or shadow RAM and will load two ROM images from the disk and reset the machine. If your sideways RAM is an unusual type and this operation fails then you may have to use other methods to load the files IAPL0 and IAPL1 into ROM image. On a BBC B you should then set the mode you wish to use as in this machine it is unsafe to change mode from within APL. Then *APL will start I-APL running.

If you have EPROM writing equipment you may write the two files into EPROMs and insert them into two sideways ROM sockets. With these installed *APL <return> starts I-APL instantly.

We decided not to supply I-APL in EPROMs because the price of the EPROMs would be almost as high as the price of 32K sideways RAM. The many other benefits of sideways RAM persuaded us that very few people would be wise to choose the EPROMs. We will be sending out details of the 32K sideways RAM from Brainsoft, of which we have tried one sample. This only occupies a single ROM socket. N. B. I-APL Ltd. does not guarantee this or any other product.

Archimedes

The Archimedes implementation is as similar to the BBC as we can make it. It is also fast.

RML Nimbus

I-APL for PC clones now runs on the Nimbus PC and other Nimbus machines. Version 1.0 would not run even with the IBMULATOR program because it used hardware features to scroll the display and the Nimbus PC is not fully hardware compatible. Version 1.1 includes an option to use software scrolling and, though slower, this works on all the Nimbuses we tried on a visit to the RML office. Type IAPL ? <return> for a list of all the options.

New Books

The instruction manual has been revised and extended to cover the BBC B, Master, the Archimedes and PC clones and near-clones such as the RML Nimbus PC.

We regret that we don't yet have an introductory guide suitable for people who are both new to APL and not expert at their machine. We have done our best to put all the facts you need to know in the instruction manual or in the HELP files, but you may need to read them twice and study the examples carefully. The HELP function is included in workspaces such as USEFUL, PGRAPH, BBCFNS, DFILE, PGCANON and maybe others. It will read and display pages from the corresponding .HLP file. We will do our best to help if you really get stuck.

The second edition of the Tutorial is ready. A few improvements have been made.

"APL in Social Studies" is a completely new book by Tama Traberman who has taught APL in the USA. It uses simple and easy APL to do things such as drawing maps with characters. If you or your pupils find the Tutorial hard going this may be just right.

We still have some copies of "An APL Encyclopedia" by Garry Helzer in stock. Garry is working on the second edition which we will print as soon as it is needed.

Several schools now have introductory courses which use I-APL and we are hoping to document these, print them and supply them cheaply to anyone who wants to learn or teach using I-APL. Please let us know of any course you run of which we could print details. If you could write an introductory book or pamphlet we are always keen to publish suitable materials. We pay a small royalty.

New Workspaces

BBCFNS and DFILE on the BBC and Archimedes machines are new.

DFILE gives access to a single native file at a time in the same way as the PC DFILE workspace. DFILE contains a set of functions of the same names which do the same things as the PC functions. Note that the maximum file size is 32767 bytes.

BBCFNS contains an intelligent VDU function, SOUND, ENVELOPE, OSCIL, OSBYTE, OSWRCH, OSWORD and OSRDCH. The VDU function accepts a vector of VDU arguments which are converted into the correct one or two byte form and executed. A complete drawing can be displayed with a single VDU command. Please read the HELP screens and the manual carefully and save your work before trying anything; it is very easy to make a mistake and lose control of the computer.

FSCREEN is a new workspace for PC clones. The full screen functions are implemented by holding a buffer of the characters displayed on the screen. The WINDOW, WPUT and WGET functions specify a rectangular window on the screen (the character position of the top left corner and the number of rows and columns) and reading from or writing to such a window is very fast indeed. There is a HELP file with examples.

APL LESSONS - Zdenek V. Jizba has put together a disk full of lessons in APL which run on a PC. We hope to have these converted to run on BBC and Archimedes by the time this is published.

Graphics - Morten Kromberg has written a nice graph drawing workspace which uses PGRAPH to draw scaled graphs of any function. GRAFDEMO draws sine and cosine curves. For anyone who finds the facilities of PGRAPH hard to master this example may be just the ticket. N.B. For the best effect use the graphics on as fast a machine as possible and with a fully populated EGA board, so that you can display the 256 colours. On a CGA only four graphics colours can be used and at rather low resolution.

Order Form: I-APL Ltd February 1989

The address for queries is:

2 Blenheim Road

St Albans

Herts AL1 4NR

The address for orders is:

56 The Crescent, Milton

Weston-super-Mare

Avon BS22 8DU

Please supply:

Part	Item	(pounds sterling)		
		No copies	Price	Total
PC12	PC disk & manual (counts 1 item)	_____	£4	_____
BBC12	B/Master disk & manual (counts 1 item)	_____	£4	_____
A12	Archimedes disk & manual (counts 1 item)	_____	£4	_____
T3	Tutorial by Thomson & Alvord (1 item) (Paper 45pp)	_____	£2	_____
M2	I-APL PC BBC & Archimedes manual (1) (Paper about 64pp)	_____	£2	_____
E4	Encyclopedia by Helzer (1 item) (Paper vii+299pp)	_____	£5	_____
S5	APL in Social Studies by Traberman (1) (Paper 36pp)	_____	£2	_____
PLEASE	Postage 50 pence per item _____ items		x 0.50	_____
PAY BOTH				
POST AND	Packing charge per order		£1	1.00
PACKING				

(Optional donation _____) I enclose cheque to I-APL Ltd for: _____

We take no credit cards.

Name: _____

Full address: _____

_____Please send me a VAT invoice (tick) _ Vat is only chargeable at 15% on the disk (27p).
The above prices include VAT. Regd No 479 1634 13

Office use only

Counting in APL - Part Two

by Alan Sykes

Oh no, not more counting! Nothing like counting to send the reader to sleep. Let's arouse your interest by considering the following set of data consisting of a long list of the symbols B,G and |.

```
|BGBB|BBGBBGGG|GBBB|GB|GBBGGGGGG|GGGBGGB|GBBGG|GBBBGB|BB|GBBGGGG|B
BGG|BBG|GBGBBBBBB|BG|BGG|BGBG|BBGBBB|BGBGG|BBBBB|BGGGBGGG|GBGG|GG
BGBBB|GBBBGB|GBBGBGG|BBGGB|GBGGGBB|GGGBBBB|BGGGG|GB|GGGBBGG|BG|GB
GB|GBBGGBBB|BBGB|BGBB|BGBB|GBB|BGBBBB|BGBGB|BBGBBBB|GGG
GGBBB|BBGB|GBBB|BGG|GGGBBBGGG|B|BBGBBBB|GBBGBBGB|BGBGBBGGG|BB
BBGBBBB|BGBBGB|GBBGBGG|BBGG|BBGB|BBBGBBGG|GBBGG|GBBBB|GBBGBBGG
GGG|GBBBB|GGGG|GBGG|BBBGGGG|GBBB|BBB|BGBBGGGG|BGBBGBBBB|BGBB|GBG|B
BGBB|GBG|BBBB|BBG|GGGBBGGGGGG|GGGBBBGBB|BGBBBB|GGGBB|BBBGBBBB|GG
BBGBB|BGB|GBBBBBBB|GGGGGG|GGGGG|BGBB|BGBG|BBBBB|BBGBBB|BBGBGG|GGGB
GBBGBB|GB|BBGBB|GBGG|BBBBBGBBGG|BGBB|BGBBBB|BGBGGGG|
```

This (genuine) data represents a sequence of successive births in a maternity hospital over a number of days. You will have guessed that B stands for a boy and G for a girl; not so obvious is that | denotes the division between days. To work with data such as this we need it as an APL character vector. The easiest way to enter the data from the keyboard is by way of the simple function CHARINPUT.

```
∇ R←CHARINPUT;X
[1] R←''
[2] MORE:X+⍴ ⋄ +(0=p,X)/0 ⋄ R←R,X ⋄ +MORE
∇
```

This allows small chunks of the list to be entered (say a day at a time):

```
SEXORDER←CHARINPUT
|BGBB|
BBGBBGGG|
.
.
BGBGGGG|
```

Each new chunk is appended to the characters previously entered, and entry stops when a blank line is entered (i.e. RETURN is pressed with no prior characters).

Try entering some (not necessarily all) of the data. If all the data is entered as one long character vector of length 648, then the results of my previous article on counting in APL applied to SEXORDER produce some interesting preliminary results. For example

```
+/'BG|'∘.=SEXORDER
283 267 98
```

shows us that 97 days are recorded, and that in total there are 283 boy-births and 267 girl-births. The proportion of boy-births is therefore $283 \div 283 + 267$ or 51.4%. (Incidentally, this could have been calculated directly using the expression $\div 1 + \div / + 'GB' \circ . = \text{SEXORDER.}$)

Before we investigate the data in more detail, let me quickly review some more operations on vectors that we need, and to do this I shall use the vector:

```
X←2 1 3 5 9 2 4
```

We know already that the elements of X may be summed using the APL expression $+ / X$. The symbol $/$ is an APL operator, that may be combined with $+$ (or other functions such as $\times \div - \star$ and the comparison functions $< \leq = \geq > \neq$ and others) to produce another function. The scan operator $\backslash$ can also be used similarly with interesting effect. For example with $+$ we get cumulative sums.

```
+ \ X
2 3 6 11 20 22 26
```

Very often it is convenient to be able to take some of the elements of a vector either by choosing elements from the beginning or the end, or by dropping elements from the beginning or end. This is achieved by using $\uparrow$ to 'take' elements, or $\downarrow$ to 'drop' elements. Thus:

```
2 \ X
2 1
```

```
3 \ X
5 9 2 4
```

If a negative number is used, then the operation of taking or dropping is performed on the end of the vector.

```
-1 \ X
2 1 3 5 9 2
-2 \ X
2 4
```

Finally, we can select elements from a vector according to a boolean vector (logical recipe) using the replicate function $/$ preceded by a boolean vector.

```
(X≠5)/X
2 1 3 9 2 4
(X=2)/1ρX
1 6
```

(because $1\rho X$ is the vector 1 2 3 4 5 6 7).

So back to our next task in counting - to count the total number of births for each day. To illustrate the method I shall use the first three days of the data, namely `S+'|BGBB|BGBBGGG|GBBB|'`. Applying the tools just illustrated, we can calculate the vector `N` which tells us where each '|' is.

```

      N+(S='|')/lpS
1 6 15 20

```

(In this last expression `'|'` merely outputs the results to the screen as well as assigning them to `N`.)

What is required from `N` is the successive differences (6-1), (15-6), and (20-15). The easiest way to achieve them is to difference the two vectors created by dropping one element from each of the front and the back:

```

      (1+N)-~1+N
5 9 5

```

Since these differences include one |, the frequencies required are one less and so the frequencies for the first three days are given by:

```

      ~1+(1+N)-~1+N+(S='|')/lpS
4 8 4

```

Applying to the complete data `SEXORDER`, we can obtain the vector `DAILYFREQ` of daily frequencies:

```

DAILYFREQ+~1+(1+N)-~1+N+(SEXORDER='|')/lpSEXORDER

```

Using the functions `FREQTAB` and `BAR` introduced last time, we may process the frequency vector as follows:

```

      FT+~.5 1 FREQTAB DAILYFREQ
0 1
1 1
2 6
3 8
4 20
5 15
6 10
7 14
8 7
9 9
10 3
11 3

```

```

      BAR FT
0 | 
1 | 
2 | 
3 | 
4 | 
5 | 
6 | 
7 | 
8 | 
9 | 
10| 
11| 

```

Now we can see that the most likely number of births in a day is 4, but the average is 5.67 (calculated from DAILYFREQ using $(+/\text{DAILYFREQ}) \div \rho \text{DAILYFREQ}$ or from FT using $(+/\times/\text{FT}) \div +/\text{FT}[:,2]$).

Slightly more demanding, is to determine the frequency of boy-births for each day. With our small sample S introduced earlier, the secret is to calculate cumulative sums of boy-births:

```

      +CB++\S='B'
0 1 1 2 3 3 4 5 5 6 7 7 7 7 7 8 9 10 10

```

The vector $N+(S='|')/\rho S$ used before tells us where each | is. Indexing CB on N yields M, the cumulative number of boys for each day:

```

      +N+(S='|')/\rho S
1 6 15 20
      +M+(+\S='B')[N]
0 3 7 10

```

M can then be differenced as before to obtain the daily number of boy births. Here is the whole calculation pieced together though it is no sin to do it in separate stages!

```

      (1+M)-1+M+(+\S='B')[N+(S='|')/\rho S]
3 4 3

```

Using this expression with S replaced by SEXORDER and assigning the result to BOYFREQ, we can calculate the boy-birth daily frequencies, and hence by subtracting this from DAILYFREQ, the vector GIRLFREQ can also be calculated. Here are the bar charts, first for BOYFREQ and secondly for GIRLFREQ:

```

0 | 0000
1 | 0000000000000000
2 | 0000000000000000000000000000
3 | 0000000000000000000000000000
4 | 0000000000000000
5 | 000000
6 | 0000
7 | 00

0 | 0000
1 | 000000000000000000000000
2 | 0000000000000000000000000000
3 | 000000000000
4 | 00000000
5 | 0000000
6 | 00000
7 | 0
8 | 0
9 | 0

```

So much for our interesting example. Let's now see how these same techniques may be used to good effect to provide more efficient APL methods for counting.

Suppose x is the vector $x = [2 \ 1 \ 2 \ 2 \ 1 \ 3 \ 4 \ 3 \ 2 \ 4]$

Then it may be ordered (increasingly) by indexing X on the vector $\uparrow X$. ($\uparrow$ stands for 'upgrade', and it is the vector whose first element is the index (position in X) of the smallest element of X , whose second element is the position of the next smallest element in X , and so on.)

$$\begin{array}{cccccccccc} & & \uparrow X & & & & & & & \\ 2 & 5 & 1 & 3 & 4 & 9 & 6 & 8 & 7 & 10 \\ & & X[\uparrow X] & & & & & & & \\ 1 & 1 & 2 & 2 & 2 & 2 & 3 & 3 & 4 & 4 \end{array}$$

If we rotate this vector one place to the left as follows:

$$\begin{array}{ccccccc} & X+X[\downarrow X] \\ & \downarrow \\ 1\phi X \\ 1 & 2 & 2 & 2 & 2 & 3 & 3 & 4 & 4 & 1 \end{array}$$

then the indicator vector $1 \mapsto (1 \phi X) \cdot X$ has 1's in the positions of the last occurrence of each of the values occurring.

$$\begin{array}{ccccccc} & \square + I \leftarrow (I \oplus X) \neq X \\ 0 & 1 & 0 & 0 & 0 & 1 & 0 & 1 & 0 & 1 \\ & \square + N \leftarrow I / l p I \\ 2 & 6 & 8 & 10 \end{array}$$

By differencing the vector 2 6 8 10 with 0 2 6 8, we can deduce that there are 2 1's, 4 2's, 2 3's and 2 4's.

```

      N-0, -1+N
2 4 2 2

```

One slight technical problem with this approach is that if all the elements of the counted vector are the same, then the above procedure must be modified. The technique illustrated is used in the following two functions to count elements of a numeric vector. COUNT returns the frequencies in the right argument of each element of the left argument:

```

      1 2 3 COUNT 3 1 1 2 1 5
3 1 1

```

COUNTALL on the other hand returns a frequency table whose first column is a list of unique values, whilst the second column gives the associated frequencies:

```

      COUNTALL 3 1 1 2 1 5
1 3
2 1
3 1
5 1

```

```

▽ FREQ+T COUNT X;I;N;V;F
[1] X+X[AX] ◇ V+,X[1] ◇ F+PX
[2] +(X[1]=X[px])/SETFREQ
[3] I+{(1PX)≠X ◇ F+N-0, -1+N+I/lpI
[4] V+I/X
[5] SETFREQ:FREQ+(F,0)[VIT]
▽

```

```

▽TAB+COUNTALL X;I;N
[1] X+X[AX] ◇ R+ 1 2 p(1+X),pX
[2] +(X[1]=X[px])/0
[3] I+{(1PX)≠X ◇ N+I/lpI
[4] TAB+{(I/X),[1.5]N-0, -1+N
▽

```

As you can see, the new method is somewhat more involved than that used previously. For large sets of data however, the method used here is much quicker, because it does not involve the construction and row summation of a very large matrix. In addition, the techniques described here may be adapted to deal with character arrays with only minor changes.

This is because Δ or upgrade will work on a character matrix as long as the required ordering of the characters is specified. For most purposes, the order of the characters as contained in the atomic vector $\square AV$ is appropriate. For example, let M be the 4 by 4 matrix given by:

```

M←4 4p'FREDALGYPETEALAN'
M
FRED
ALGY
PETE
ALAN

```

Then `⊔AV⊔M` returns the vector 4 2 1 3, so that `M[⊔AV⊔M;]` lists the names in alphabetical order.

```

M[⊔AV⊔M;]
ALAN
ALGY
FRED
PETE

```

(N.B. the semi-colon is necessary as we wish to print the rows of the matrix out in the order determined by `⊔AV⊔M`.)

Given this, the functions `COUNT` and `COUNTALL` may be suitably altered to deal with character lists. But perhaps that's best left for another time!

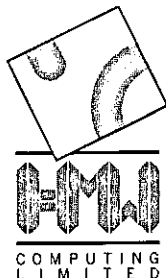
British APL Association News

by Graham Parkhouse

Committee meetings of the British APL Association happen six times a year at the British Computer Society's Headquarters near Oxford Circus. They often occupy the morning of the day we hold an afternoon meeting at the Royal Over-Seas League, to minimise travelling. The starting time has just been moved from 9.30 to 10.00 to give those of us coming from a distance a chance to be in on the start of the meeting, when we begin going through the outstanding actions list. By the January Actions List we had generated 48 actions for the year, 28 of which had already been done. Some of the work we do is routine, like organising meetings, editing VECTOR, writing EDUCATION VECTOR, and never gets mentioned on the actions list. Would that everything went as smoothly! Some actions seem permanently outstanding, but then when I remember that we are all volunteers struggling to earn our living outside the BAA, I begin to be less critical of our shortcomings. But some of you have been wondering what has gone wrong with the association. I was recently asked "Has the BAA gone to the dogs?" in a letter from a member, having not received his November VECTOR by February, and losing 1-5 to Phil Goacher's answerphone during December and January. Yes, we still have problems.

But we are getting on top of them. Alison Chatterton's name and address now features in VECTOR, and as our paid administrator is just getting into top gear. She began in February to manage the membership list. Please direct queries regarding subscriptions to her.

Statisticians recognised the usefulness of APL from its very early days, and there are now many universities in the UK where APL is a vital tool in the statistics department. For some years the APL Statistics Users Group have been considering developing a library of APL statistical functions. Following a meeting of the Group in October, when our Projects Officer, David Eastwood, was present, Tony O'Hagan from Warwick University forwarded a project proposal to David for the BAA Committee to consider: the project, to be financially controlled by the BAA, should result in a documented set of functions useful to the statistics community. The Committee has agreed to accept this as a project, and a conference is being planned for the autumn as a first meeting for contributors.

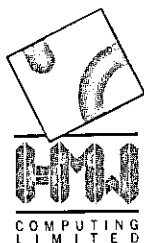


SOLUTIONS FOR THE CITY

We are a specialist firm of consultants with a prestigious list of clients including major City institutions.

For one of these clients we have recently developed a foreign exchange dealing system with the following features:— deal entry, deal enquiry, multi-currency position keeping, basket currency calculators, arbitrage calculators and ticket production.

If you are interested in foreign exchange dealing, treasury operations, portfolio management, commodity broking or similar areas we would be delighted to discuss how we could help. Call Stan Wilkinson, HMW COMPUTING LIMITED Hamilton House 1 Temple Avenue London EC4Y 0HA Telephone 01-353 4212



EXCITING POSITIONS AT THE LEADING EDGE

With top City institutions and leading Blue chip companies amongst our client list we have a lot to offer experienced APL programmers.

Call Stan Wilkinson to discuss your future.

HMW COMPUTING LIMITED
Hamilton House 1 Temple Avenue
London EC4Y 0HA Telephone 01-353 4212

1989 British Design Award to 'Resolve' from Metapraxis

notes by David Preedy

RESOLVE, the world's first Executive Information System, has won one of the 1989 British Design Awards. RESOLVE has been developed by the management consultancy Metapraxis, based at Kingston-on-Thames, and is written in APL*PLUS/PC.

Executive Information Systems are designed to provide directors and senior managers of large organisations with personal access to their corporate databases, and APL has played a major role in their development. Besides RESOLVE, the British Airways system, called AIMS, is also written in APL, as was one of the earlier, trail-blazing systems at Imperial Group, as described in VECTOR 1.2.

One of the main features of RESOLVE is its wide range of colour graphics formats, each carefully designed to pin-point issues of business importance. It also has the ability to provide business forecasts and to detect exceptional performance so that appropriate action can be taken before a crisis develops. All of these facilities can be called up within seconds at a Board meeting using an infra-red keypad which can be passed to whoever holds the floor.

RESOLVE was launched in Spring 1985, but since then has evolved incorporating feedback from an active client base in over seventy major organisations, including health authorities, public agencies and third-world governments, as well as retailers, manufacturing corporations, banks and insurance companies. The backbone of the development team has been provided by Martyn Adams, Mark Bassett and David Preedy, but over the years about fifteen other people within Metapraxis have also been directly involved.

Over that time, Metapraxis has grown from a company of eight employees to over 50, with an annual turnover of 2.3 million. Last year it launched an American subsidiary based in Manhattan, and its clients now extend to Europe, Hong Kong and West Africa.

Metapraxis' Development Director, David Preedy, said "The use of APL has been fundamental to the design philosophy underlying RESOLVE. The original choice of APL as the development language reflected the personal expertise of the original team and the experience of previous developments in the related areas of Decision Support Systems and Graphics. Since then, however, that decision has been vindicated by the power it gives us to incorporate enhancements arising from our clients' experience.

"A new idea is initially provided experimentally, then incorporated as a customised feature for use by a small group of users. Depending on its acceptance in the business environment and how generally it will be useful, it may then be developed as a standard feature. APL has enabled us to undertake this evolutionary development without having a different version of the system for each different client."

British Design Awards are given annually by the Design Council and are recognised as the premier design accolade in Britain. This year's winning manufacturers and designers will be presented with their Awards by HRH The Duke of Edinburgh at a special ceremony in May. An exhibition of all the winning products is open to the public at the London Design Centre in the Haymarket, off Piccadilly Circus, until March 31st. Altogether 16 products have been chosen for the 1989 Awards, including three computer software products.

Results of a Survey Carried Out at APLication by APL People

notes by Jill Moss

For APLication last year we devised a survey for people to fill in. Although it was by no means constructed according to orthodox statistical procedures, we hoped that our little questionnaire would tell us something about the people attending the Conference, and thereby perhaps provide us with a snap-shot of a 'typical' APLer (if there is such a thing). We have finally got round to analysing the results and hope that readers of VECTOR will find them as fascinating as we did!

Our first question asked which types of APL people used. IBM will be pleased to learn that APL2 won in the popularity stakes (I can assure readers that this was not because we had a lot of people from IBM fill in the forms!). Most people, however, said they used more than one type of APL. After APL2, it was pretty close between VS APL and APL*Plus (mainframe and PC). (Sorry Dyadic and IP Sharp!)

Our second question asked how many people the respondent knew personally who write APL programs. Answers here varied from "NIL" (from Richard Webber up at Doncaster Metropolitan Institute of Higher Education) to "HUNDREDS" (Dick Bowman exaggerating, of course!). In fact, the highest 'score' goes again to IBM, where one respondent claimed to know 200 other APL programmers.

A totally statistically meaningless question (just to really get up the nose of all those statisticians) asked people to hazard a guess as to how many people they thought write APL programs in the UK, Europe and the World (we left out the Universe). Surprisingly, most people said about the same - between 500 and 2000 in the UK, between 5000 and 10,000 in Europe and 10,000 to 50,000 world-wide. (There were some silly answers of

course, which I have conveniently omitted). Has anybody got any real figures here? It would be interesting to know.

We then had a really sexist question - 'How many women do you know who write APL programs?' We included this because we suspected that the majority of APLers are male and we wanted to check this out. However, although 18% said they didn't know any, and 25% only knew one, surprisingly 21% said they knew more than 10 women who write APL programs, so perhaps there are more of them about than we suspected! Despite the fact that 90% of our respondents were male, over half of them said they worked with women who were writing APL programs, which I find very encouraging (so why weren't more of them at the Conference then, or maybe they just don't like filling in questionnaires?).

Our next question asked how many days per month the respondent spent writing APL programs. Of the replies, exactly half said that they spent more than 10 days per month, although almost 18% said that they spent no more than 2 days per month actually writing APL programs, and 3 people said they did no programming at all in APL (one of these is a Manager, one an Economist and I don't know what the third does but I suspect he is also a Manager as he later put down that he had learnt APL over five years ago).

Although the vast majority of the people who completed our questionnaire were employed by very large companies, half said that, where they worked, there were no more than five other people writing APL programs. This confirms our suspicions that APL tends to be found in 'small pockets' within large organisations. These organisations included companies in the Banking, Insurance, Manufacturing and Energy Industries, with colleges of further education being quite well represented. Surprisingly, there were not many people from government or local authorities (perhaps because their budgets are too tight to allow employees to attend APL conferences?).

A telling question was 'When did you first learn APL?'. This was designed to establish the 'age' of the APL population and to find out whether there is much new blood coming into the language. A staggering 71% learnt APL over five years ago, with only 10% having come into it within the past three years. (One person - the Economist - put that he had never learnt it. It would be interesting to discover if he found the conference useful and whether he was inspired to learn APL afterwards.)

Most people had been on a course to learn, and the vast majority of them (88%) said that they thought the course worthwhile (a pat on the back for the C & D trainers here, I suspect!).

Slightly more than a third write APL programs to produce results which are used by other people, a third produce systems which are used by non-programming staff, while just over a quarter produce systems and programs which are used by other APL

programmers. Unfortunately, 79% said they did not have any budgetary authority (were we preaching to the converted at Canterbury and perhaps more emphasis should be placed on attracting Managers and others who hold the purse-strings if the use of APL is to increase?)

More than 50% said that they thought their company used contract programmers occasionally, but by far the most telling (and depressing) indicator of the extent to which APL usage is expanding came with the results of our last couple of questions, concerning the recruitment of permanent staff. We asked 'How many people have been recruited by your company/department over the past year?' and 'Of these, how many were replacements for people who had left, and how many were new vacancies?'. Almost 60% said that no APLers had been recruited where they worked over the past 12 months, and a total of over 98% said that the number recruited was between NIL and 2. Of these, over half of all new recruits were to replace people who had left. Not very good news for those of us keen to see an expansion in the use of the language.

So what picture does this give us of our typical APLer? I would say, from these results, our typical APLer is probably male, works within a large organisation employing a handful of other people writing APL programs (some of whom might be female!), learnt APL more than five years ago and uses APL2, VS APL and maybe APL*Plus to write programs, the results of which are used by other people within the company. He probably found his job (via APL People, of course!) when the person who was there before left. In fact, what we knew all along! Apologies to all those who don't fit this stereotype, which is probably most of you reading this article.

APL Product Guide

Compiled by Cathy Dargue

VECTOR's exclusive APL Product Guide aims to provide readers with useful information about sources of APL hardware, software and services. We welcome any comments readers may have on its usefulness and any suggestions for improvements.

We do depend on the alacrity of suppliers to keep us informed about their products so that we can update the Guide for each issue of VECTOR. Any suppliers who are not included in the Guide should contact me to get their free entry - see address below.

We reserve the right to edit material supplied for reasons of space or to ensure a fair market coverage.

The listings are not restricted to UK companies and international suppliers are welcome to take advantage of these pages. Where no UK distributor has yet been appointed, the vendor should indicate whether this is imminent or whether approaches for representation by existing companies are welcomed.

For convenience to readers, the product list has been divided into the following groups:

- * Complete APL Systems (Hardware & Software)
- * APL Timesharing Services
- * APL Interpreters
- * APL Visual Display Units
- * APL character set printers
- * APL-based packages
- * APL Consultancy
- * APL Training Courses
- * Other services
- * Vendor addresses

Every effort has been made to avoid errors in these listings but no responsibility can be taken by the working group for mistakes or omissions.

Note: 'poa' indicates 'price on application'.

All contributions to the APL Product Guide should be sent to:

Cathy Dargue,
60 Downhall Ley,
BUNTINGFORD,
Herts SG9 9TL.
Tel: 0895-31313 ext 3038

Recent APL Product Announcements

Exchange of Business between Cocking and Drury and Mercia Software

As from January 1989, Mercia Software have transferred all their existing APL*PLUS business to Cocking & Drury. Cocking and Drury have transferred their Statgraphics business to Mercia.

This exchange was undertaken to consolidate both organisations' positions in the respective marketplaces - Cocking and Drury preferring to concentrate efforts in the APL systems/consultancy/training sphere, and Mercia specialising in applications software for the Statistics, Maths and Operational Research community. As a result, both companies are now in a stronger position to market and support these two diverse STSC product lines.

Existing Mercia APL*PLUS customers can now obtain support, upgrades etc. direct from Cocking and Drury, and vice versa from Cocking and Drury Statgraphics customers.

Both companies will continue to supply and support their own large corporate accounts for both APL*PLUS and Statgraphics.

Statgraphics Version 3

Statgraphics v.3, written in APL, allows the user to analyse larger data sets without losing any flexibility in graphics, editing, or data manipulation capabilities. Users can now save graphics in the popular HPGL format to give them greater choice with output facilities.

Version 3 has also tackled the ideas of SPC and QC with more vigour, providing a new range of process capability charts and gauge repeatability and reproducibility analyses, along with the full range of quality control procedures already provided.

"A comprehensive stats/forecasting/decision-making package with integrated graphics and a user-friendly, user-supporting environment." PC User, February 1989.

COMPLETE APL SYSTEMS

COMPANY	PRODUCT	PRICES(£)	DETAILS
Dyadic	APL Coprocessor	3,500+	32-bit coprocessor board for IBM PC. NS32000 cpu with FPP, up to 4Mb RAM, 16Mb virtual memory. Software includes Unix V.2, Dyalog APL, graphics support, DOS interface. Provides multi-user Unix/DOS environment.
	IBM 6150	15,000+	Multi-user Dyalog APL system with Fast 32-bit RISC processor, FPP, up to 8Mb RAM, 210Mb Disk, 16 users. Interface to SQL, graphics and APL support for standard IBM peripherals.
	Altos 3068	25,000+	Multi-user Dyalog APL system with MC68020 cpu & MC68881 FPP. Also features a LAN which supports IBM PCs as Dyalog APL terminals.
	Sun 3	15,000+	Multi-user Dyalog APL systems which can be configured as a network of workstations and/or a traditional time-sharing cpu. With its 25MHZ 68020 cpu, the Sun 3/200 is the fastest APL microcomputer on the market.
Inner Product M.B.T.	IBM PC	2,000-6,000	IBM PCs supplied for turnkey applications
	MBT Series 10	poa	UNIX/68010 based multi-user APL system
MetaTechnics	TORCH	poa	68000/Z80 multiprocessor
	-	poa	Details on application -- IBM PC compatible
MicroAPL	Aurora	20,000+	Multi-user APL computer using 68020 CPU. Std. configuration 1 Mb RAM, 16 RS232 ports, 68 Mb hard disc, 720K diskette
	Spectrum	7,000+	Expandable multi-user APL computer using Motorola 68000. Std. configuration 1 Mb RAM, 12/36 Mb disc, 12 ports.
	Spectrum/386	2,000+	80386-based PC to run APL*PLUS II. Fully expandable.
	Atari 1040ST	799	1 Mb Mono/Colour System, includes 1 Mb disc drive & mains transformer built into Console.

APL TIMESHARING SERVICES

COMPANY	PRODUCT	PRICES(£)	DETAILS
Mercia	APL*PLUS	poa	STSC's Mainframe Service - MAILBOX etc.
I.P. Sharp	SHARP APL	poa	International Network application systems and public databases.
Uniware	APL*PLUS	call	STSC's mainframe service

APL INTERPRETERS

COMPANY	PRODUCT	PRICES(£)	DETAILS
APL Software	Dyalog APL	1,000-8,000	See Dyadic Systems entry
Cocking/Drury	APL*PLUS/PC Rel 7	475	STSC's full featured APL for IBM PC, PC/AT and PS/2. Now with virtual workspace, quad-MF, GSS graphics, Network and VGA support.
	Run-time	poa	Closed version of APL*PLUS/PC which prevents user exposure to APL.
	APL*PLUS II	poa	site-license and runtime versions available
	APL*PLUS UNX	poa	STSC's 2nd gen hi-performance APL for IBM PC/RT, Sun, DEC, HP and major Unix computers and workstations.
	APL*PLUS VMS	poa	STSC's 2nd generation APL for DEC VAX computers running under VMS. Comparable in feature to APL*PLUS II system, APL*PLUS Unx, and APL*PLUS mainframe.
	APL*PLUS mainframe	poa	STSC's 2nd gen APL interpreter. Enhances VS APL with component files, nested arrays, formatting, user command processor, fastfns and other productivity features. For VM/CMS, MVS/TSO: offers simple upgrade from VS APL.
Dyadic	Dyalog APL	795-10,000	2nd gen. APL for UNIX systems, e.g. IBM 6150, Sun, Vax, NCR, HP9000, AT&T, Altos, Apollo, Whitechapel, Sperry, etc.
Gen. Software	APL*MYRIADE	poa	Runs on Texas Instruments TI990 range.

IBM UK	IBM PC APL	poa	Event-handling & APs for full-screen I/O disks, diskettes, asynch. comms.
IBM USA	IBM PC APL2	500	APL2 for the IBM PC. Program 5799-PGG/1, PRPQ number RJ-0411, USA distribution only - call your IBM representative for more details.
Inner Product	VIZ::APL	250-350	8-bit Zilog Z-80 CP/M
	APL*PLUS/PC	600	See under Cocking & Drury
M.B.T.	Dyalog APL	poa	See Dyadic Systems entry
	MBTAPL	poa	Enhanced Dyalog APL for MBT hardware.
	VIZ::APL	poa	Customized for TORCH hardware
MetaTechnics	APL*PLUS Rel 6	475	Discount on quantity.
MicroAPL	APL68000	1,000-2,000	Full implementation with component files, error trapping etc. for SPECTRUM, HP300, SUN, NCR etc.
	QL/APL (keyword)	87	Full keyword APL for QL with many extra features.
	QL/APL (APL chars)	87	VSAPL compatible APL for QL with many extra features.
	APL68000 Apple Macintosh	87	
	APL68000 Apple MacII	260	
	APL68000 Amiga	87	Full APL interpreters with support for windows, mouse, graphics etc.
	APL68000 for Atari ST	87	
	APL68000 for IBM-PC	995	
	APL*PLUS/PC - REL 7	425	
Portable	PortAPL	\$195	IBM PC Software
		\$275	Mackintosh
		\$2,995	DEC VAX
I.P. Sharp	APL-LAB/PC + Q pack	480	STSC APL plus object packaging for the PC
	Upgrade V6 to V7	120	
	Upgrade any to V7	200	
	RTS	70	Closed version of APL-LAB/PC
	Sharp APL/PCX V1	1,650	For IBM XT/AT
	Sharp APL/PC V2	40	shareware for IBM PC/XT
	" diskettes only	10	
	" manuals only	30	
		1,000+	For IBM mainframes
	Sharp APL/PC	325	For IBM PC or PC/XT
Uniware	APL*PLUS/PC	495	STSC's full feature APL for IBM PC/XT/AT, Compaq, Olivetti
	Run-Time	call	Closed version of APL*PLUS/PC which prevents user exposure to APL
	APL*PLUS/UNIX	call	STSC's full feature APL for UNIX based computers.
	PortaAPL	280	PORTABLE SOFTWARE's APL for APPLE MACINTOSH.
		2,995	PORTABLE SOFTWARE's APL for the DEC VAX.

APL VISUAL DISPLAY UNITS

COMPANY	PRODUCT	PRICES (£)	DETAILS
Dyadic	Lynwood j300	1,560	Monochrome ANSI 3.64 APL vdu, 15-inch high quality screen, Tek graphics, local macro keys.
	Lynwood j500	2,295	Colour ANSI 3.64 APL vdu, 15-inch high quality screen, Tek graphics, local macro keys.
	IBM 3163	791	Low-cost Monochrome APL vdu. Supports downloaded Dyalog APL font.
	IBM 3164	1,093	Low-cost Colour APL vdu. Supports downloaded Dyalog APL font.
Farnell	Tandberg TDV 2221	995	Ergonomic design APL terminal, 50-19200 baud, 1518i/f5 anti-reex screen, low profile keyboard
	Tandberg TDV 2271	1,195	Combined APL/ANSI ergonomic terminal as above.
Gen. Software	Mellordata	400	Second-hand Elite 3045A

M.B.T.	various		Contact MBT for details
Meta Technics	IBM EGA compatible	299	Emulates EGA & Hercules, Half Card
MicroAPL	Insight VDT-1	795	Inexpensive APL VDU
	Insight GDT-1	1,450	With monochrome graphics
	Concept201	1,295	APL VDU with 8 page memory
	Concept 201G	1,650	Graphics VDU
Shandell	HDS2010	1,215	ANSI3.64 DEC VT52/100/220 compatible. 15" tilt/swivel screen, low profile keyboard 8 page memory, windows, viewports, 80/132 columns, full overstrike, 2 or 3 comms, ports, 55 PF keys, NVM storage.
	HDS2010G/GX	1495+	As above plus Tektronix 4014, Retrographics VT640/D0640 and Visual 500 compatible. 1024 x 390 or 1024 x 780 resolution.
Tektronix	4114B	13,500+	19" D.V.S.T.:Graphics: 3120 x 4096 displayable; Intelligent: up to 800K memory; APL keyboard (option 4E)
	4125	21,550+	19" 2D colour graphics; Workstation (1280 x 1024);Intelligent: up to 800K memory; APL keyboard (mod AP)
	4128	26,822+	As 4125 plus 3D wireframe

APL PRINTERS

COMPANY	PRODUCT	PRICES (£)	DETAILS
Datatrade	Datasouth DS180+	1,295	180 cps matrix printer with 4K buffer, 9 x 7 dot matrix and APL option.
	Datasouth DS220	1,695	Letter quality; graphics capability, APL option (both available with IBM Twinex or Coax interface).
Dyadic	IBM 4201 Proprinter	poa	100, 200, 40(nlg) cps, matrix printer, with graphics. Supports downloaded Dyalog APL font.
	Toshiba P351	poa	24 pin high-quality matrix printer 100 cps letter quality, 192 cps draft.
Inner Product	Epson FX80	500	Soft char. set, 160 cps, 80 column
	Anadex 9620	1,150	200 cps., 132 col., tractor feed
	Siemens PT88	620	180 cps., 80 col., silent
	TGC Starwriter	1,180	40 cps., letter quality
M.B.T.	Facit 4565	poa	40 cps letter-quality
	Facit 4510/11/12	poa	Matrix printers
MetaTechnics	Quen-data	295	Low-cost APL Daisy-wheel printer
MicroAPL	Datasouth DS180+	1,195	See Datatrade entry
	Philips GP480	2,137	Matrix printer with letter & draft quality and APL (480 cps).
	Qume Letterpro20	549	APL/ASCII Daisy-wheel printer

APL PACKAGES

COMPANY	PRODUCT	PRICES (£)	DETAILS
APL-385	APL-385	50(PC),125(mf)	including ...
	FSM-385		Screen development
	DRAW-385		Screen design
	DB-385		Relational W.S.
	GEN-385		Miscellaneous Utilities
APL Software Ltd (mainframe)	AFM/AP	11,035	Interprocess Software for VM/CMS & MVS/TSO.
	- Keyed Access	2,650	Component File Management System (VSAPL/APL2)
	- Interactive Link	1,325	
	- Mail Exchange	2,650	
	CALL/AP	4,030	Non-APL program execution (VSAPL/APL2)
	APLPRINT	2,205	Output to high speed line printer or 328x devices (VSAPL/APL2)

	ENHANCED FORMAT	2,205	Extends Format operator to full "Quad-FMT" status (VSAPL/APL2)
	ISP	750	Input and Output Stack Processors for manipulating terminal I/O
	OSP	2,205	with facilities for Error Trapping (VSAPL)
	DISPLAY CAPTURE	poa	Allows terminal output to be collected and held for retrieval by an APL function (APL2)
	UCF	poa	User Communication Facility for data transfer between users (APL2)
	RDS	poa	Relation Data Base System
	PANEL	poa	Fullscreen management system
	PFS	poa	Program File System - APL Systems development aid
	IPLS	poa	Project Management System
	REGGPAK	poa	Regression Analysis Package
(microcomputer)	POWERTOOLS	295	Assembler written replacement function for commonly used CPU-consuming APL functions, includes a Forms Processor.
	REGGPAK	poa	Regression Analysis Package
	RDS	990	Relational Database System
Beta-plan	BETA-FONT	poa	Multiple font PC character generator. Dealers required for non-Scandinavian countries.
APL IMPETUS	Impetus	poa	Hierarchical Planning System
Butel	Merlin	5,000	Mainframe APL spreadsheet runs under VM/CMS, TSO, VSPC
	Merlin/PC	poa	Version for APL*PLUS/PC
Cocking/Drury (for VSAPL)	E'MENTS & SHAREFILE	poa	Component files, quad- functions & nested arrays for IBM VSAPL under VM/CMS & MVS/TSO
	COMPILER	poa	First APL compiler. Available with APL*PLUS enhancements and Sharefile under VM/CMS & MVS/TSO
	FILEPRINT	poa	Print APL component files
	FILECONVERT	poa	Convert non-APL files to APL files
	FILEMANAGER	poa	Extends APL primitives to database management
	TOOLS + UTILITIES	poa	APL Software development tools
	DATAPORT	poa	Powerful Information Centre spreadsheet incorporating data exchange between APL and FOCUS, IFPS, SAS, APL/DI, ADRSII, LOTUS123, VISICALC, MULTIPLAN, DIF files
(for APL2)	SHAREFILE/AP	poa	STSC's shared access component file system for APL2. Comparable to all APL*PLUS file systems: multi-user storage of APL2 arrays with efficient disk usage.
	FMT	poa	Full featured FMT for APL2
	WSDOC	poa	Workspace documentation utilities
	FILEMANAGER	poa	Extends APL primitives to database management
(microcomputer)	STATGRAPHICS Rel 2.6	645	Integrated Statistics and graphics on IBM PC, AT, PS/2 and compatibles
	Update 2 to 2.6	140	
	Update 1 to 2.6	330	
	APL*PLUS PC Tools	325	Utilities including: RAM disk, full screen data entry, menu input, report generation, file documentor, exception handling and games.
	IRMA Module	120	327x IRMA support.
	Fin & Stat. Library	350	Financial & statistical routines
	SPREADSHEET MGR	195	APL-based spreadsheet for APL*PLUS/PC. Cell arithmetic; transfers to ASCII, LOTUS
E&S	PROTOPAK		Packages for prototyping management information systems - consisting of: PC & mainframe modules
	RMS		Relational databases.
	AMS		Multi-dimensional arrays
	RAMS		Combined RMS & AMS.

	BMS		Dynamic financial modelling & forecasting
	FMS		Full-screen handler for APL*PLUS/PC. (AP124-based)
	CMS		Communications package.
	SOS	poa	Scheduled ordering and stock control.
FASTCODE	FASTCODE, MONITOR	\$239	Assembler written monitor for APL applications in APL*PLUS/PC
Gen. Software	PROPS	500+	Spreadsheet system for Product and/or Project Planning.
H.M.W.	INPUT	poa	Matrix manipulation package for data entry & report generation
	PRINTPAK	poa	Block printing for VM/CMS
	VIEWPAK	poa	AP124 Protocol emulator for IBM/PC
	4XTRA	poa	Front-end Foreign Exchange dealing / pos keeping
	Arbitrage	poa	Arbitrage modelling
	Basket	poa	Basket currency modelling
	Menu-Bar	poa	pull-down menu for APL*PLUS/PC
Inner Product	Viewcom	150	Control Viewdata from APL
	APL/dBASE II	150	Interface APL with dBase II
	APL/dBASE III	150	Interface APL with dBase III
	APL/LOTUS	150	Interface APL with Lotus
	APL/WORDSTAR	150	Interface APL with Wordstar
	APL/MULTIPLAN	150	Interface APL with spreadsheet
	CEMAS	3,500	EEC monetary and agrimonetary analysis.
M.B.T.	RHOMBUS	poa	Integrated Ofsu3suce System
	HASLEMERE	poa	Hotel Accounting System
Mercia	STATGRAPHICS 3.0	685	Integrated stat. graphic system for PCs.
	Upgrade 2.1 to 3.0	395	
	Upgrade 2.6 to 3.0	175	
	MICROSPAN	250	Comprehensive APL tutor
	LOGOL	poa	Logistics management system for PC, Forecasting, Inventory Control, Scheduling, Distribution, etc
MetaTechnics	MetaScreen	99	Full-screen handler for APL*PLUS/PC, based on VSAPL AP124
	MetaPack	495	Comprehensive utilities package for APL*PLUS/PC. Includes. MetaScreen, MetaWS, Browse, Toolbox, Numeric Editor.
	APL-IEEE488	99	Controls IEEE488/GPIB Bus from APL*PLUS/PC.
	PLOT/PC	99	2D & 3D Graphics package. Includes interactive diagram Editors.
	Browse	99	Scrolling of DOS files, large APL variables.
	ADAPTA DLS	poa	Production & purchasing scheduling for process manufacturing.
	ADAPTA MSP	poa	Job-shop loading & scheduling for multi-stage production.
MicroAPL	MicroTASK	250	Product development aids
	MicroFILE	250	File utilities and database
	MicroPLOT	250	Graphics for HP plotters etc
	MicroLINK	250	General device communications
	MicroEDIT	250	Full screen APL editor
	MicroFORM	250	Full screen forms design
	MicroSPAN	250	Comprehensive APL tutor
	MicroGRID	poa	Ethernet & other networking
	APLCALC	400	APL spreadsheet system
	MicroPLOT/PC	250	For APL*PLUS/PC product
	MicroSPAN/PC	250	For APL*PLUS/PC product
	PC TOOLS Vol 1	295	
	STATGRAPHICS Rel 1	495	
	STATGRAPHICS Rel 2	535	
Parallax	ExecuCalc	\$5,000	Mainframe-based electronic spreadsheet for VM/CMS & MVS/TSO with links to micro products.

I.P. Sharp	ExecuPlot	\$5,000	Mainframe-based colour graphics with micro links.
	ACT	poa	Actuarial system
	APS	poa	Financial Modelling
	BOXJENKINS	poa	Forecasting technique
	CONSOL	poa	Financial Consolidation
	COURSE	poa	APL Instruction
	EASY	poa	Econometric Modelling
	FASTNET	poa	Project Management
	GLOBAL LIMITS	poa	Exposure management for banks
	MABRA	poa	Record maintenance/reporting
	MAGIC	poa	Time series analysis/reporting
	MAGICSTORE	poa	N-dimensional database system
	MAILBOX	poa	Electronic Mail
	MAILBOX/PC V.2	65	Full screen front end to IPSA mailbox
	upgrade to V.2	15	
	MICROCOM	poa	Mainframe to micro link
	Q PACK V.7	45	Object packaging for APL-LAB/PC
	Q PACK upgrade	10	Upgrade v.6 to v.7
	Q PACK RTS	10	Run time version
	SAGA	poa	General graphics, most devices
	SIFT	poa	Forecasting system
	SNAP	poa	Project management
	SUPERPLOT	poa	Business graphics
	VIEWPOINT	poa	4GL - Info centre product
	XTABS	poa	Survey Analysis
Sugar Mill	Stat 1	\$129.95	Statistical toolbox, menu driven
Uniware (mainframe)	STSC's ENHANCEMENTS	10,715	Quad-functions & nested arrays for IBM VSAPL under VM/CMS and MVS/TSO
	STSC's SHAREFILE	10,715	Component files for IBM VSAPL under VM/CMS and MVS/TSO and for IBM APL2
	PROGRAMMER TOOLS & UTILITIES	5,715	
	FILEPRINT	5,715	
	FILESORT	5,715	
	FILECONVERT	5,715	
	FILEMANAGER (EMMA)	5,715	STSC's database package.
	APL*PLUS COMPILER	21,430	First APL compiler. Complements APL*PLUS enhancements and Sharefile under VM/CMS and MVS/TSO.
	EXECUCALC	3,995	Mainframe spreadsheet compatible with VISICALC and part of LOTUS 1-2-3 under VSAPL (VM or TSO).
	STATGRAPHICS	725	Statistics & Graphics for PCs.
	STATGRAPHICS FCA	140	An add-on module to STATGRAPHICS: Factorial Correspondence Analysis.
	APL*PLUS/PC TOOLS - VOL1	325	Incl. 327 x IRMA support, RAM disk, full screen data entry, menu input, report generation, games.
	- VOL2	125	Incl. File documentor, screen editor, exception handling.
	SPREADSHEET MNGR	250	APL spreadsheet with built-in ASCII, LOTUS and SYMPHONY interfaces.
	APL*PLUS/PC FIN. & STAT.LIBRARY	350	Collection of financial and statistical utilities.
(microcomputer)	POCKET APL	140	Smaller version of APL*PLUS/PC.
	UNIASM	275	Collection of assembler routines for APL*PLUS/PC users.

UNITAB	240	APL*PLUS/PC spreadsheet-like data entry and validation system.
The APL DEBUGGER	105	First released APL*PLUS/PC debugger.
OVERLAYS	250	Fast assembler routines to handle overlays in APL*PLUS/PC.
R.BRIDGE	380	Interface between APL*PLUS/PC & R:BASE 5000.
DMA	380	A version of EMMA (APL database manager) for APL*PLUS/PC users.
APL2C	295	Interface between APL*PLUS/PC and DATALIGHT C language.
ADAPTA/DLS	33,333	Production & purchasing scheduling for process manufacturing.
Warwick University		
BATS	500	Menu driven system for time series analysis and forecasting using Bayesian Dynamic modelling. Price is reduced to 35 for academic institutions.
FAB	free	Training program for the above.

APL CONSULTANCY

(prices quoted are per day unless otherwise marked)

COMPANY	PRODUCT	PRICES (£)	DETAILS
APL Consulting	Consultancy	180-550	Expertise in APL system design, project management, prototyping, financial applications, decision support systems, MIS, links to non-APL systems, documentation.
APL People	Consultancy	from 120	Consultants available at all levels, with experience in: VS APL, APL*PLUS, APL2, Sharp APL, Dyalog APL, APL68000, C/Unix, TSO/MVS, VM/CMS, graphics. Additional skills available: OR, statistics, financial, commerce, retail, engineering.
APL Software Technology	Consultancy	poa	Technical & business systems, micros, networking & communications a speciality.
Buckland Management Systems	Consultancy	poa	Business and Technical systems in commerce and industry. Designing, programming and implementing applications of O.R. and statistics.
Camacho	Consultancy	poa	Specialising in programming & manual writing.
Chapman	Consultancy	150-300	24-hour programmer: APL, C, assembler, graphics; PC, mini, mainframe, network.
Cocking/Drury	Consultancy	120-150 140-200 185-300 275-400	Junior consultant Consultant Senior consultant Managing consultant
Peter Cyriax	Consultancy	100-150 120-200 160-300	Junior Consultant Consultant Senior Consultant
Delphi	Consultancy	poa	Specialising in management reporting systems and APL on microcomputers.
Dyadic	Consultancy	poa	APL system design, consultancy, programming & training for Dyalog APL, VSAPL, APL*PLUS, IPSA APL etc.
E & S	Consultancy	150-250	System prototyping: all types of information system.
FASTCODE	Consultancy	poa	Specialise in improving performance of APL applications on micros & mainframes.
Gen. Software	Consultancy	100+	
H.M.W.	Consultancy	poa	System design consultancy, programming. HMW specialize in banking and prototyping work.
Inner Product	Consultancy	200	On-site micro-mainframe APL, PC/DOS & Assembler
Lloyd Savage	Consultancy	poa	Decision support, particularly specialising in Sales & Marketing systems.
M.B.T.	Consultancy	poa	
Mercia	Consultancy	poa	APL*PLUS & VSAPL consultancy.

MetaTechnics	Consultancy	poa	Management Information & Production. Engineering, APL - C/Assembler custom programming
MicroAPL	Consultancy	poa	Technical & applications consultancy.
M.T.I.	Consultancy	poa	Specialise in Maintenance and development of existing APL systems
Parallax	Consultancy	\$750	Introductory APL, APL for End-user & Advanced Topics in APL
QB On-Line	Consultancy	200	Specialising in Banking, Financial & Planning Systems.
Rochester Group	Consultancy	poa	Specialise in MIS using Sharp APL
I.P. Sharp	Consultancy	poa	Consultancy & support service world-wide.
Uniware	Consultancy	call	Junior to managing consultancy in APL.

OTHER PRODUCTS

COMPANY	PRODUCT	PRICES(£)	DETAILS
APL People	Employment Agency	poa	Permanent employees placed at all levels. Contractors supplied for short/long-term contracts, supervised or unsupervised. Executive Search service available.
HMW	Employment	poa	Contractors and permanent employees placed.
I.P. Sharp	Productivity Tools	poa	Utilities for systems, operations, administration & analysts; auxiliary processors, comms software, international network.
	Databases	poa	Financial, aviation, energy and socioeconomic.

VENDOR ADDRESSES

COMPANY	CONTACT	ADDRESS & TELEPHONE No.
APL 385	Adrian Smith	Brook House, Gilling East, York. Tel: 04393-385
APL Consulting	Jill Moss	The Old Malthouse, Clarence St, BATH, BA1 5NS. Tel: 0225-462602
	Tim Perry	Jubilee House, Chapel Rd, HOUNSLOW TW3 1XT. Tel: 01-577 3541
APL Impetus Ltd	Cedric Heddle	Rusper, Sandy Lane, Ivy Hatch, SEVENOAKS, Kent TN15 0PD Tel: 0732-885126
APL People	Jill Moss	The Old Malthouse, Clarence St, BATH, BA1 5NS. Tel: 0225-462602
APL Software	David Alis	Jubilee House, Chapel Rd, HOUNSLOW TW3 1XT Tel: 01-577 3541
APL Software Technology	John Hagger	14 Rosewood Avenue, Alveston, Bristol BS12 2PP Tel: 0454415737
	Per Hultin	46 Vicarage Road, South Benfleet, Essex SS7 1PB Tel: 0374550501
Beta-plan APS	Kim Andreassen	Stengrade 75, DK-3000 Helsingor, Denmark. Tel: 45 2 21 48 48
Buckland Management Systems	John Buckland	Westwood, 9 Grove Road, Camberley, Surrey, GU15 2DN Tel: 0276 684327
Butel Technology Ltd.	Mike Munro	Butel House, 3 Great West Rd., London W4 5QJ Tel: 01-995-1433
Anthony Camacho		2 Blenheim Road, St. Albans, Herts AL1 4NR. Tel: St. Albans (0727) 50130
Paul Chapman		18, Trevelyan Road, London, SW17 9LN Tel: 01-767 4254
Cocking & Drury Ltd.	Romilly Cocking	180 Tottenham Court Road, LONDON, W1P 9LE Tel: 01436 9481
		155 Friar Street Reading RG1 1HE Tel: 0734-588835
Datatrade Ltd.	Tony Checkfeld	38 Billing Road, Northampton, NN1 5DQ. Tel: 0604-22289
Delphi Consultation	David Crossley	Church Green House, Stanford-in-the-Vale, Oxon SN7 8LQ. Tel: 03677-384
Dyadic Systems Ltd.	Peter Donnelly	Park House, The High Street, Alton, Hampshire. Tel: 0420-87024
E & S Associates	Frank Evans	19 Homesdale Road, Orpington, Kent BR5 1JS. Tel: 0689-24741
Farnell International Instruments Ltd.	R. Fairbairn	Jubilee House, Sandbeck Way, Wetherby, W. Yorks. Tel: 0937-61961
	Roger Attard	Davenport House, Bowers Way, Harpenden, Herts. Tel: 05827-69071

FASTCODE	Andrew Dickey	P.O. Box 281, Croton-on-Hudson, New York 10520, U.S.A. Tel:(914) 271-3200
General Software Ltd.	M.E. Martin	22 Russell Road, Northolt, Middx. UB5 4QS. Tel: 01-864 9537
H.M.W. Computing Ltd.	Stan Wilkinson	Hamilton House, 1 Temple Avenue, Victoria Embankment, LONDON EC4Y 0HA Tel: 01-353 4212
IBM UK Ltd	Chris Sell	PO Box 32, Alencon Link, Basingstoke, Hants. RG21 1EJ. Tel: 0256-56144
Inner Product Ltd.	Dominic Murphy	Eagle House, 73 Clapham Common Southside, London SW4 9DG. Tel: 01-673 3354
Lloyd Savage Ltd	Philip Johnson	Cambridge House, Oxford Road, Uxbridge, Middx. UB8 2UD. Tel: 0895-58826
Mercia Software Ltd.	Gareth Brentnall	Aston Science Park, Love Lane, Birmingham B7 4BJ. Tel: 021-359 5096
MetaTechnics Systems	John Stenbridge	Unit 216, 62 Tritton Road, London, SE21 8DE Tel: 01-670 7959
MicroAPL Ltd.	David Eastwood	South Bank Technopark, 90 London Road, LONDON SE1 6LN Tel: 01-922 8866
M.T.I.	Ray Cannon	7 Pine Wood, Sunbury-on-Thames, Middx. TW16 6SH Tel: 09327 80848
Parallax Systems Inc.	Kevin Weaver	60 West 9th Street, New York, New York 10011, U.S.A. Tel: 212-475-4001
Peter Cyriax Systems	Peter Cyriax	213 Goldhurst Terrace, London NW6 3ER Tel: 01-624 7013 (Answerphone) 0860-377963 (Mobile)
Portable Software	Richard Smith	60 Aberdeen Ave, Cambridge, Mass. U.S.A. 02138. Tel: 617-547-2918
QB On-Line Systems	Philip Bulmer	5 Surrey House, Portsmouth Rd Camberley, Surrey, GU15 1LB. Tel: 0276-20789
The Rochester Group	Robert Pullman	164 Pinnacle Rd., Rochester NY 14620 Tel:716-481-3169
Shandell Systems Ltd.	Maurice Shanahan	12 High Street, Chalfont St. Giles, Bucks HP8 4QA. Tel: 02407-2027
I.P.S.A. Ltd.	David Weatherby	10 Dean Farrar Street, London SW1. Tel: 01-222 7033
Sugar Mill Software Corp.	Lawrence H. Nitz	1180 Kika Place, Kailua, Hawaii 96734 Tel: (808) 261-7536
Tektronix UK Ltd.	Paul Morgan	Fourth Avenue, Globe Park, Marlow, Bucks SL7 1YD. Tel: 06284-6000
Uniware	Eric Lescasse	15 Rue Erlanger, 75016 Paris, France Tel: (1) 45-27-20-61 Telex: 648348F UNIWARE
Warwick Univ.	Dr Andy Pole	Dept of Statistics, University of Warwick, Coventry, CV4 7AL Tel:0203-523066 x2158

RENAISSANCE DATA SYSTEMS
BOOK DIVISION

*** Enlightenment Thru Information Processing ***
1989 Catalog Excerpts (as of January 1, 1989)
+++ All APL Books in Print +++

		Quantity	Total
		Price	Cost

Important for the Professional APL Programmer:			
=====			
THE APL IDIOM LIST, Perlis and Rugaber - Funded by Mobil. Written at Yale.	\$ 6.50.	.	.
Very terse, but includes nuggets of gold. 1977, 50 p.	.	.	.
APL - A DESIGN HANDBOOK FOR COMMERCIAL SYSTEMS, Smith - Readable, technical,	\$44.00.	.	.
thorough. Includes a small collection of good idioms. 1982, 180 p.	.	.	.
APL - ADVANCED TECHNIQUES AND UTILITIES, Bergquist. Good discussion of	\$44.95.	.	.
alternative approaches to a wide range of programming tasks - efficiency,	.	.	.
searching, dates, workspace documentation, file design, boolean techniques,	.	.	.
financial utilities. Assumes knowledge of the basics. Many idioms. 1986, 450 p.	.	.	.
Diskette for above. Contains all functions described in text. Format is	\$15.00.	.	.
for STSC APL*PLUS/PC but is uploadable to Sharp APL, VS/APL, APL2.	.	.	.
APL HANDBOOK OF TECHNIQUES, IBM - Utilities. Many comments. 1978, 109 p.	\$18.00.	.	.
A Bit of APL History:	.	.	.
=====	.	.	.
~~~~~ New Printing! ~~~~~	.	.	.
A PROGRAMMING LANGUAGE, Iverson - The book that started it all. A computer	\$59.95.	.	.
science classic. APL's roots. Reprinted by RDS on acid free paper in a	.	.	.
high quality, soft cover edition. Only 25 copies available! 1962, 286 p.	.	.	.
A SOURCE BOOK IN APL, Falkoff & Iverson - By the famous duo. A must for	\$15.00.	.	.
serious APLers. The whys and the wherefores. Includes "Notation as a	.	.	.
Tool of Thought," Iverson's 1979 ACM Turing Award Lecture. 1981, 139 p.	.	.	.
So you want to learn APL?	.	.	.
=====	.	.	.
APL2 AT A GLANCE; Brown, Pakin, Polivka. Solid, unimimidating, introduction	\$26.00.	.	.
to APL2. Clear illustrations, modern exercises in each chapter. Gets you	.	.	.
started. Three sample applications. 1988, 444 p.	.	.	.
APL - AN INTRODUCTION, Peele - Especially for the computer-shy. 1986, 459 p.	\$29.95.	.	.
----- Proceedings from APL Conferences -----	.	.	.
APL AS A TOOL OF THOUGHT, PROFESSIONAL DEVELOPMENT SEMINARS, NY/SIGAPL. They	.	.	.
cover a wide range of topics in business and education.	.	.	.
1987 - Logic, insurance, statistics, A.I., accounting, fractals, linked lists.	.	.	.
... 1984, 1985, 1986, 1987, 120 to 150 pages each	each \$15.00.	.	.
1983-1987 as a package.	\$55.00.	.	.
Sub-total:	\$	.	.
Postage and handling - U.S. and Canada (\$2.50 each item over \$25, \$2.50 minimum.):	.	.	.
Postage and handling - International (please add 15% surface mail, 50% airmail):	.	.	.
NYC Residents please add 8 1/4% sales tax, NY State, 4%:	.	.	.
Total:	\$	.	.
=====	.	.	.

Please send complete catalog: _____

Date: _____

Name: _____

Street: _____

City, etc.: _____

Please make check or money order payable  
in \$US and send with this form to:

Renaissance Data Systems  
P.O. Box 20023  
Park West Finance Station  
New York, New York 10025-1510  
U.S.A.  
(212) 864-3078

Postal  
Code: _____ Country: _____

**Book Reviews:****APL Is Easy!****by Jerry R Turner (STSC,Inc.)****APL2 at a Glance****by James A Brown, Sandra Pakin and Raymond P Polivka**

Reviewed by Bill Ross

**About the Reviewer**

Bill Ross teaches Modern Languages at Ampleforth College (one of the UK's major independent schools). His interest in APL has come both from the academic angle (a love of Russian and Chinese helps), and from an appreciation of its usefulness in simple tasks like working out end of term averages and grades.

**Summary**

The authors of both the above texts - all of whom are experienced APL educators - have given us two very useful primers for beginners.

**APL is Easy!**

As a "natural" linguist and non-APL speaker, my curiosity about the syntax and semantics of the language has rapidly been satisfied by APL Is Easy! The author makes no bones about the modest scope of his manual, which admirably achieves its aim of getting the reader to 'learn the language easily' and of teaching him some useful things to do with what he learns. The exposition of the text is a model of clarity (as is the page layout) and the copious examples and exercises are conceived in such a way that the reader gains confidence in his ability to solve problems using APL and gains insights into some useful everyday applications. Solutions are provided at the back, making the text eminently suitable for self-study. The author views his book as an introduction to a more substantial treatment of APL, such as Gilman and Rose. Having embarked on the quest for APL by using rather more arcane materials of dubious pedagogical value, I was relieved and overjoyed to discover a book which actually sets out to explain and teach the language to the non-initiate.

The 10 chapters are:

1. What is a Program
2. Simple Arithmetic, Error Correction, and Data Storage
3. Character Data, More Arithmetic and More APL Functions
4. Selecting and Analyzing Data
5. Writing Programs in APL
6. Formatting
7. Storage Facilities: Workspaces
8. Storage Facilities: Files
9. APL Programming Techniques
10. Exploring APL

The book concludes with solutions to the exercises, 3 appendices, a glossary and an index.

At £15 it is hardly a "steal" at the price, but for those seeking to learn the language autonomously, it is well worth the money.

APL Is Easy!

by Jerry R Turner (STSC, Inc.)

Paperback, 180pp text plus 66pp appendices and index

Published by John Wiley & Sons, Inc., New York, 1987

Price £15

### APL2 at a Glance

APL2 at a Glance is an introduction to the programming language APL2, the PC version of which has recently been released in this country by IBM. APL2 is more versatile than APL and opens up a wealth of new programming paradigms that do not exist in APL. This text is also aimed at the reader without any prior knowledge of APL, though its authors consider that it will also be of interest to fluent speakers of APL 'because of its emphasis on the APL2 data structures and the style of programming brought about by its new functions and operators.'

Once again the exposition is gradual and clear, though the layout is rather less pleasing than the Turner text. (What a shame that the laminated paper cover of the book will not lie flat! - a bibliophile's quibble, albeit.) In common with the Turner text, each chapter includes a wealth of examples which demonstrate the notions discussed and the exercises help one understand the capabilities and practical applications of APL2. Unfortunately I was unable to work through all of the exercises on my computer, since I do not yet possess a copy of APL2. The exercises require the student to evaluate expressions or to work problems involving the creation of expressions and functions.

The fact that this expository text is so highly readable and accessible to the beginner is, no doubt, the result of the authors' close acquaintance with the implementation and

teaching of the 'old' APL as well as APL2, both within and without the IBM organisation. The authors are to be congratulated for this considerable pedagogical achievement. The work is suitable for either independent study or classroom use.

APL2 at a Glance contains 8 chapters that present the basic features:

1. Working with APL2
2. Working with Vectors
3. Working with Programs
4. Working in the APL2 Environment
5. Working with Arrays
6. Working with Data
7. Working with Program Control
8. Working with Applications

APL2 at a Glance has several appendices - some highly entertaining -, a glossary, a bibliography containing selected titles on APL" and works cited in the text and an index. I found appendix E, which gives the nomenclature for the APL2 character set particularly helpful.

APL2 at a Glance

by James A Brown, Sandra Pakin and Raymond P Polivka

Paperback, 373pp text plus 88pp appendices, index, etc.

Published by Prentice Hall, Englewood Cliffs, New Jersey 07632

Price £21



## JOIN THE WINNING TEAM

OUR APL TEAM HAS JUST WON A **DESIGN COUNCIL AWARD**, WHAT NEXT?

Five years ago Metapraxis established a new market for Executive Information Systems with its PC-based system RESOLVE. Since then we have led the way in helping senior directors of large organisations to use computers as serious business tools to give them bottom-line benefits.

Our approach has been implemented in more than 70 major companies in the UK and abroad and has developed to meet our clients' increasingly sophisticated needs and uses. We're embarking on a major initiative to develop the next generation of information technology tools for top executives. This will keep us at the forefront of technological innovation and be designed to bring immediate benefits to users.

Now we're looking for people with at least 5 years experience of APL, expertise in software design in a business environment who are very stimulated at the thought of breaking new ground in a rapidly developing and exciting field.

**Perhaps you will win the next Design Council Award for Metapraxis.**

Please write enclosing your c.v. to

David Preedy, Director of Development, Metapraxis Ltd,  
Hanover House, Coombe Road, Kingston-on-Thames, KT2 4AH,  
or ring on 01 541 1696.

---

## RECENT MEETINGS:

This section of VECTOR is intended to document the seminars given at recent meetings of the association; it is of particular value to members who live away from London. It also covers other selected events which may be of general interest to the APL community.

We are dependent on the willingness of speakers to provide a written version of their talk, ideally with copies of diagrams or slides used.

If you would like to speak at one of the regular British APL Association seminars, please ring the Activities Officer (address on inside back cover) who will respond enthusiastically to your offer.

### Contents

APL at the Leading Edge	November 18th 1988	56
Summary of Meeting	Adrian Smith	56
Parallel Processing	John Adams	56
Expert systems in APL	Allan Prys Williams	57
Integrated Information: APL and SQL	Brian Lochran	75
Function Point Analysis in APL	January 13th 1989	79
Introduction to FPA	Romilly Cocking	79
FPA in IBM Systems Development	Alan Williams	80
FPA in APL Systems at BA	Maurice Jordan	83
APL: Future Plans	February 17th 1989	84

---

## **APL at the Leading Edge Nov 18th at the Royal Over-Seas League**

introductory notes by Adrian Smith

Allan Prys Williams contrasted the way APL works with the way a rule-based system is built, and ended by showing how you could combine the approaches to provide existing software with an intelligent front-end. His notes follow this introduction.

Brian Lochran showed how United Glass had moved a substantial APL planning system from a mainframe (ADRS II) environment to a network of IBM 6150 machines using Dyalog APL and SQL. His notes are on page 75.

John Adams reviewed his experiences at the recent APL/AI conference in New York. His notes appear in VECTOR 5.2.

Finally, Tom Lake of Intercept Systems gave us an overview of parallel systems. This was one of those useful talks which came from a non-APL perspective. Tom defined 'parallel' as "several computers under the control of a single program". He described the evolving 'culture of the parallel' in applications such as:

- the simulation of physical systems. Fluid flow, weather forecasting, aerodynamics and so on. Also analysis of stress in engines and structures.
- animated graphics. This could be anything from Walt Disney to studies of molecular folding in drug design.
- image processing, particularly of radar and sonar images.

There are two approaches to building a parallel system:

- by repetition. The pipeline approach: instructions follow each other in short order through custom-designed chips. 20 Mflops are achievable if you keep them fed.
- by replication, e.g. DAP which has 4096 little simple processors all doing the same thing. Programmed in a sort of FORTRAN using vector and array types in a vaguely APL-ish sort of way.

Tom closed by noting that parallel people are willing to put a lot of work into the program to make life as easy as possible for the compiler. APL is something of a paradox, being a language where someone put a lot of work into the interpreter to make life as easy as possible for the programmer, and yet it seems eminently suitable for parallel processing. As was said of Chi Chi and An An "why don't they get on with it!".

ABSTRACT

An expert system is described which runs the functions in an existing APL workspace as though it were a diligent first-year statistics student. The rules that it operates under are contained as logical data and can be reviewed and modified by an expert without any need to alter APL functions. The special functions used provide a model for a wide class of systems that can run existing APL software.

### 1. The Object

Inexperienced users of Statistics are often faced with a bewildering array of options when asked to analyse their set of data. In an APL environment, this typically means that they have a variable containing numerical data and need to choose the appropriate functions from an existing workspace to test it and display it. This calls for an expert system that will act as the equivalent of an experienced user looking over their shoulder and tell them what to do. Generically, expert systems:

1. Use sets of explicit rules.
2. Ask questions and remember the answers.
3. Give the user suggestions as to what they should do and perhaps do it for them.
4. Have massive overheads from complex logical systems.

In an APL environment, the expert system will itself run in APL, but it can be distinguished from 'ordinary' APL functions which:

1. Have their rules hidden.
2. Return a unique answer for unique data.
3. Require users to know which function they want.
4. Represent perhaps years of investment.

The expert system is a statement about the large amount of expertise already embodied in the APL functions.

The present expert system contains knowledge about the rules of Statistics and (perhaps) about the data, organised so that a standard function can carry out an appropriate analysis, asking the user questions if it has to. Except for the few special functions, all the analyses can be repeated or varied by the user from the keyboard if he or she wants to find out more. In other words, the expert system function uses the existing workspace as though it were a diligent first-year Statistics student.

The fact that the rules are held as data in a logical form, rather than inside APL functions, means that they can be varied easily by someone with a minimal knowledge of APL. This removes the need to be simultaneously expert in both APL and Statistics.

Standard APL logic structures are explained in [1] and [2] and the provision of an ability to implement existing APL functions is explained in [3]. The reader can either refer to them or take the logical functions as read. The 'inference engine' itself is a function called PROLOGT which acts on vectors of Prolog-like rules and allows the user to respond by asking for another solution or for an explanation of its reasons.

### 1.1 The Existing Workspace

We start with a substantial array of standard APL functions for displaying and analysing data.

#### Display Functions

HIST        Displays a histogram of (pooled) data across the screen, each cell being marked by its mid-point.

BTBHIST	A similar function that compares two sets of data, back-to-back.
UNIQUFREQ	Gives a frequency distribution for (pooled) data.
PLOT	Scatterplot comparing two variables.
QQPLOT	Diagnostic aid in judging Normality. A straight line would indicate an exactly Normal distribution. Real samples from Normal populations are allowed to wobble a bit.
BREAKDOWN	Display summary statistics for mult-group data.
<u>Test Functions</u> Unless otherwise stated, all these functions return a 2-sided tail probability.	
IOD	Gives the tail probability from an Index of Dispersion test. A highish value is expected from a Poisson distribution.
CHISI	The probability from a standard Chi-squared test for independence in a contingency table.
TTEST	Matched-pairs t-test.
WTEST	Matched-pairs Wilcoxon test.
TWOT	2-sample t-test for location.
TWOW	2-sample Wilcoxon test for location.
PEARSON	Tests Pearson correlation using t-approximation.
SPEARMAN	Tests Spearman correlation using t-approximation.
KRUSKALL	Tests difference between groups using Kruskal-Wallis method.
ANOVA	One-way Analysis of Variance.
FTEST	Test for difference of scatter between two samples

## 1.2 The Special Functions

The cover function takes as right argument the name of the data variable, and has the significance level as an optional left argument. If this is not specified, it reports exact probabilities. This is usually taken to be better practice, but it can have have strange side-effects if carried out with logical thoroughness.

```

V {R}+{LA}LOOKAT RA;⌈TRAP
  A Run expert system on data in RA at sig LA
[1]  ⌈TRAP+11 'E' '' 'Right argument must be the name o
    f an object' ⌈ →0'
[2]  →((O=pp⌈RA)v2;=⌈RA)/badshape
    A Expects fairly simple data objects
[3]  ⌈(O=⌈NC'LA')/'LA+''ΔLevel''
    A Monadic case: report probabilities
[4]  LA+,⌈LA
    A Convert LA to character string: RA must be a NAM
[5]  R+(('(<'suggest'RA'ΔTest'LA))PROLOGT STATDB
    A Runs logic program
[6]  ⌈(R=O)/'' 'No suitable test found by set of rules
    in STATDB'' ⌈ →0
[7]  badshape:RA,' has too complicated a structure for
    this function'

```

Exhibit 1 shows the rules of the expert system, reduced to a logical format; the APL object being in fact a nested character vector of great complexity. Each rule, except the first, gives conditions that must be true before a particular test is tried, and runs the test if they are all passed. The first rule tells the system to look for a test, and, if it finds one, looks for an appropriate display. Most of these are adapted from a student project by David Austin who was developing a similar system, but one in which much of the structure was held inside a function, rather than being free-standing logical rules.

The first special utility function is

## ASCIFY STATDB

```

suggest(Name, Test, Level) <-
  sig_test(Name, Test, Level) , SHOW(Name, Test)
sig_test(Name, IOD, Level) <-
  VERIFY(Name, discrete) , VERIFY(Name, same_var) ,
  UNLESS(RUNTEST(Name, IOD, Level))
sig_test(Name, TTEST, Level) <-
  VERIFY(Name, same_var) , VERIFY(Name, 2-sample) ,
  VERIFY(Name, matched) , VERIFY(Name, normal) ,
  RUNTEST(Name, TTEST, Level)
sig_test(Name, WTEST, Level) <-
  VERIFY(Name, same_var) , VERIFY(Name, 2-sample) ,
  VERIFY(Name, matched) , RUNTEST(Name, WTEST, Level)
sig_test(Name, FTEST, Level) <-
  VERIFY(Name, same_var) , VERIFY(Name, 2-sample) ,
  VERIFY(Name, normal) , RUNTEST(Name, FTEST, Level)
sig_test(Name, TWOT, Level) <-
  VERIFY(Name, same_var) ,
  VERIFY(Name, 2-sample) , VERIFY(Name, normal) ,
  UNLESS(RUNTEST(Name, FTEST, Level)) ,
  RUNTEST(Name, TWOT, Level)
sig_test(Name, TWOW, Level) <-
  VERIFY(Name, same_var) , VERIFY(Name, 2-sample) ,
  RUNTEST(Name, TWOW, Level)
sig_test(Name, PEARSON, Level) <-
  VERIFY(Name, 2-sample) , VERIFY(Name, matched) ,
  VERIFY(Name, normal) , RUNTEST(Name, PEARSON, Level)
sig_test(Name, SPEARMAN, Level) <-
  VERIFY(Name, 2-sample) , VERIFY(Name, matched) ,
  RUNTEST(Name, SPEARMAN, Level)
sig_test(Name, ANOVA, Level) <-
  VERIFY(Name, same_var) , VERIFY(Name, many-sample) ,
  VERIFY(Name, normal) , RUNTEST(Name, ANOVA, Level)
sig_test(Name, KRUSKALL, Level) <-
  VERIFY(Name, same_var) , VERIFY(Name, many-sample) ,
  RUNTEST(Name, KRUSKALL, Level)
sig_test(Name, CHISI, Level) <-
  VERIFY(Name, discrete) , VERIFY(Name, matched) ,
  VERIFY(Name, counts) , RUNTEST(Name, CHISI, Level)
sig_test(Name, pooled, Level) <- VERIFY(Name, same_var)

```

Exhibit 1. The Rules For The Expert System

```

V R←RUNTEST PASS;N;F;L;⍋TRAP;⍋PP
A Runs test F on data in N at level L
[1]   ⍋TRAP←((3 4 5 11)'C' 'R+0') ⍋ ⍋PP+2
      A Index,rank,length,domain error
[2]   N F L←PASS ⍋ +(v/'A'≡'N F')/R+0
      A Fail if data or test not defined
[3]   R←F,' ',N
      A F must be a monadic function on numerical data
[4]   ⍋('A'≡L)/'L','+',⍋ SIGFIG R' ⍋ R+1 ⍋ +0'
      A giving a probability
[5]   R←R≤L
      A N.B. N and F are names of APL objects, L is character

```

This returns a 1 if the test is significant [i.e. returns a probability less than the significance level] and always succeeds if a probability is returned without a level being already specified. In such a case the exact probability becomes the value of 'Level'. The display is handled by

```

V R←SHOW PASS;N;F;X;⍋TRAP;⍋PP
A Goal that shows data N according to F
[1]   ⍋TRAP←((3 4 5 11)'C' 'R+1 ⍋ +0')
      A Index,rank,length,domain errors
[2]   N F←PASS
      A N is name of object, F is test that was confirmed
[3]   X←GETPROP F'show' ⍋ R+1 ⍋ ⍋PP+2
      A Look for display function in APL
[4]   ⍋(X≠'')/(≡X),' ',N
      A If so, use it. Always succeeds
[5]   X←(≡PRAP)1<N ⍋ X←(X≡≡PRAP)/X
      A Reference to N in global memory
[6]   ⍋(0;≡X)/'X',X[;2]/(X+1≡PRAP[X])[;1]'
      A Find properties of N
[7]   ⍋(0;≡X)/'N',N believed to be ' ',X'
      A List to screen

```

which looks in a global memory using the standard GETPROP function for an appropriate display function. It also gives some reasons. The checking function is

```

V R←VERIFY PASS;N;F;X;␣TRAP
  A Goal that checks contents of PASS
[1]  ␣TRAP←((3 4 5 11)'C' '→R←O')
      A Index,rank,length,domain errors
[2]  N F←PASS
      A N is name of object, F is function that may be true
[3]  X←GETPROP N F
      A See if we already know the answer
[4]  ␣(X≠'')/'R←X ␣ →O'
      A If so, exit with existing answer
[5]  X←GETPROP F'test'
      A Look for test function in APL
[6]  ␣(X≠'')/'R←',(⊃X),' ',N
      A If so, run test and report answer
[7]  ␣(X≠'')/'→(R⊂O 1)/O'
      A Exit if answer useful
[8]  X←GETPROP F'text'
      A Look for text to expand defn. of function
[9]  ␣(X≠'')/'X←⊃CF'
      A If no text, use name of function
[10] X←F'Are the contents of ',N,(⊃X),'? Y/N [N]'
      A Construct query
[11] ␣X ␣ R←␣ ␣ R←(⊃pX)←R
      A Delete question from response
[12] R←v/'yy'←R ␣ R←PUTPROP N F R ␣ →O
      A Store for future reference

```

which asks questions if necessary and updates the global memory so that it never has to ask the same question twice. Exhibit 2 shows the initial contents of the global memory. It is a nested vector with a name designed to discourage accidental deletion. The utility functions cited in the global memory are simple ones that work directly on numerical variables without conversion to logical notation.

```

V R←VIFM X
[1]  R←X A Default passes X straight through
[2]  ␣(1=≡X)/'R←,X' A Matrix converted to nested vector

```

allows for the complicated variety of objects that can occur in second-generation APLs, reducing them to nested vectors as necessary. The remaining functions are tests, corresponding to a quick glance by a human.

```

      ↑PRAP
TTEST      show  HIST⇒-/VIFM
TWOT       show  BTBHIST/VIFM
FTEST      show  BTBHIST/VIFM
ANOVA      show  BREAKDOWN VIFM
KRUSKALL   show  BREAKDOWN VIFM
pooled     show  HIST⇒,/VIFM
WTEST      show  HIST⇒-/VIFM
IOD        show  UNIQFREQ⇒,/VIFM
SPEARMAN   show  PLOT/VIFM
PEARSON    show  PLOT/VIFM
discrete   test  DISCRETE
2-sample   test  2⇒p
matched    text  matched across cases      test  MATCHED
many-sample test  MANY
normal     test  QQSHOW
same_var   text  all the same variable      test  SINGLE
TWOV       show  BTBHIST/VIFM

```

### Exhibit 2. The Initial State Of The Global Memory

```

▽ R←DISCRETE X A Test if elements of X are integers
[1]  R←X=IX

▽ R←MANY X A Check if X contains more than 2 samples
[1]  →((1=≡X)∧1=ppX)/R+0 A Fail if a vector
[2]  R+2<⇒pX A Applies to matrix or nested vector

```

The next function is more interesting. It can only fail, or return a null result, since the fact that two groups are the same size is not evidence that they are matched.

```

▽ R←MATCHED X A Test if X could be matched data
[1]  →((1=⇒pX)∨(1=≡X)∧1=ppX)/R+0
[2]  →((1=≡X)∧(2=ppX))/R+'''? →0' A Possible if a matrix
[3]  R+(-∧/(⇒pX)=⇒p''1+X)/0
      A If nested vectors, see if same length

```

A similar one has a choice between success and nullity:

```

▽ R←SINGLE X A Check if X must be a single variable
[1]  R+(((1=ppX)∧1=≡X)∨(1=pX)∧2=≡X)/1 A Result is 1 or null

```

The last of this group is simply an oblique way of calling up a QQ plot as an assistance to the user:

```

      ▽ R+QQSHOW X A Cover function for QQPLOT
[1] X+VIFM X ◊ X+▷,/(X--"MEAN"X)+""SDEV""X
[2] R+"'" ◊ QQPLOT X ◊ 'Standardised residuals, all groups/
    variables'

```

It never gives a definite answer.

The last set of functions are ordinary ones that can be run easily from the keyboard and could in fact be used as such by a very retarded student.

The only restriction (and it is not absolute) on the functions that the machine can run is that they should be self-contained and not ask the user unexpected questions, such as required cell widths, when the user is assumed not to be working interactively with the data. Functions that take all their data as a right argument are preferred, this being easy to arrange in second generation APLs, but the use of 'PLOT/' shows that this is not an absolute restriction either.

## 2. Running The System

### 2.1 The Learning Process

The VERIFY function has nothing particularly statistical about it. It simply learns from experience about the characteristics of the variables (or file-reading functions) in the workspace. This saves a lot of time on the second run through the data and in general provides a way of teaching the expert system new facts. Often, for instance when judging normality, the background knowledge that the user has will be needed to supplement the diagnostics built into the expert system. Notice that the default is always to avoid making an assumption.

We show the expert system examining the variable CORN_METHODS which contains data on four different methods of growing corn. There is a background assumption that they are likely not to be normal.

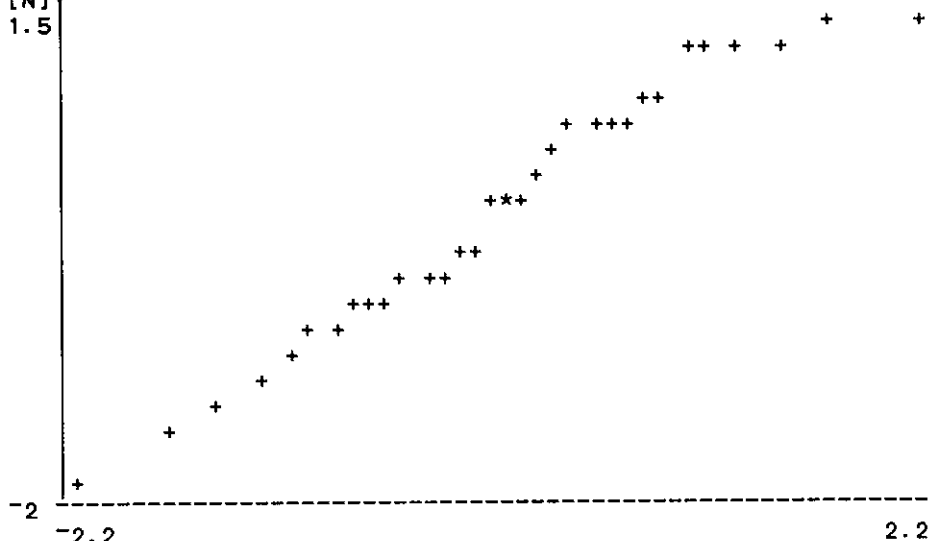
LOOKAT 'CORN_METHODS'

Are the contents of CORN_METHODS discrete ? Y/N [N]

Are the contents of CORN_METHODS all the same variable ? Y/N

[N]Y

1.5



Standardised residuals, all groups/variables

Are the contents of CORN_METHODS normal ? Y/N [N]N

The system has now asked all its questions and has remembered the answers. The machine comes back with a display:

Group:	1	2	3	4	Overall
n:	8	10	7	8	33
Mean:	91	86	96	80	88
S. dev.	3.9	3.8	3.6	1.7	6.5

CORN_METHODS believed to be same_var  
 suggest(CORN_METHODS, KRUSKALL, 0.000015)  
 ;

The ';' response is a prompt for another suggestion:

```

7.2E1 |
7.5E1 |
7.7E1 | ███
8.1E1 | ████████
8.4E1 | ██████
8.7E1 | █
9.0E1 | ██████████
9.3E1 | ██████
9.6E1 | ███
9.9E1 | █
1.0E2 | █

```

```

CORN_METHODS believed to be same_var
(Level)suggest(CORN_METHODS, pooled, Level)
;

```

The system asks appropriate questions and ends up applying the appropriate Kruskal-Wallis test. When fed a ';' it goes back and looks for another answer, finding that the data could be pooled. It reports the exact significance level, 0.000015 for the first and 'all levels' for the second. The new state of the global memory is shown in Exhibit 3, so that the second run is much shorter.

#### .01 LOOKAT 'CORN_METHODS'

Group:	1	2	3	4	Overall
n:	8	10	7	8	33
Mean:	90.8	86.4	95.7	79.9	87.8
S. dev.	3.85	3.78	3.64	1.73	6.52

```

CORN_METHODS believed to be same_var
suggest(CORN_METHODS, KRUSKALL, 0.01)
;

```

```

71.5 |
74.5 |
77.5 | ███
80.5 | ████████
83.5 | ██████
86.5 | █
89.5 | ██████████
92.5 | ██████
95.5 | ███
98.5 | █
101.5 | █

```

```

CORN_METHODS believed to be same_var
suggest(CORN_METHODS, pooled, 0.01)
;

```

```

      ↑PRAP
TTEST      show  HIST=~/VIFM
TWOT       show  BTBHIST/VIFM
FTEST      show  BTBHIST/VIFM
ANOVA      show  BREAKDOWN VIFM
KRUSKALL   show  BREAKDOWN VIFM
pooled     show  HIST=~/VIFM
WTEST      show  HIST=~/VIFM
IQD        show  UNIQFREQ=~/VIFM
SPEARMAN   show  PLOT/VIFM
PEARSON     show  PLOT/VIFM
discrete   test  DISCRETE
2-sample   test  2=⇒p
matched    text  matched across cases      test  MATCHED
many-sample test  MANY
normal     test  QQSHOW
same_var   text  all the same variable      test  SINGLE
TWOV       show  BTBHIST/VIFM
CORN_METHODS same_var 1                      normal 0

```

### Exhibit 3. The Global Memory After Looking At CORN_METHODS

The second run used a fixed significance level of 0.01. The fact that the expert system may know things about a variable necessitates its repeating this knowledge after each result. Because the current user may have altered the data, they must be told what previous users told the system about it.

## 2.2 The Effects of Rigour

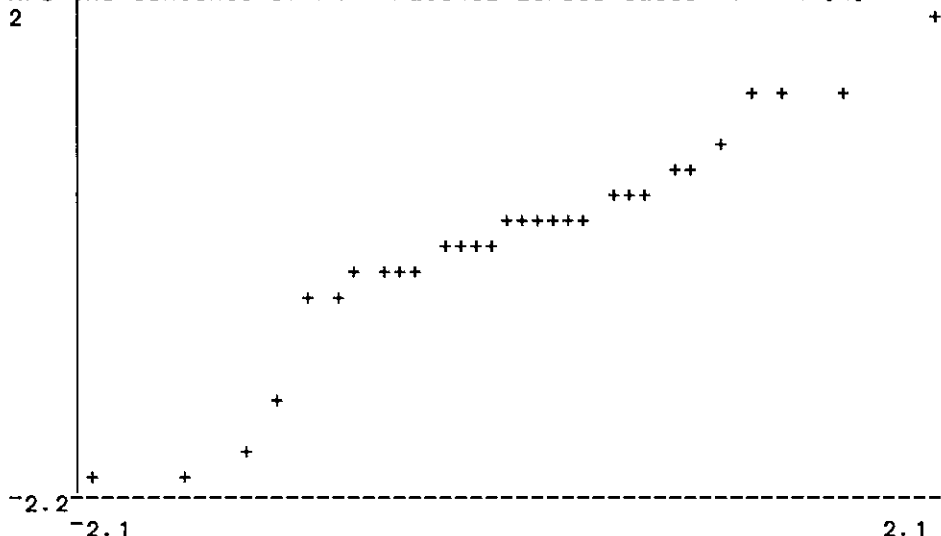
The effects of the rules that your expert puts forward may not be at all what they expected. In this example, two of the rules in Exhibit 1 use the logical UNLESS function. Thus the second rule only records a success if the Index of Dispersion test fails to return a significant [i.e. small] probability, since it is a large probability that should occur if the population is Poisson distributed. Similarly the TTEST rule only carries out that test if an F-test is not significant.

If the 'best practice' of not specifying a level in advance is followed, the unexpected effect is that neither of these tests is ever reported. Both the Index of Dispersion and the F tests always succeed at some level, provided they are considered appropriate at all. F2 contains simulated Normal data.

LOOKAT 'F2'

Are the contents of F2 all the same variable ? Y/N [N]Y

Are the contents of F2 matched across cases ? Y/N [N]



Standardised residuals, all groups/variables

Are the contents of F2 normal? Y/N [N]Y

The system now knows that F2 is normal, same-variable and 2-sample (the latter it worked out for itself) so it duly applies an F-test:

```

** 6.5
   9.5 *
***** 12.5 *
**** 15.5 ***
** 18.5 *****
   21.5 **
   24.5 *
F2 believed to be same_var normal
suggest(F2, FTEST, 0.48)
;

```

A human being would conclude that a probability of 0.48 didn't signify and go on to apply a two-sample t-test. The system is forced to skip onwards to the Wilcoxon equivalent:

```

      ** 6.5
      9.5 *
***** 12.5 *
      **** 15.5 ***
      ** 18.5 *****
      21.5 **
      24.5 *
F2 believed to be same_var normal
suggest(F2, TWOW, 0.0009)
;

```

It then finishes by giving the pooled distribution:

```

6.5 |  □□
9.5 |  □
12.5 | □□□□□□□□
15.5 | □□□□□□□
18.5 | □□□□□□□□□
21.5 | □□
24.5 | □

```

```

F2 believed to be same_var normal
(Level)suggest(F2, pooled, Level)

```

When an exact significance level is specified, the t-test is duly carried out. The F-test is not reported as it was not significant at this level.

.05 LOOKAT 'F2'

```

      ** 6.5
      9.5 *
***** 12.5 *
      **** 15.5 ***
      ** 18.5 *****
      21.5 **
      24.5 *
F2 believed to be same_var normal
suggest(F2, TWOT, 0.05)

```

The basic principle is that provision of explicit rules as an APL variable allows the human expert to fine-tune the system without getting into APL programming. In this particular case, one solution is to introduce the logical function

```

      V R←DEFAULT PASS;V;D
      A D is the default setting for variable V
[1]   V D←PASS
[2]   A('A'≡V)/'R+D 0 →0'
[3]   R←V

```

and reword the sixth rule

```

      STATDB[6 2 4]
AUNLESS RUNTEST AName FTEST ALevel
      STATDB[6 2 4 2]
      RUNTEST AName FTEST ALevel
      STATDB[6 2 4 2 1]
      RUNTEST AName FTEST ALevel
      STATDB[6 2 4 2 1 4]
ALevel
      STATDB[6 2 4 2 1 4]←c'DEFAULT' 'ALevel' '.05'
      CONSTRUE STATDB[6]
(OName)(NLevel)(sig_test(Name, TWOT, Level) ←
  (VERIFY(Name, same_var) ^
  VERIFY(Name, 2-sample) ^ VERIFY(Name, normal) ^
  UNLESS(RUNTEST(Name, FTEST, DEFAULT(Level, .05))) ^
  RUNTEST(Name, TWOT, Level)))

```

When we try F2 without a specified level we now get roughly what we want, starting with the exact F-test

```

      LOOKAT 'F2'
      ** 6.5
      9.5 *
***** 12.5 *
      **** 15.5 ***
      ** 18.5 *****
      21.5 **
      24.5 *
F2 believed to be same_var normal
suggest(F2, FTEST, 0.48)
;

```

and continuing with an exact t-test

```

      ** 6.5
      9.5 *
***** 12.5 *
      **** 15.5 ***
      ** 18.5 *****
      21.5 **
      24.5 *
F2 believed to be same_var normal
suggest(F2, TWOT, 0.00075)

```

at the price of an arbitrary 5% significance level buried in the axioms.

### 3. Restrictions and Precautions

#### 3.1 Utility Functions

The 'inference engine' PROLOGT has a trace facility built in. For instance, after it suggested an F-test for F2, we could ask why by responding with a query:

```

?
Depth 1 Rule 1
(⊃Name)(⊃Test)(⊃Level)(suggest(Name, Test, Level) +
  (sig_test(Name, Test, Level) ∧ SHOW(Name, Test)))
ΔLevel ← ΔLevel
ΔName ← F2
ΔTest ← ΔTest
Enter '?' for more

```

This first response would not be very informative unless we did not know the general structure of the expert system. We therefore probe further:

```

?
Depth 2 Rule 5
(⊃Name)(⊃Level)(sig_test(Name, FTEST, Level) +
  (VERIFY(Name, same_var) ∧ VERIFY(Name, 2-sample) ∧
  VERIFY(Name, normal) ∧ RUNTEST(Name, FTEST, Level)))
ΔLevel ← ΔLevel
ΔName ← F2
Enter '?' for more

```

Enter ';' to try for another proof

Certain functions are provided for getting out of trouble. An unwanted variable can be removed from both the workspace and the global memory by the FORGET function. The corresponding function that preserves the variable, but edits the global memory, is called FORGETABOUT.

It is quite easy to get embedded blanks in a rule. For instance, a reference to a logical variable 'Name' is entered as

```
'AName'
```

which can hardly be distinguished on the screen from  
' AName'

when in the middle of a complex nested vector. The function CHECK X verifies if X contains errors in notation, and the function POLISH 'X' cleans X up.

### 3.2 Shadowed Variables

The fact that it is the name of the data that is passed around the system means that the user cannot be allowed names that are used as local variables in the logic functions. There is no problem with local variables in the ordinary workspace functions, as these are fed the data itself. The restricted variables are:

```
AR; B1; BR; CGS; DB; DBI; GAOL; GI; GSUBS; LI; NEW; OG;  
OGCHK; RA; SG; USUBS; VCOUNT; G; L; R; S; T; X; Y; Z
```

Given that names of data variables, except perhaps for test data, should be explanatory, this does not seem too restrictive.

### 3.3 Logical Variables

APL logic reserves variable names that begin with a 'del' and frequently does a global

```
□EX 'A' □NL 2
```

erase on the lot. This means that the practice of using the same category of names for global variables is not feasible. My own practice is to replace the first vowel in the natural name of a global variable with a 'del', as with the global memory whose natural name would be PROP. I claim priority

for logical variables since, while it is necessary to treat them as a class, this is not needed for global variables.

#### 4. Punch-Line

The original statistical workspace took several APLers some months to construct. The expert system system took one man-week: one day to write the utilities VERIFY, SHOW, RUNTEST and of course LOOKAT, together with the first version of the rules and facts, and four days to sort out the unexpected consequences of the original version. This ratio is very much what has been reported for other expert systems. The total is incredibly quicker than rewriting the statistical functions in a logical language such as Prolog, even if efficient algorithms could be constructed, which I doubt.

Anyone interested in a beta-test version of the underlying system should write to the author.

#### References

1. Brown et al., 'Algorithms for Artificial Intelligence in APL2' in APL86 Tutorials, ed Camacho, London 1986, pp 87 - 150.
2. Prys Williams, A. G., 'Extending IBM's Logic Algorithms in Dyalog' in Vector Vol 4.2 (July 1987) pp104-122
3. Prys Williams, A. G., Providing Direct Execution Inside APL Logic Goals Technical Report, Department of Management Science and Statistics, University of Wales, Swansea SA2 8PP.

## APL and Relational Databases

Brian Loughran (INTI Ltd)

### Introduction

Once upon a time .... a leading UK manufacturing company had a problem perhaps not unfamiliar to Vector readers:

- an IBM mainframe was already fully loaded and committed growth would either reduce response times or generate substantial mainframe upgrade costs.

The cheapest solution of converting to popular PC packages was not sufficient for the "power users" (planning, pricing, sales departments) whose decision support systems consumed 50% of mainframe capacity.

### Users required:

- comprehensive facilities for departments to define and amend their own decision-support applications;
- a high level of data integrity;
- ease-of-use through full screen displays, automatic verification of input, context sensitive help;
- simple commands for selecting, sorting and displaying data for update;
- simple commands for defining complex reports and analyses;
- for experienced users and system developers, a command mode with advanced functions including an EXPRESS option to permit a series of updates, calculations and displays to be performed in a pre-defined sequence;
- data transfer between applications and with the mainframe;
- the flexibility to enhance their applications and analyses without fear of hitting an insurmountable barrier.

These requirements have been met with a package known as INTI (Integrated Information) which uses SQL and APL to do what each is best at.

SQL ..... manages the relational data-bases, including data tables which define the data, the applications, the input verification rules, the calculations for deriving results, the definitions of report layouts and all the corresponding documentation.

APL ..... manages the user interface (both in full screen and command mode) and translates simple statements of user requirements into the corresponding SQL statements, circumventing in the process, the many niggardly restrictions imposed by ANSI standard SQL. And, by having the full power of APL available if needed, the applications developer is assured that there can be no statistical or business modelling problem so complex that it cannot be handled within the environment.

### Design Principles:

**Data Integrity.** Any decision support system must confront the conflict between data integrity and data redundancy. For reports, data redundancy is essential, but redundancy threatens data integrity. INTI resolves this conflict by:

- a) storing each non-derivable data element only once in a pool of relational tables;
- b) for derivable data, storing not the data, but only the derivation rules;
- c) presenting the user with displays of redundant data but immediately updating the whole display whenever any data element is changed. Thus the redundant displays are always self consistent.

**Simplified Data Maintenance.** When a user updates data and STORES the changes, the new data become available to other applications. For example, if a new department were added, then all other users of the DEPARTMENT table would gain access to it.

**Ease of Application Development.** Each "entity" in the business has its own table, with each row having its own unique identifiers. New applications can be generated by "mixing and matching" selected attributes of existing tables, or new tables or columns can be generated without disturbing existing applications. The overall pool of data tables (the schema) can evolve to model whatever aspects of the business users may require.

When defining an application (SETUP), each new data element is given a name (unique within its data-table as required by SQL) and a number (unique within the application). Then the attributes of the element are given, including width, decimal places or data-type (character or date), heading (default for reports), format (default for reports), comment (a description for the system catalogue), and a data-table. Optionally, an element may have a derivation rule. Input validations are specified as table look-ups or

range checks. When SETUP is completed, INTI generates or modifies all the necessary data-base tables and indices through SQL.

The application SETUP is itself stored as a relational table so that, although the user perceives application definition as a separate task from data update, for INTI the same process is involved - updating an SQL table - and therefore exactly the same functions are used.

Thereafter, when defining reports or calculations for example, the user need only refer to the unique column number and all the information is already available for INTI to generate the appropriate SQL. Again, since all tasks are reduced to updating relational tables, a consistent user interface is maintained.

**Ease of Use.** The normal "end user" mode provides full screen menus and context-sensitive help so that users can run their applications with little computer difficulty.

A simple language is used to specify reports, calculations and graphics. This language permits column numbers, arithmetic operators, constants (numeric or alpha or date), variables and conditional expressions. INTI generates SQL statements which meet the user specification. For straightforward reports and calculations, SQL can do all the work. For requirements which SQL cannot meet (e.g. graphics, reports with control break calculations, or statistical analyses), INTI issues the appropriate SQL SELECT statement(s) and then manipulates the results.

**Power and Flexibility.** The COMMAND mode of INTI delivers speed and advanced functions to experienced users, including an EXPRESS mode. Moreover, the applications developer has access to a comprehensive set of tools, including:

- screen painting of menus, data input screens and reports
- device-independent graphics through a Dyadic supplied interface to GSL (Advanced Display Graphics Support Library)
- links to operating system facilities including mainframe transfer, networking and DOS conversion
- libraries for storing and sharing named APL objects.

Finally, for something really special, the applications developer can incorporate APL functions to build systems of any degree of sophistication.

## Conclusion

The first implementation of INTI has been on a network of two IBM 6150's using Dyalog APL. Applications converted from the mainframe range from simple clerical systems through to a complex suite of ten linked planning applications. Naturally, some lessons have been learned, especially about SQL.

In my personal opinion, the language is messy, verbose, non-orthogonal, full of petty restrictions and as user friendly as a wounded bear. Nevertheless, it exists: there is an ANSI standard and there is a lot of it coming (OS/2, Lotus, DBASE are all adding SQL database engines). Its disadvantages are minimised if we regard SQL not as an end-user language in itself, but rather a database handler to be embedded in another language.

Two major advantages arise from embedding in APL rather than the traditional compiler languages:

1. The APL interpreter and a good full screen interface, provide an excellent basis for end-user flexibility.
2. Developing and debugging the SQL generator part of INTI has been a time consuming task even in APL with its high programmer productivity. Anyone considering less productive environments should be aware that the Universe could implode in less than 150 billion years. It could be a race against time itself!

APL and SQL are both demanding of machine resources and the combination would probably have been prohibitively expensive before the arrival of the latest high powered workstations (and especially the new RISC chipsets). These provide the power and the price/performance advances that will enable SQL and APL, having been joined together in INTI - in accordance with the laws of Codd by the rites of Iverson - to live happily ever after.

## **Function-point Analysis and APL January 13th at the Royal Over-Seas League**

notes by Adrian Smith

### **Introduction**

Another well-attended and well-run meeting; what's more we even had the computing press along, and got a reasonably visible splash in 'Computing' as a result.

Romilly Cocking gave us an introduction to the benefits of FPA, based mainly on a recent conference in the USA. Alan Williams of IBM followed with a more detailed description of the technique, and finally Maurice Jordan (British Airways) gave us a brief case study of the use of FPA on a small APL project.

The overall message was that FPA tends to measure APL projects as around 10 times more effective than the typical 3GL approach. What no-one could agree on was how 'fair and objective' this measure is, when applied to an environment for which it was never designed. Still, it makes for good headlines!

### **The Benefits of Function Point Analysis**

talk by Romilly Cocking

"A slightly strange process of counting up system elements, assigning times, and adding them up"

"Function point is an objective way of showing that APL really is as productive as we've always suspected"

You can't manage what you can't measure. 'Functionality' is what FPA measures; this broadly corresponds with what you are giving the user. It also correlates with the cost of development and also with cost of maintenance. FPA is independent of the method of implementation; it is based on what the user sees: input, output, files, enquiries and so on. The cost of development can be given in 'Fn pts per man-week': for Cobol, figures are typically between 10-30; for APL, 300-500 is common.

All very nice in theory ... here are three concerns:

1. The method was originally devised and tested in the late 70's when computing was very much simpler than it is today. Life was about transactions, records and simple displays. Now we have graphics, relational data and so on. Do these invalidate the FPA validations?

2. FPA is not appropriate (and has never been advocated) as a measure of individual performance. A lot of APL projects are completed by 1 or 2 people!?
3. FPA is a terrible temptation. We've been looking for this for years .... of course FPA must be valid: it tells us just what we want to know!!

## **The Use of FPA Techniques within IBM**

Alan Williams (IBM Project Management)

### **Why Measure Productivity?**

Productivity is always one of the objectives of Systems Development; they might plan to improve it. What factors affect it? May be team size, environment are possibilities. To manage this improvement you first need an objective measure. Productivity is essentially the yield from the development process .... 'work product' / 'work effort' or 'output' / 'input'.

### **The History of FPA**

It all began in the mid 70's as a requirement within IBM for an estimating system. They looked at factors like environment, programming language, system size. By 1978 this had evolved into a need for measuring the output 'work product' independent of the environment.

The first thing was to decide what characteristics the 'function point' should have:

- it should be based on the user's view of what he gets
- it should be independent of technique or environment
- it must measure the output from the whole process, not just 'analysis' or 'coding'
- it needs to be measurable early in the development process
- it must be easy to count!

So ... what does the user get? He gets: input, output, stored data, enquiries, processing. This is how you count them:

1. Decide what the system is. Draw the boundary.
2. Count the Inputs, Outputs, Logical Master Files, Interfaces, Enquiries.

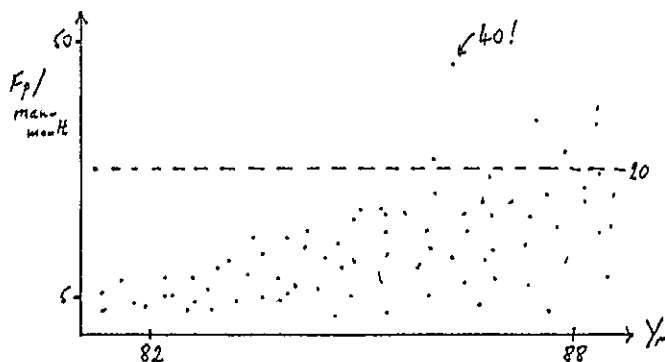
3. Categorize these as Simple, Average, Complex.
4. Apply standard weights.
5. Add 'em up. This is called 'the unadjusted function count'.

Processing complexity is assessed on a score of 0-5 for 14 factors (e.g. distributed, complex scientific) and added up to give a maximum score of 70. This is then normalized so that 1 represents a system of average complexity, and used to multiply the count resulting from item 5 above. The result is a 'magic number' which has no meaning on its own.

### When Do You Do It; What Do You Do With It?

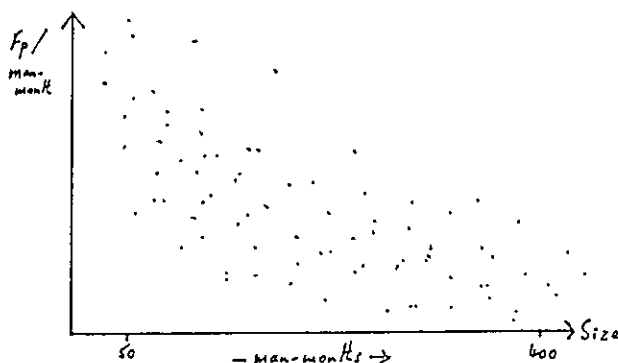
The FPA is best done at two points: at the end of the external design, and when you implement the system. It is interesting to compare the two! ("I know we were two months late, but look ... you got 200 more function points.") The FPA counts can be done by questionnaire or some kind of automated tool; it should be agreed with the user, and checked by an FPA co-ordinator to ensure consistency.

The data can be used to measure Fn-pts/Man-month during development, and Man-mths/FP during maintenance, both over a long time period. The data is relatively dirty, and to see anything but noise you need to take a long view, say several hundred projects over at least 5 years:



For conventional developments the figures fall between 5 and 20, with the odd outlier as high as 40. Often these outliers tell you something interesting! In the example given it was all PC-related code for a front-end to an existing package.

Another useful plot is to show the productivity as a function of project size ...



... this is sort of  $1/x$  but flattens out above 50 man-months. One consequence of this effect is that IBM now limit individual projects to a maximum of one man-year.

### Current Status of FPA

FPA is the IBM internal standard for application development, although not for systems software ... what is an 'output' from MVS (!). It is mainly aimed at business-type systems and is much less good where there is complex processing on little data.

The technique is widespread in the USA, and spreading in Europe. However it has the drawback of being impossible to update, as this immediately loses the benefit of consistent historical comparison. In response to Romilly's three points:

1. Is it 'old hat'? The basic paradigm of 'Input & Output' hasn't changed, but how do you measure the 'Output' of a graph in terms of information content. However people are still making good decisions based on FPA measures.
2. Measurement of individual's performance. This has never been done within IBM mainly because of the management culture and style. It can cause people (and teams) to fudge the figures! All you are trying to measure is team performance, however there is nothing in the method which prohibits a 1-man team.
3. It is not clear how APL could skew the analysis, so as far as we know the method is as valid for APL as for anything else.

If you want to know more, you need the "Productivity Measurement Guide" IBM reference SH19-6387, which fully describes the FPA technique.

## A Short Case-study in British Airways

talk by Maurice Jordan

FPA is a technique which makes it simple to involve the user, but is broad-brush; it may be very misleading for small amount of data. However, British Airways like to make things measurable, and when you see figures like:

Environment	Fn pts / man-month	
	Average	Maximum
3GL	3-30	20-50
4GL (Focus)	50-120	
APL	200-300	300-500

... based on largely anecdotal evidence, you want to know more. The correlation with the size of task and with programmer experience (who got the results at the 500 end?) would be interesting. It would also be fascinating to know what the 'productivity surface' looked like ... there are lots of non-linear functions in there!

### An Example

This really falls outside the true domain of FPA, however it is typical of the things we do in APL. The system involved 12 new functions ... the application was really made up of 8 of these (170 lines) with another 4 fns (270 lines) generated automatically to handle screen I/O. The data was held in blocked inverted files, with non-zero values compressed (say 80 mbytes per file), keyed for fast access. The screens show: (a) the file contents; (b) a series of sample compressions to help to select the best algorithm. Typically, picking the best algorithm can save some 42% of the current file space. The other info shown gives: (c) details about the keys; (d) a summary of the file blocking. There is help behind every screen panel.

The unadjusted function point count was 54. The process complexity was 33 (out of 70), giving an adjusted count of  $54 \times 0.98 = 53$ . Not counted were: reports (these are automatic, and users may not actually want them); the help screens. These yield up to 40 further function points, which are not claimed in the above total.

How long did it take? The activity-log says 5 days, with a lot of work on 4 of those days. This gives a figure in the 200-250 FP/MMnth range, which looks reasonable. Other very small systems have been in the 300-400 bracket.

## Discussion

I think most people at the meeting agreed with the basic premise that FPA was measuring something sensible, and was a worthwhile technique for measuring APL systems. However the idea of using the figures to promote the case of APL as the new 'wonder machine' was not accepted; we would all like to see the evidence from a decent-size project or two! Maybe APLers have the luxury of being able to choose what they work on, and gravitate to 'interesting' systems (those with an intrinsically high fn-pt count) which they know they can do quickly? Maybe APL just attracts the brightest brains in the business, and we would still score high in COBOL if that was all we had? Yes, MJ's comparison is a valid measure of something, but we don't quite know what!

---

## **APL: Future Directions February 17th at the Royal Over-Seas League**

a brief report by Adrian Smith

We should all thank Jonathan Martin for both organising this excellent meeting, and getting the publicity out in time to get the attendance back up to scratch. Several of the speakers provided notes in advance, and all promised a speedy summary of what they should have said. These notes will all appear in the April VECTOR, so I shall confine this report to the significant news items, and some of the better quotes.

### **Romilly Cocking (representing STSC and C&D)**

"If it really were my APL, I would be rich and famous, rather than just notorious"

STSC's plans include: version 2 of PLUS II, with  $\square$ NA to interface to C and assembler (Spring 89); PLUS/VMS release 8 also with  $\square$ NA (late 89); distributed sharefile (multiple mainframes 1st, then PC networks); compatible graphics PC/VAX/Mainframe (as long as you are using the PC as your workstation).

C&D's plans are for a link from mainframe APL*PLUS to Oracle (late 89) and to DB2 (SQL/DS) in early 1990.

Further ahead, there lies OS/2. Maybe 2 years away ... nobody is yet making serious use of this, however:

"By the time that situation has changed, I'm confident that STSC will have a product"

At the moment mainframe APL*PLUS is still based on VS APL, which is old technology. It will eventually move to a mainframe version of the UNX/VAX, II system.

**Peter Donnelly (Dyadic Systems)**

A full Oracle interface will be available to all versions of Dyalog APL (the current is only to the IBM badged version of Oracle). Concerning this interface:

"The thought of what people are doing with this terrifies me, but it seems to work"

□SM to define and manipulate the screen (before nested arrays you couldn't do this), and a really exciting new session manager:

"We have a strange relationship in Dyalog between the commercial and the technical people: the technical people never do what they are told to do. Fortunately they tend to come up with something better!"

Dyalog APL for PC DOS is in preparation (386 machines only). This will have □SM, the new development environment, and CGI graphics. It will lead on to an OS/2 version, to be developed by Dyadic in collaboration with MicroAPL.

**Richard Nabavi (MicroAPL)**

Phar Lap and IBM's DOS extender are very roundabout ways of accessing memory. Why not use OS/2? Because it's not a 386 operating system, but being cleverer than DOS it can stop you doing all the naughty things you can get away with in DOS! Let's wait for OS/2-386!

Meanwhile there is APL68000 for the smaller machines, and further work on the interfaces into the Mac environment. In the long term this is the approach which will be needed for OS/2 Presentation Manager ... however writing this kind of software is not easy: OS/2-PM demands a very new approach.

**Mark Turner (IPSA)**

There will be continued emphasis on 'Dictionary' APL, with Sharp APL m/f converging on SAX. Future systems will need to support 'massive' user loads, say using a file system over multiple machines for 10,000 - 15,000 users. No longer can the system rest for 8 hours on Sundays: it will need 24-hr operation with online dumping, and the ability to survive at least one dead CPU.

# ADVERTISEMENT

▽ Z←PK64;R;X;S;NUB

[1] R YOU HAVE:

[2] R YOU MIGHT HAVE AN:

[3] R COULD YOU BE:

[4] R THEN CALL

[5] R OR WRITE TO US AT

[6] R←DFX 2 15 ρ'Z←NUB W Z←((WlW)=lρW)/W'

[7] X←(Xl;11=' ')ΦX←PCR 'PK64'

[8] R←NUB '<?',,X

[9] Z←((12ρ2)τ2342)τ 5 47 ↑ 1 2 ↓X

[10] S← 7861 17325 26512 19925 13989 29022 21328 12989 17363

[11] S←S, 16909 1644 16718 16926 21916 5813 31693 17148 23938

[12] S←S, 11792 5808 23984 11248 27152 24368 14000 30032 23760

[13] S←S, 17136 17296 1872 16816 17264 8720 16880 19120 24496

[14] S←S, 21712 11120 12624 17008 16848 16816 17392

[15] Z[2 5 8 ;]←(3,-1↓ρZ)↑R[(3ρ32)τS]

[16] Z[10;10+132]←(48ΦR)[Rl'(L(α5:ρGW:)G)AMACGL:G;ΦRO?YG406Φ']

[17] X←'F↑8UIαIIAXYOEUGαS4IU4C]TαXCαXULFZU↓TAUUUUUUUUU'

[18] Z[11 12 ;18+123]←(61ΦR)[Rl 2 23 ρX]

[19] Z←Z,[1147↑Φ')SIHT EKIL EDOC ETIRW UOY FI TON TUB('

▽

---

**GENERAL ARTICLES**

This section of VECTOR is oriented towards readers who may neither know APL, nor be interested in learning it. However we hope you are curious about how, under the right conditions, such impressive results can emerge so quickly from APL programmers

---

## APL-PLUS Runtime Assurance Quotation Services

by Harry Phillips

Three years ago, an assurance company wanted to design and launch a new product for employers leaving firms and taking their pension benefits with them in a "Section 32 Buy-Out". In order to make the contract financially attractive and capable of matching a wide variety of existing pension schemes, it had to rely on complex actuarial logic. The resulting procedures and calculations could not be managed manually. Although a computerised quotation service was required before the launch in two months time, the marketing staff were free to modify the contract up to the last minute.

The cost and delay associated with a Head Office quotation service meant that a DIY facility in the hands of our local staff or even our clients would give us a competitive edge.

Initial decisions to be made:

1. System to be constructed by computer or user department.
2. Mainframe or Micro. system.
3. Batch or Interactive Procedures.
4. Software to be used.

If the system were to be built by the computer department it would have to be scheduled in with other work and a fixed specification provided. The time limit meant that work had to start immediately without an agreed specification. Thus it had to be a user department job. Batch processing procedures are, by their very nature, unsuited to a quotation service supporting salesmen in a competitive market. Interactive procedures giving immediate results were required if at all practical.

A system on line to the mainframe would restrict usage to those with the right equipment, only be available during normal office hours and offer variable response times. The computer department would want to know what volume of traffic to plan for and what response times would be required. With this information they would quote a price and ask for authority to buy the extra hardware. Since answers to these questions were not known and negotiating time was unavailable, a PC based system was examined. This would run on any IBM-PC compatible machine, any time anywhere and give known response times.

The choice of software was made by a process of elimination:

- a) Since support for the volume and variety of existing software was a major drain on resources, any proposal which added to the problem would be questioned. APL was well established on the mainframe and its ever growing cost was

prompting requests reconsider its use. Developing a quotes programme in APL for the IBM PC would be a step in the right direction and would not be resisted. This eliminated introducing 'C'.

- b) Since the quotes service had to run on up to 200 remote PC's in our sales offices or the clients office, the cost of running the service per PC was important. Few new PC's would be required as many were already in place and the volumes per machine would be small. APL-PLUS offered a run time version whereby we could obtain 200 copies at a reasonable price. Our marketing department were happy to foot the bill in advance so that the quotes service could be distributed without recipients having to buy software. We also had to agree to deliver in 2 months. This eliminated the IBM version of APL for PC's and most integrated packages.
- c) Speed of system development was critical and the complexity of actuarial logic was a major consideration. Experience had shown 'BASIC' to be too slow in development and 'Dbase' inadequate for complex calculations. The staff who were to develop the quotation service had substantial expertise in APL and had access to actuarial subroutines already written for mainframe systems. They were happy to use APL on the PC as long as we provided an XT with a maths co-processor as the development machine (that dates the project).

Having chosen to base the quotation service around the IBM PC using the run time version provided by APL-PLUS we had to solve the following:

- 1) The end product must fit on one floppy disk for distribution. This must include operating instructions and help screens, so that paper instruction could be restricted to a single sheet.
- 2) The service had to run on a variety of configurations, twin floppies and hard disk, colour and monochrome, most common printers, with and without a maths co-processor. Speed on the slowest configuration had to be reasonable.
- 3) Since the actuarial terms and conditions, including the calculation of all rates, were to be incorporated in the programme, security was essential. The ability to quickly change terms and conditions without recalling disks was also critical, particularly when interest rates are volatile. The actuaries had to be convinced that they still had complete control over the terms and conditions being quoted on behalf of the society.

These problems were all overcome and the new contract launched with its supporting quotation service operational from day one at Head Office. Distribution of floppy disks soon followed after the contract stopped being modified. The framework which we developed for this contract has been extended, and now offers a series of pensions contracts via a menu.

# MicroAPL for...

## APL on the widest range of micros

### PCs

#### MultiAPL - APL.68000 under PCDOS

Sample configurations:

68000/1MB	£ 995
68000/1MB/Maths processor	£1295
68020/1MB/Maths processor	£2995
68020/4MB/Maths processor	£5495

Prices include: Plug-in card, APL.68000, Keyboard stickers, Manual, STSC emulation software

STSC's APL*PLUS £425

STSC's APL*PLUS II PC £1395

### Single user micros

APL.68000 for the Apple Macintosh	£86.91
APL.68000 for the Apple Macintosh II	£260.82
APL.68000 for the Atari ST	£86.91
APL.68000 for the Commodore Amiga	£86.91

Prices include: APL.68000, Keyboard stickers, Manual, Reference card

### Multi-User systems

APL.68000 runs under the following operating systems:

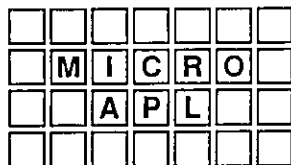
Unix, CPM/68K, Wicat MCS, Mirage

on computers which include:

Sun, NCR, Wicat, HP200/300, Spectrum, Aurora, Stride

Prices range from £1200 to £2000

All prices quoted exclude VAT



MicroAPL Ltd.  
South Bank Technopark  
LONDON, SE1 6LN  
Telephone: 01 922 8866

Access and Barclaycard  
Orders accepted

## Case Study: How Expensive is APL?

by F.H.D. van Batenburg and J.W. Kijne

### Abstract

This paper describes our experiences with APL in a scientific project: investigation of the growth of root hairs of an agriculturally very important plant family. We introduce you first to our institute and to the people who work there. Next, we paint the puzzle we wanted to solve. We will show you how we gradually achieved a small but important step by means of simulation. Finally we will discuss the value of APL in acquiring our results and we will make an educated guess how this could compare to the use of PL/1.

### Introduction

The Institute for Theoretical Biology forms together with other biological units the Department of Biology of Leiden University, the oldest University of the Netherlands (1575).

The people who populate the Institute for Theoretical Biology (ITB for short) form a mixed lot of scientists. One finds among the permanent staff a mathematician, a statistician, 2 computer scientists and a philosopher of biology. Together they strive to solve biological puzzles in a fruitful mixture of biological intuition, mathematical or computer skill and solid experimental data.

In order to expose the strong and weak points of APL, we want to introduce you first to one of the scientific projects in which we were involved in the past five years. We will introduce you to the biological background briefly and present some solid data about the requirements of each project next. Next we intend to discuss the various characteristics in which APL distinguishes itself from other languages. Because of the proficiency in PL/1 of the first author (17 years of intensive use) we will confine ourselves mainly to comparisons with that language. We have no doubt, however, that our observations can be generalised to many other "conventional" languages which are closely related to PL/1 like ALGOL, BASIC, C, FORTRAN, PASCAL, and many others. Finally we draw up the balance sheet of the costs of these projects.

To make our point(s) stick, we should have data at our disposal of each project carried out both in APL as well as in another language. As you probably surmised, unfortunately we can only provide solid data about the application of APL. However, we intend to make those comparisons nevertheless by making educated guesses at the PL/1 equivalent. Although the lack of real experimental data is detrimental to an absolute belief in the

correctness of the comparisons, seventeen years of PL/1 proficiency make up at least for enough backing to make them plausible.

### The Problem

In 1982 the first author started a project with J.W.Kijne of the Department of Plant Molecular Biology. In that department, research was underway to study the processes involved in infection by Rhizobium bacteria of leguminous plants. These plants count many important species among them like alfalfa, pea, bean, clover, lupin, soybean and many other plants which provide food for humans and for cattle. These plants are important because of their nitrogen-fixing ability, due to symbiosis with Rhizobium and Brady-rhizobium bacteria. Although nitrogen is indispensable for plant growth, plants are unable to absorb free nitrogen and they get their nitrogen from nitrates instead. So when the soil is exhausted of nitrates, farmers who are unable to buy sufficient N-fertilizer grow legumes before seeding the next crop. By ploughing in these legumes, they enrich the soil sufficiently well by bound nitrogen to allow for another type of crop next year. So the process of binding free nitrogen is very important for agricultural food production.

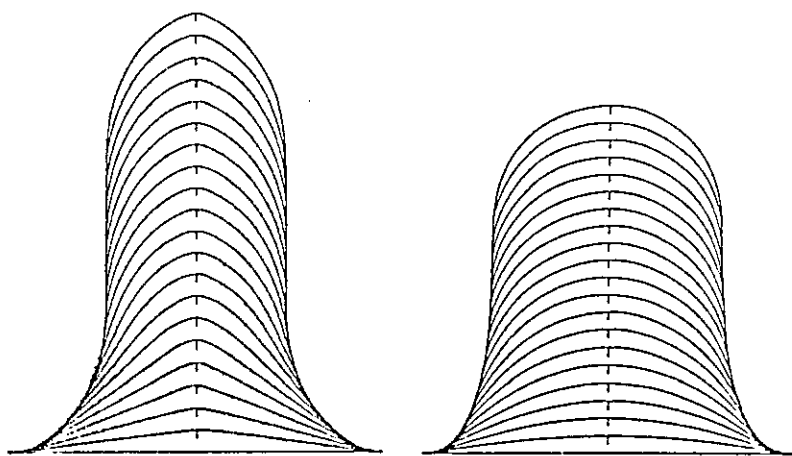
Rhizobial infection of legumes is essential for establishing a nitrogen-fixing symbiosis. Some legume varieties appear unable to get infected by Rhizobium and consequently those plants are unable to bind nitrogen.

Rhizobia infect the root hairs (root hairs are small outgrowths of root epidermal cells). The morphological result of such a root hair infection is curling and modified growth of the root hair. Such a marked curling is called a "shepherds crook" after its form (see fig.2). Such a prolonged curling finally results in development of a small nodule. These nodules are the small factories in which nitrogen binding takes place. Also, during that process of curling a very small filament (called "infection thread") is growing inward (see fig.2).

At the moment at which the first author joined the team of the Department of Molecular Biology, Jan Kijne was developing alternative theories about the mechanism which could explain the curling of root hairs. Could it be, for example, that one of the bacteria attaches itself to the root hair wall and starts to stimulate the growth excessively at that spot? Would this result in the formation of a shepherds crook? And how would that interfere with the regular growth which is concentrated at the top of the root hair? Or alternatively, could an attached bacterium bring regular growth to a complete stop one-sidedly? This would force the opposite side of the root hair to overwhelm the side of the bacterial infection.

### The Answer

In order to analyse the consequences of these hypotheses we developed a simulation model in APL. At first we simulated normal growth of a root hair under various assumptions of tip-growth. See fig.1 for the results.



**Fig.1: Simulation results of model 1 with normal growth; each line is the cell wall at consecutive time-steps. Left a triangular, at the right a cosine shaped growth gradient.**

Next we started to introduce the growth of an infection thread. Our idea was that the growth of an infection thread itself would modify root hair growth and would automatically result in a shepherds crook.

Fig.2 shows such a shepherds crook and in the centre of the root hair one distinctly observes the infection thread.



Fig.2: A roothair of Pisum sativum showing a shepherds crook and a distinct infection thread.  
Photograph by Dr.J.K. van der Have.

Several hypotheses about the possible mechanism of infectious thread growth were examined. For example, we considered a local inversion of the normal growth gradient, a local diminishing of growth, a local stimulation of growth, and several others.

However, none of them proved to be satisfactory (see fig.3).

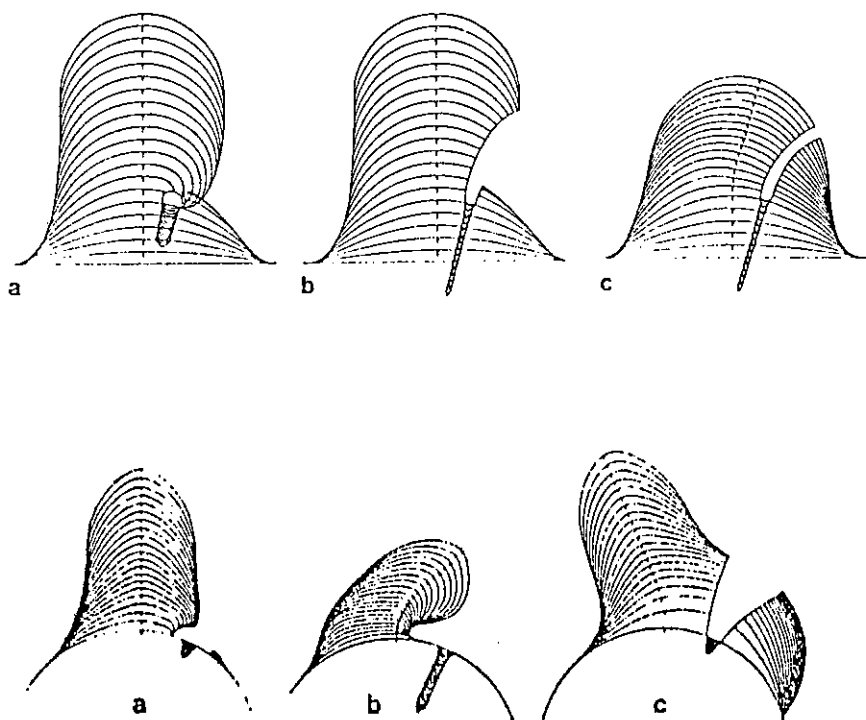


Fig.3: Simulation results of model 2 with infection thread.

Although the results were unsatisfactory because apparently our ideas didn't conform with the real mechanism, these simulations were valuable nevertheless. Because, next to refuting our ideas, the results also showed us the flaw in our previous reasoning: if standard growth of the top continues (as we assumed in all hypotheses) then the top will gradually move away from the infected spot (see fig.3). This does not conform to what we observe in nature. So, apparently the bacterium cannot operate locally and more or less independently of the normal top growth; it must influence the standard top-growth much more severely.

It took our group a while to reassess the situation. Apparently we should look for a mechanism in which the bacterial infection actually competes with the top over the growth process itself. Our new ideas were incorporated into the simulation model. The results are depicted in fig.4 and fig.5.

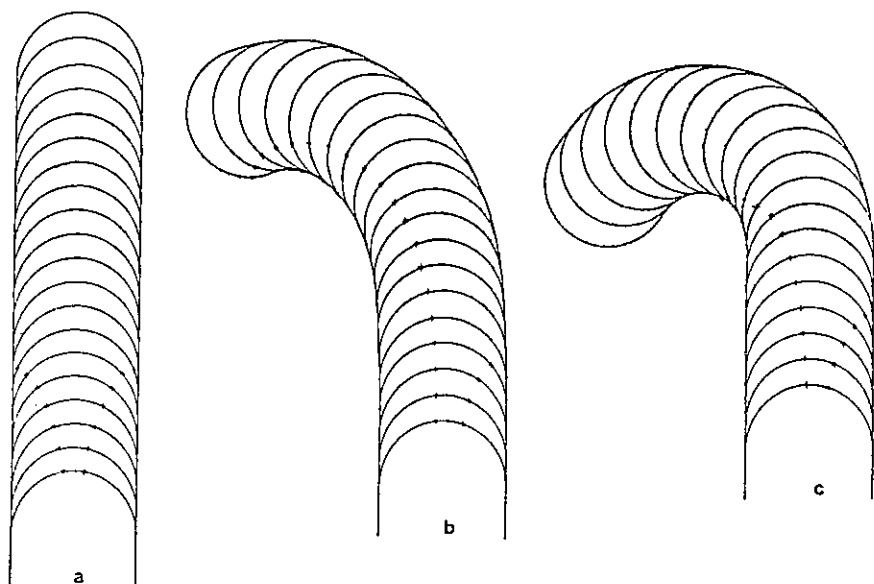


Fig.4: Simulation with competing attractions  $a = 5:5$ ,  $b = 5:1$ ,  $c = 10:1$ .

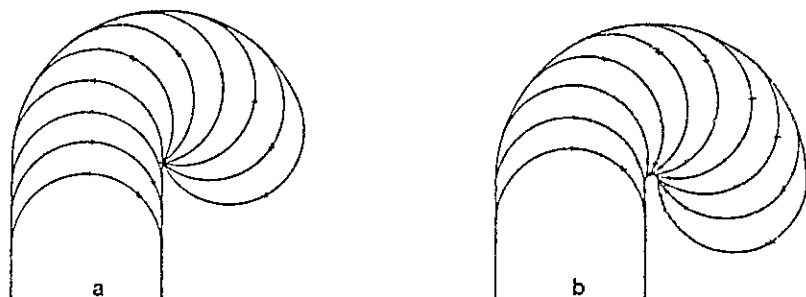


Fig.5: Simulation with bacterial competition against a central growth mechanism  
a = small influence range, b = large influence range.

Comparison of fig.5 with fig.2 shows that the newer proposed mechanism could explain the growth mechanism involved in root hair curling.

At this moment, all our ideas are purely theoretical. However, the simulation facilitated the design of an experimental approach by elimination of some improbable hypotheses. So, here ended the involvement of the first author (as well as the application of APL), because the next stage of the research was to look for the physiological and chemical basis of the proposed mechanism.

## APL

The whole simulation program was developed in APLSV and run on an Amdahl V7B. In this paragraph we explain in what respect the choice of APL was fortunate.

First we will give you an indication of the size and composition of the final system. Next we consider the characteristics speed, development time, understandability and costs.

## System Composition

At the end of the project, the simulation system was composed of 48 functions, totalling 729 lines (of which 253 were comment-lines, and 476 executable lines). That is an average of 15 lines/function, but a better indication of size is the median (7.5) because a couple of big utility functions from the public libraries were included.

```

      0          10          20
      |.....|.....|
1:
2:XXXXXXXXXX
3:XX
4:XXXX
5:XXX
6:XXX
7:XX    <=== MEDIAN
8:XX
9:XX
10:
11:XX
12:X
13:
14:
15:X    <=== MEAN
16:XX
17:
18:X
19:
20: outliers: 22, 22, 24, 30, 32, 36, 37, 37, 43, 45, 45, 48, 86
      |.....|.....|

```

Fig. 6: Distribution size (number of lines with comment included) of functions.

## Speed

One of the first objections to the choice of APL could be its speed (or rather its lack thereof). The very thing which simulations require is speed, so one might think APL is too slow to be useful in simulation. Our experience was different, however. Thanks to the interactive nature of APL we could easily explore the various ideas. The computation and drawing of a full-time-development of a root hair took between one and two minutes. This yielded sufficient information to ponder upon for several minutes, or more often for hours, or even days. Besides, many blatantly erroneous results were quickly discovered in the first 10 seconds and the execution could be interrupted immediately.

The alternatives available at our computer centre at that time were batch processing in ALGOL, FORTRAN or PL/1. As the turn-around in those languages took several hours (compared to the few minutes by the use of interactive APL) for correct as well as for erroneous simulations, a run would take several hours before the results were available.

This is not so much of a problem for runs which require a long time to ponder upon afterwards, but it is very frustrating for a group of associated runs in which the parameter setting of each run depends upon the results of the previous run. For such a series of runs, an APL session of a few hours would probably require a week by application of PL/1.

So even though APL might be slower than any other available language, in our situation we couldn't care less because the throughput of the alternatives was considerable worse.

### Development Time

As the previous exposition already hinted, there were no clear waterfall-type (Boehm,1976) phases in the software development. After an initial development-phase, and a subsequent implementation phase, the final use of the model was also a continuous alternation of use, assessment, adaptation (or augmentation) and use again. So we had no clear mile-stones marking the beginning and end of development. Nevertheless an educated guess of all the time devoted to development can be given as slightly over 1 man-month.

Although we did not repeat this project in another language, the first author feels sufficiently knowledgeable in PL/1 (after 17 years of extensive experience with that language) to make an educated guess about the development-time in PL/1. If he was very lucky, the development would take half a year, but a more realistic estimate is about a year or more.

### Understandability

It is often said that "APL is a write-only language". Our experience taught us differently. The student Richard Jonker joined the project as a rather inexperienced APL programmer. Nevertheless he understood the root hair program-system within a week and was even able to make his initial modifications after that first week. Remember; this is a system of 476 executable statements (which may roughly amount to 5000-7000 statements in a conventional language, say more than 100 pages). This was only possible, of course, by applying a rigorous modularity of small statements without spaghetti-code.

At the moment we enforce this spaghetti-less code by only permitting jumps in either one of the following constructions:

```

LOOPS:      ---WHILE---                      ---UNTIL---
            B1:→(CONDITION)/E1+1             B2:...
            ...
            E1:→B1                           E2:→(CONDITION)/B2

---IF BLOCK---
            B3:→(CONDITION)/E3+1
            ...
            E3: ...

---IF STATEMENT---
            BE4:→(CONDITION)/[LC+1] ◇ CONDITIONAL EXPRESSION

```

The first line of each block is marked with a "B.:" label, the last line (inclusive) with an "E.:" label.

### Costs

The final aspect we would like to consider is costs. As an interpreter, APL is much more expensive than a compiled program.

Again, however, such an obvious statement is not that obvious in closer inspection. What are costs? One should consider two different costs: CPU costs and manpower costs.

1. CPU costs. During development in APL small errors are corrected immediately and the program is continued without extra CPU costs. In PL/1 the majority of syntax errors are discovered in the first run (unless some errors abort compilation). Other types of errors, however, are found one by one. Each discovery takes a new compilation after correction. These compilations are very CPU-intensive. Of course, one might reduce these compilation costs by clever partitioning, but this probably raises linkage-editor cost and manpower-costs for sure. So we surmise that APL is not more expensive than PL/1 during development, probably even cheaper.

Contrary to our initial expectation, APL was much cheaper during execution. First, because in APL we could interrupt many obviously erroneous or uninformative simulations. In our PL/1 batch-environment all those unfruitful simulations would have run their due course, before we discovered the error of our ways.

Secondly, APL was cheaper because we did not have a long sequence of execution runs with the same program, but a continuous cycle of development and execution and development again. Granted, for many runs only some parameter values were varied; but every so often we varied the very code itself. This would require new, expensive compilations in PL/1.

Although we cannot retrieve the CPU costs of this project we do know the costs of a comparable project. This was about Dfl 5000 (about \$2500) and we estimate that this project cost Dfl 5000 too (for development and for all executed simulation runs together).

2. Manpower costs. The CPU costs, however, are inconsequential compared to the manpower costs for program development. Manpower costs in this APL project were about Dfl 8000 (about \$4000). Now, my experience with PL/1 and the opinion of several of my colleagues gives reason to estimate that development in PL/1 is about 5 to 10 times as slow. So for development time in PL/1 this would amount to Dfl 48000 - Dfl 96000.

Note that during execution the PL/1 equivalent would take considerable more time for each run. This was not so much of a problem during runs with interesting results which forced us to reflect a few days, for other runs it was very frustrating at least. However it would not increase the costs, although this would slow down the turnaround time of the project considerably.

So APL proved to be much cheaper than PL/1 in this project, mainly because of the manpower costs.

## Conclusion

The previous paragraph showed the benefits of APL clearly.

The first benefit was obtaining results faster, due to the interactive execution (and subsequent correction or improvement in obvious erroneous or uninformative cases). Also, APL was cheaper because at the first hint of an unsuccessful result, we could decide to steer away towards another idea, whereas in batch many uninteresting cases would have run their due course.

The main benefit was the very fast development time. We showed that this was not cancelled out by a supposedly slower execution time during the simulation itself. We expressed this benefit in terms of money; the use of APL costed about Dfl 13000 (of which Dfl 8000 manpower during development), the PL/1 equivalent would probably require Dfl 45000 (of which Dfl 40000 manpower costs) or even more.

## References

1. Batenburg, F.H.D.; Kijne, J.W.; Iren, F.v.; Libbenga, K.R.L. (1983): Simulation of marked root hair curling in Rhizobium-legume symbiosis. *Physiol.Plant*(59)363-368
2. Batenburg, F.H.D.; Jonker, R.; Kijne, J.W. (1986): Rhizobium induces marked root hair curling by redirection of tip growth: a computer simulation. *Physiol.Plant*(66)476-480
3. Boehm, B.W.(1976): Software engineering  
*IEEE trans.on computers* c25(12)1226-1241
4. Bosch, F.v.d.; Metz, J.A.J.; Diekmann, O.(in prep.): The velocity of population expansion. *J.Math.Biol.*
5. Metz, J.A.J.; Diekmann, O. eds.(1986): The dynamics of physiologically structured populations. Springer, Berlin (Springer Lecture Notes in Biomathematics 68)
6. Metz, J.A.J.; Roos, A.M.; de Bosch, F.v.d.(1988): Population models incorporating size structure: a quick survey of some basic concepts and an application. in: Ebenman, B. & Persson, L.(eds.), Size structured populations; Ecology and Evolution. Springer, Berlin.
7. Zandee, M. Roos, M.C.(1987): Component compatibility in historical biogeography. *Cladistics* 3(4)305-332

F.H.D.van Batenburg, J.W.Kijne,  
Institute for Theoretical Biology,  
Dept. Plant Molecular Biology of Leiden University,  
Groenhovenstraat 5,  
Nonnensteeg 1,  
2311BT Leiden,  
HOLLAND

## Generic Local Objects

by Zdenek V Jizba

### Abstract

A proposal is made to provide a facility to define classnames in the header of a user defined function. Rationale for this proposal is based on requirements for building Expert Systems with APL.

### Structure of Expert Systems

An Expert System Shell is a collection of specialized software to support the many different types of activities that such a system requires. For example, a shell allows for the maintenance of "rules", supports one or more "Inference Engines", and provides for communications with external data sets. Of course it also supports many other functionalities.

To build an Expert System, the Knowledge Engineer starts with the shell, and adds specialized information, also known as the "Knowledge Base". This information usually consists of a collection of rules to define the behaviour of the system, special hierarchical dictionaries to relate various technical terms of the domain, and may also include specialized software.

Over the last decade I have been studying the possibility of using APL (and APL2) to build such Expert Systems. As I gained experience, a number of concepts have evolved that integrate the unique aspects of APL with the paradigms that are commonly used in the field of Artificial Intelligence. It appears that no matter what the computer language, the specialised needs of a new paradigm always bump against the limitations imposed by that language. The solution of these problems has often in the past been to design new languages. I believe that a much better solution would be to expand the scope of existing language. This is especially true of APL.

The STSC APL*PLUS for the PC supports a number of features that facilitate the development of Expert Systems. There are, however, a number of features that would enhance even more the power of APL in building applications in the field of Artificial Intelligence. This note describes one such feature. It deals with a way to define classes of local names in the header of a function. Although I give a specific proposal, the form is not particularly important. What is important is the underlying concept. In what follows, I attempt to explain my justification for the need.

## Shell and Domains

An Expert System deals with one or more domains of expertise. (These will hence be called just domains.) In its APL implementation, a domain may consist of a collection of APL functions, rules, variables, dictionaries, and other data structures. A cardinal rule of good Expert System design is a complete separation between the shell and the domain. This means that before the domain is invoked, the shell must not have ANY domain specific objects. When the domain dependent task is activated, the many domain objects are brought into the workspace, or are created. On completion, the results of the process are stored in appropriate objects, and the domain is wiped out from the workspace, returning to the clean initial state.

One might ask, why this need for separation? There are many reasons, but consider a complex problem that requires not one, but a series of domains to be applied sequentially. Then add the requirement for any one domain to call another domain as a kind of sub task. In such an environment any attempt to maintain global domain objects (functions and variables) would create chaos.

There is a simple way to separate domain objects from shell objects in APL. All one needs, is to define special domain specific functions called "drivers". A driver function includes in its header the names of all domain specific functions and variables. Unfortunately, there is a limit on the length of defined function headers. This is a significant constraint in the design of some domains.

The need for restricting object names is not limited to domain drivers. An inference engine is a global object in the workspace, but it processes rule sets that are local to the domain. Such rule sets typically require communication of data between different rules. This can be handled by creation within rules of variables that must be global to all rules, but local within the function driving the rule set. Here again the natural APL solution is to define specific variables that are local to the APL inference engine driver. The more of these variables that can be defined in the header, the more flexibility one has in designing the rule set.

Individual rules also may be quite complex. In my approach, a rule contains at least three components. One is a (generally unique) tag that identifies the rule, another is the antecedent, and one is the consequent. Both the antecedent and the consequent are valid APL expressions. In the antecedent there may be any number of expressions, including assignments, separated by the APL*PLUS diamond. (Only the last expression must return a result. This result must be a boolean scalar.) The consequent may also contain multiple expressions separated by the diamond.

This structure and form of rules allows generation of variables that can be used in making decisions. These variables form a kind of scratch pad memory and should be local

to the APL function that processes a single rule. This function, therefore, must also include local variable names that can be used only as long as that rule is being evaluated.

One way of thinking of rules is as a special type of defined function. A rule has a name, produces no result, and has no loops or branching other than a decision whether to evaluate the consequent. A rule also produces side effects that are global to the rule, but local to the rule set. Yet, a rule is not a separate object. It is a part of a larger object called a rule set. In my implementation, rule sets are defined as segmented strings (see APL*PLUS for definition of segmented strings). There is a segmented string for each component of a rule set. Rule sets are stored in component files when under development in the shell. For run time versions, rule sets can be brought into the workspace as segmented string variables.

I have named three types of APL defined functions that require localization of generic object names. These are the drivers for specific domains, inference engines to process rule sets, and individual rule processors. In some of these, the number of named objects may be so large that the current limit on the length of the header may be inadequate.

### Implementation of Generic Variables

Although I have been severely constrained by the limit of header length, so far I have found ways of getting around such limitations. In general, I give single letter names to generic variables, and three letter names to domain specific APL functions. Within the inference engine function, eight single letter variables in lower case are global to all rules within a rule set. Within the rule evaluation function, another eight upper case letters are used for scratch pad memory. (These two functions also require many other local object names that are specific to the function task, so the header is quite long.) The remaining upper and lower case single letters are available for domain specific variables.

When a new domain-specific APL function is defined, the domain driver must also be re-defined by adding the new function name to its header. The access to the workspace of the newly defined function must also be defined. Some of this extra effort could be eliminated and the headers could be streamlined if APL supported the naming of generic object names.

In future I foresee domains where the number of named objects will exceed the available space in function headers. Hence the following proposal.

### Proposal

One way to implement generic localization of names would be to apply the same techniques that are already in use elsewhere:

ONE* ..... would localize all names beginning with ONE

TWO? ..... would localize all four letter names TWOx

In addition I would suggest the following convention:

\$aa would localize all two letter names starting with two lower case letters

\$Aa would localize all two letter names starting with an upper case letter followed by a lower case letter

\$a* would localize all names of arbitrary length in lower case

\$Aa* would localize all names starting with an upper case letter followed by lower case letters.

I am not acquainted with the inner workings of APL interpreters. It is possible that some of my proposals may be impractical to implement. For example, the usage of the * cannot be implemented until specific named objects are created. This may be difficult to achieve, and even if possible may slow down the interpreter too much. The usage of the ? and of the \$ on the other hand is more like a short hand notation, because the number of valid names defined by these symbols is limited.

If neither of my suggestions is feasible, there is still the possibility for expanding the maximum length of the header.

---

**TECHNICAL SECTION**

This section of VECTOR is aimed principally at those of our readers who already know APL. It will contain items to interest people with differing degrees of fluency in APL.

---

## Technical Editorial

### Second Generation APL on Personal Computers

by Jonathan Barman and Dave Ziemann

There is some excitement in the APL community about the second generation APL interpreters that are now available on the 386 based machines, and other high-performance personal computers. The main reasons for this interest are the additional power of the new APLs, the availability of the whole machine memory, and the increase in processing speed. Also, all three major second generation APLs are now available, so that it is possible to use the system of your choice. The next issue of VECTOR will contain detailed reviews of IBM's APL2/PC, STSC's APL*PLUS II for the 80386, and Sharp APL/UX, or SAX, as it is more commonly called.

Second generation APLs are more powerful than vanilla flavoured APL, so much more so that the increase has been compared with the difference between Fortran and APL. The restrictive universe of ISO standard arrays can place constraints on the way of thinking. All kinds of subtle algorithms have been invented to get around these limitations, which make newcomers to APL gasp and stretch their eyes. Rather than increasing complexity, nested or enclosed arrays are a more natural way of thinking about, and structuring data. With generalised operators, these tools often provide simpler solutions to problems which can otherwise encourage the programmer to search for an elusive and perhaps unreadable 'parallel' solution.

Of course, there are the usual problems with added power. Huge arrays can be created with a few keystrokes, and as it looks so simple the perpetrator fails to realise quite how much data has been created or how much redundant computation is performed. The condensed notation can take a little while to get used to, and can appear complex, in just the same way as ordinary APL does at first.

At long last it is possible to use all the memory in a machine. Memory is now quite cheap, and most modern PCs already have more than the 640K limit of the old processors. Getting around the 640K barrier was always difficult. APL*PLUS PC release 7 removed some of the pain with the an automatic overlay technique, but this could only be regarded as a stop gap measure pending a proper solution. Additional programming is often necessary where large amounts of data are involved, but the extra memory should relieve the programmer from having to overlay code as well.

The performance of the new offerings is surprisingly good, perhaps up to mainframe standards. It is now possible to develop production systems that compete with similar mainframe applications, but with all the benefits of the immediate response of a dedicated PC.

Perhaps the most significant benefit is that many mainframe APL solutions can be transferred to run on personal computers with little change. All the major APL vendors now offer mainframe-equivalent APL systems in the PC environment. The mainframe

workspace size can be matched and usually far exceeded by buying sufficient memory. Most mainframe applications are well within the 16 Megabyte limit, and the vast majority have workspaces of less than 2 Megabytes. If data has to be shared, then networking PCs can provide equivalent solutions, though with large data volumes it could be difficult to match the performance of the big teleprocessing systems.

The three systems mentioned above all show a high degree of compatibility with their mainframe counterparts, although there are significant differences between them. IBMs APL2/PC comes in 16bit and 32bit versions, only the latter providing large workspaces and 4byte integers. The implementation is close to mainframe APL2, with nested and mixed arrays, generalised and defined operators. The IBM approach is to use shared-variables wherever possible, and Auxiliary Processors and the necessary support code are supplied with the product. The key features here are probably mainframe compatibility and connectivity.

STSC's method of integrating the system with its environment is to use specialised quad-functions; the same ones appear throughout the APL*PLUS family of interpreters on all machine sizes. The system provides the same functions, nested and mixed arrays, and generalised operators as APL*PLUS mainframe. The APL*PLUS II key benefits are high performance, environmental integration, and cross-range compatibility. The under-exposed Sharp APL/UX interpreter is a Unix based extension of the Sharp APL/370 language. In fact this system closely approaches the ideal APL language as described by Ken Iverson in his Dictionary of APL. It supports enclosed arrays, mixed arrays and generalised operators, as well the full range of operators available on the mainframe system. Key features here might well be perceived to be language purity and completeness, leading to a highly productive application development environment.

All in all, interesting and exciting prospects. Get your own machine and run your application without having to put up with unpredictable response times.

The theme for the next issue of VECTOR will be second-generation APLs on personal computers. Our reviewers will be taking you through the options and you can make up your own mind which of these APLs must be yours. Following on from our technical reviews, we would like to publish the experiences of users of these systems. VECTOR will be pleased to receive letters, articles and case-studies on the use (or abuse) of second-generation APL products on PCs; perhaps you have found them too resource hungry, or the perfect vehicle for delivering high-performance solutions. Extended APLs may relieve the programming burden considerably, or on the other hand you may feel that they encourage unreadable and unmaintainable code. Either way, please let us know. Print-files on DOS-format disks are perfectly acceptable, and for longer articles we encourage you to also provide high-quality diagrams where possible.

At the other end of the scale, a forthcoming issue of VECTOR will be featuring I-APL version 1.1, STSC's Pocket APL, and Sharp APL/PC.

# Technical Correspondence

From: Iain Hayward

10th January 1989

Well, in the end I went ahead and wrote a font designer in APL. It was fun and it gave me an excuse to use some of the special features of APL68000 on the Amiga. Having written my own program, I couldn't resist the temptation to try it out, and ended up having designed a font to my own taste.

I am sending you the font design because I want to put it into the public domain before any enterprising individual tries to copyright it and sell it, thereby making life harder for other APLers. I am therefore making it available to the British APL Association well before I give it to anyone else.

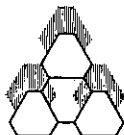
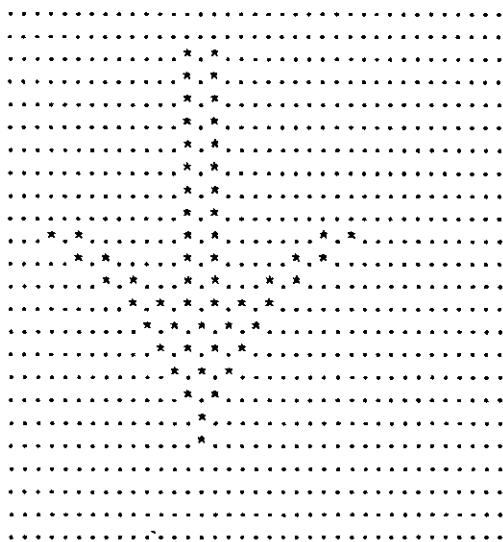
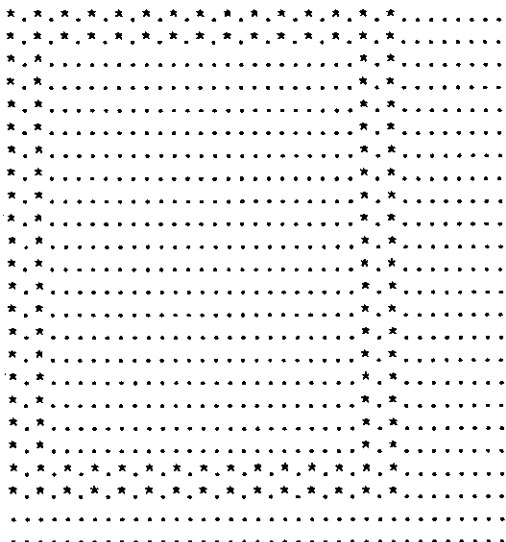
I have sent you the design on paper (a selection of sample characters are printed opposite ... Ed.) because it could probably be used on any 24-pin printer. I have also enclosed a listing of the APL68000 character set as it appears on my NEC CP6 using my font. Later on I can send the binary file and the printer driver.

Yours sincerely

Iain Hayward  
280A rue de Neudorf  
L-2222 Luxembourg  
Grand Duchy Luxembourg.

Ψ	0	-	*	φ	P	"	Å	ê	À	p		
Δ	..	1	α	?	A	Q	#	À	ä	a	q	
Φ	)	2	ι	ρ	B	R	%	Ç	í	O	b	r
ø	<	3	η	Γ	C	S	&	È	ì		c	s
ø	≤	4	l	~	D	T	@	ú	í		d	t
●	=	5	ε	+	E	U	£	Ö	ï		e	u
I	>	6	_	U	F	V	`	Û	ñ		f	v
⊞	∇	]	7	∇	ω	G		á	ó		g	w
±	v	8	{	∞	H	X		à	ò	Ø	h	x
⊞	∇	^	9	ι	†	I		â	ô	ø	i	y
∇	∇	∇	(	°	C	J		ä	ö		j	z
⊞	/	+	[	'	+	K		ã	õ		k	Δ
⊞	A	,	;	□	Γ	L		á	ú	ß	l	
∇	∇	+	x		→	M		ç	ù	ÿ	m	
∇	∇	!	.	:	T	≥		é	û		n	
∇	⊞	/	\	o	-	O		è	ü		o	

Copyright December 1988 by Iain Hayward.



## **APL CONSULTING**

**QUALITY APL CONSULTANCY SERVICE  
SPECIALIZING IN:**

- ★ Project Management
- ★ Prototyping and Database Design
- ★ Marketing Information Systems  
for Management
- ★ Design and implementation of  
large financial systems

We can solve your APL problems, large or small,  
and will supply a single consultant or a supervised  
team according to your needs.

Contact Jill Moss or Tim Perry on 0225 462602  
APL Consulting Ltd, The Old Malthouse, Clarence St., Bath BA1 5NS

## Hackers Corner: An Undocumented Auxiliary Processor in APL2

by John Sullivan, National Westminster Bank, plc.

We have long felt a need for a 'bottom-up' workspace-management tool, i.e. one that allows the administration team to look at what objects are in which workspaces. Up to now the only way to do this is to load individual workspaces (which can be a pain if the latent function calls up a full-screen dialog) and to list the objects from the workspace. What is needed is a single central workspace, whose tentacles can reach into all the other workspaces on the system and pull out the details of the global symbol table, and other information such as fix-times for functions and operators.

What follows has been derived in an afternoon's doodling after a purely accidental decision to look at the functions in IBM's supplied workspace 2 TSOIVP (installation verification program). This was followed up by looking at the code for AP104. After documenting the features I came across a paper [1] in which they are discussed, but a full description of the arguments and error codes is not given. So far this is the only documentation of AP104 I have been able to find.

### Auxiliary Processor 104

Even though you are using Release 2 of IBM's APL2 program product, you may not have heard of Auxiliary Processor 104. There is a good reason for this: IBM has not documented it. This is because it is only 'temporary' in the program product, and it may not be present in future releases.

I have not had too much time to play with it, but I have discovered the following about its operation. First of all set up a shared variable. A single variable is all that is required; any name will do, and it doesn't need any initialization. Then set 'commands' into the shared variable, and obtain the results by referencing it. Commands must be simple character vectors, the results are nested vectors containing two numeric scalars (the minimum information returned) followed by one or more character vectors.

The following function sets up the shared variable the first time it is used, then leaves it in the workspace for use by future calls. As implied by its name it is only a test function, so don't worry too much about lines 8 and 9.

```

▽TEST104 X;A
[1]  →(2=⊞SVO 'A104')/L2
[2]  A←104 ⊞SVO 'A104'
[3]  ⊞SVE←5
[4]  L1:→(2=⊞SVO 'A104')/L2
[5]  →(0=⊞SVE)/L1
[6]  →0,⊞Q+'Can't share <A104> with AP104'
[7]  L2:A104←X
[8]  'Shape of reply is ',(⊞A),', with lengths ',⊞p'A←A104
[9]  A

```

## Commands

The available commands are LIB and MLIB (which are followed by an optional library number) and LIST, MLIST, COPY, PCOPY and MCOPY. The parameters for the last group appear to be

- an optional global/local indicator (in parentheses)
- an optional library number
- a required workspace name
- an optional name class
- an optional 'extra parameter' which serves as a string of start-letters or a list of objects to be copied (delimited by spaces)

## Defaults and Peculiarities

The global/local indicator is a G or L enclosed in parentheses. G indicates that we are only interested in global objects; L means we want localized objects, but this will only give a different result if the workspace contains suspended functions, and you only get the most local version of each object, including the globals if there isn't a local version with the same name. If you omit this parameter, the AP assumes you want L. If you omit the library number the AP assumes you want to use your private library. If the name class is invalid it is ignored. If you specify a name class the processor will list or copy all objects of that name class that start with the first string of the 'extra parameter' (further strings are ignored). If you omit the name class the processor lists everything, or copies only those objects listed in the command. The start letters restrict the output. If you specify ABCD then you will get a list of objects whose names start with ABCD. If you specify start letters with an invalid name class or none at all the start letters are ignored for LIST or assumed to be the complete name of an object for COPY. If you do not give the 'extra parameter' the processor assumes that you want to list or copy everything. This processor uses lower-case letters rather than underscores (which for those of you familiar with auxiliary processor conversion options is the BCD or 370 option). You do not have any choice here! VSAPL groups are considered to be variables (as with those in APL2). Although the APL2 commands )COPY, )PCOPY and )MCOPY will copy groups if the name is enclosed in parentheses, AP104 will not do so. In fact, the parentheses will cause the command to fail with return codes 1 0 (some objects not copied). You can only copy system variables from VSAPL workspaces (i.e. with MCOPY, not with COPY or PCOPY). This mirrors the behaviour of the associated APL2 commands.

## Examples

```
LIST (G) TEST
```

will list all of the variables, functions and operators in workspace TEST. No distinction is made between the different types of object in the listing.

```
COPY (G) 2345 TEST 3 DEF
```

copies all the global functions in workspace 2345 TEST that begin with the letters DEF.

## Results

The returned data is a nested vector, the first two items of which are the return codes:

```

0 0 ..... No error.
1 0 ..... Some objects not copied
2 0 ..... Don't know
2 1 ..... Invalid command passed to AP104
2 2 ..... Library does not exist
2 3 ..... Workspace not found. The specified name is expanded by the system
        to give the appropriate dataset name for a workspace (VSAPL, or
        APL2 private workspace or APL2 SAM library), or a search of the
        appropriate VSAM cluster is made (APL2 VSAM library). This error
        occurs if the dataset does not exist, or the workspace header is not
        found in the VSAM cluster, or items that do exist do not contain a
        valid workspace of the appropriate type. If you want to know what
        your system expects VSAPL workspaces to be called, enter )MCOPY
        TEST (where TEST is non-existent) and then enter )MORE
        immediately after the error message. APL2 tells you the name of the
        dataset it expects to find but can't. On my system VSAPL datasets
        should be called <userid.@W.wsname>.
2 4 ..... Workspace locked
2 5 ..... Library not available
2 6 ..... Library I/O error
2 7 ..... System Limit
2 8 ..... Don't know for sure but it could be 'Workspace is being used by
        somebody else, try later'
2 101 ..... User aborted operation
2 102 ..... Insufficient memory
2 103 ..... Insufficient shared memory
2 104 ..... Unexpected SVP error
2 105 ..... Unexpected system error
2 106 ..... Unexpected auxiliary processor error
3 0 ..... Unmatched shared variable offer

```

If the error codes are other than 0 0 or 1 0 then only these two items will be present in the result vector. If COPY, PCOPY or MCOPY managed to copy everything you asked for then there should be only three items, otherwise there will be seven items, unused items being null. LIB and MLIB will give you four items. The extra items (beyond the second) are as follows:

The **third** element is always null with LIB and MLIB the workspace date-and-time saved with the others. The **fourth** element is always a list of workspaces with LIB and MLIB the requested list of objects with LIST, which will be null (we hope) if you ask for Global labels (name-class 1) (see later on) a list of objects requested but not found for COPY, PCOPY and MCOPY. The **fifth** item is a list of objects not copied (only applies to PCOPY, obviously). I have not got anything other than the null vector in items 6 and 7.

Lists of workspaces are returned as 'a nine-column left-justified matrix of workspace names, ravelled'. Otherwise, all lists of objects are returned as vectors with the names separated by single spaces (and ending with a space). These lists can, of course, be converted to vectors of vectors and matrices by means of DAN (IBM-supplied with associated processor 11) and disclose.

### And finally

Be careful if you are COPYing a function and your workspace already contains a suspended function of the same name. As is documented for )IN, you do not get a state-indicator warning. Continued execution of the suspended function picks up the OLD function, any further reference to the function (even while the old one is still suspended) will pick up the NEW function. As I said, be careful!

Be careful if you use this AP in any case. Obviously IBM will not accept responsibility for anything that you might do, and nor will I. If you manage to find out more about this auxiliary processor then please let me or the editor know.

Does anybody out there know how to get Global Labels, deliberately rather than accidentally? What am I doing wrong? I seem to get them now and again in my workspaces for no good reason. Checking that )SI is clear, I enter quad-NL 1, and I get a list of items which, from my strictly-applied naming conventions can only have originated as statement labels from some function or other. Copying them into another named workspace or a clear workspace converts them into variables! What on earth is going on????

### References

- [1] Michael Deichmann: Hidden and missing features in APL2. (Maersk Data A/S, Copenhagen, Denmark, 21 October 1986). Mr Deichmann gives a brief application of the auxiliary processor in a workspace-management environment, and ends with a plea for this processor to be made more generally available. I wholeheartedly support this plea, but would also like to see extra features made available, such as the function-fix time mentioned in the first paragraph, above.

## Designing a Good Character Scatterplot in APL

by Alan Sykes

### Introduction

As some readers will know, the department of Management Science and Statistics at Swansea uses APL extensively in its teaching of Statistics. Most of our students use DEC APL on a micro-vax and although we have a comprehensive APL graphics workspace with output to a Hewlett-Packard graph plotter, nevertheless it is useful to have a character-graphics scatterplot which will work on any APL machine/terminal combination!

For too long our scatterplot program had rudimentary information on the axes. A bout of influenza gave me a chance to stay at home and use an IBM PC to produce a scatterplot that looks as good as it can when limited to using characters only. Scatterplots in the APL literature seem to stop short of a full-blown version with automatic choice of sensible tic-values and so the following program may find a use for someone somewhere.

### Requirements

What was needed was a generally useful scatterplot representation of x-y data using characters only. Axes with tic-marks and tic-values were required.

### High-level Design

After looking at a number of scatterplot diagrams in books, together with output from stats packages such as GLIM and MINITAB, I decided on the following design criteria:

**Size.** The overall size of the plot was to be variable. This was advantageous in allowing flexibility of proportions on the screen and also to allow a large plot to be sent to the printer. (One of the problems with character plots on the screen is that non-equal x or y values can become equal when printed, leading to possible distortion of underlying curves. When there are many data points this effect can be very noticeable and potentially misleading.)

**Multiple points.** Genuine or non-genuine multiple points should be denoted by different characters, so I decided on the convention of using three different characters, one for a single point, one for a double point, and one for three or more points.

**Tic-marks and axes.** The axes were to be represented by horizontal and vertical dashes interspersed at regular intervals with a '+' sign.

**Tic-values.** These should be chosen to be appropriate multiples of powers of 2,5 or 10 to give neat-looking numbers. The values should be printed adjacent to the tic-marks and (where appropriate) to left-justify them. The number of such tic- marks should be under user control as far as possible, though default settings should normally produce non-overlapping labels.

### Further Design Details

Global variables ...

$\Delta$ CHARPLOT .....a four element character vector giving the plotting characters. The default setting is ' *oo'

$\Delta$ NCHAR .....a two element vector denoting the target size of the actual plotting matrix. For a conventional 80 by 25 screen, the recommended default value for  $\Delta$ NCHAR is 60 18.

$\Delta$ NTIC.....a two element vector denoting the target number of tic marks on each axis. It is not possible to specify  $\Delta$ NTIC completely if sensible tic-values are to be chosen automatically. The actual number is usually at least that specified and values of 2 2 or 4 4 work well most of the time.

APL functions ...

PLOT (R+yvar PLOT xvar) ... the end product

$\Delta$ CHOOSE TIC (PAR+axis_number  $\Delta$ CHOOSE TIC min_val max_val)

This is an adaptation of an equivalent program for high-resolution graphics plotting. The left argument is 1 for the x axis and 2 for the y axis. The right argument is a two element vector denoting the range of the data on that axis.

The result PAR is an eight element vector of parameters defining the axis details as follows:

- PAR [ 1 ] overall length of axis in characters;
- PAR [ 2 ] tic-length in characters;
- PAR [ 3 ] number of characters before the first tic-mark;
- PAR [ 4 ] number of characters after the last tic-mark;
- PAR [ 5 ] tic-length in mathematical units;
- PAR [ 6 ] the value of the first tic-mark;
- PAR [ 7 ] the value of the last tic-mark;
- PAR [ 8 ] the number of complete intervals.

The motivation for PAR [ 3 4 ] is that one does not want to waste the use of valuable space on the screen by extending the axes to the next tic-marks beyond the range of actual values (as one might in a high-resolution graphics package).

$\Delta\text{CONV}$  (T+par  $\Delta\text{CONV}$  data)

The left argument PAR contains the parameters of the axis, the right argument contains the data for that axis, and the result is an integer representation of the data vector compatible with the choice of tic-marks and axis specification. There are a number of ways that the raw data can be converted to integer values, but care must be taken so that recorded values equal to tic-values are actually represented accordingly.

## Function Listings:

```

      V R+Y PLOT X;MIX;MAX;MIY;MAY;PARX;PARY;T;IO;C;D;MU;F;I
[1]  IO+1 0 2 1 p' '
[2]  +(0≠pR+(33×(pX)≠pY)p'ARGUMENTS HAVE UNEQUAL DIMENSIONS')/0
[3]  MIX+L/X 0 MAX+L/X 0 MIY+L/Y 0 MAY+L/Y
[4]  +(0≠pR+(16×1=(MIY=MAY)∨MAX=MIX)p'DEGENERATE RANGE')/0

[5]  A CALCULATE PARAMETERS FOR X AND Y AXIS

[6]  PARX+1 ΔCHOOSSETIC MIX,MAX 0 PARY+2 ΔCHOOSSETIC MIY,MAY
[7]  R+'TOO MANY TIC MARKS REQUESTED - CHANGE ΔTIC' 0
      +(0=PARX[2])∨0=PARY[2])/0

[8]  A TRANSFORM THE DATA

[9]  D+(PARY ΔCONV Y),[1.5]PARX ΔCONV X 0 C+PARY[1],PARX[1]
[10] R+(X/C)p0 A CALCULATE FREQUENCIES FOR EACH POINT OF X-Y PLOT
[11] I+V/D≠10D+D[ΔD;] 0 I[pI]+1 0 MU+I/D 0 F+(1+I)-1+I+0,1/1
[12] R[(C10MU)-C[2]]+F 0 R+0CpR
[13] R+ΔCHARPLOT[4][1+R]

[14] A ADD Y AXIS

[15] Y+(1+pR)p' '
[16] Y[(1+PARY[3])+PARY[2]×0,1PARY[8]]+'+'
[17] R+(0Y),R

[18] A ADD X AXIS

[19] X+(1+1+pR)p' -'
[20] X[(1+PARX[3])+PARX[2]×0,1PARX[8]]+'+'
[21] R+R,[1] ' ',X
[22] R+' ',R

[23] A CALCULATE AND ADD Y TIC VALUES

[24] Y+T((pY),1)pY+PARY[6]+PARY[5]×0,1[(-/PARY[7 6])÷PARY[5]
[25] X+PARY[1]p0 0 X[(1+PARY[3])+PARY[2]×0,1PARY[8]]+1
[26] Y+X\1Y 0 R+((0Y),[1] ' '),R

[27] A CALCULATE X TIC VALUES AND CHECK FOR SUFFICIENT SPACE

[28] X+T((pX),1)pX+PARX[6]+PARX[5]×0,1[(-/PARX[7 6])÷PARX[5]
[29] X+(0T+/\X=' ')0X
[30] T+PARX[2]-1+pX 0 +(T<0)/ER

[31] A ADD X TIC VALUES

[32] X+((2+(1+pY)+PARX[3])p' '),( -T)+X,((1+pX),T)p' '
[33] T+(1+pR)-pX 0 X+X,(0T)p' '
[34] R+R,((1+pR),0T-T)p' ' 0 R+R,[1]X 0 0
[35] ER:'INSUFFICIENT SPACE FOR TIC MARK LABELS ON THE X AXIS'
[36] 'MISSING LABELS ARE ' ',X
      V

```

```

      ▽ P←N ΔCHOOSSETIC X;RA;A;C;B;R;I
[1]  RA←--/X ◇ P+8p0

[2]  A RA IS THE RANGE: CALCULATE SUITABLE BASE FOR TIC VALUES

[3]  A←10*[10*C+RA÷ΔNTIC[N] ◇ B+C÷A
[4]  B← 1 2 5 10 [1++/B> 1.4 3.2 7.1]

[5]  A FIND R=P[1]=NUMBER OF CHARACTERS FOR RANGE OF AXIS
[6]  AP[5]= MATHEMATICAL TIC-LENGTH P[6]= VALUE OF FIRST TIC MARK

[7]  P[5]+R+B×A ◇ P[6]+R×[X[1]÷R ◇ P[7]+R×[X[2]÷R

[8]  A P[8]= NUMBER OF COMPLETE INTERVALS P[2]= TIC-LENGTH IN
      CHARACTERS

[9]  P[8]+(-/P[7 6])÷P[5] ◇ P[2]+[ΔNCHAR[N]×R÷RA

[10] A P[3 4]= NUMBER OF TIC CHARACTERS BEFORE (AFTER) FIRST
      (LAST) TIC-MARK

[11] P[3 4]+[0.5+(1 ~1)×(P[6 7]-X[1 2])×P[2]÷P[5]
[12] P[1]+1+P[3]+P[4]+P[8]×P[2]

[13] A AMEND ACCORDING TO WHETHER P[3 4] EQUAL TIC-LENGTH IN CHARS

[14] P[3 4]+P[3 4]×~1+P[2]=P[3 4]
[15] P[6 7]+P[6 7]+(~1 1)×I×P[5]
[16] P[8]+P[8]++/I
      ▽

```

```

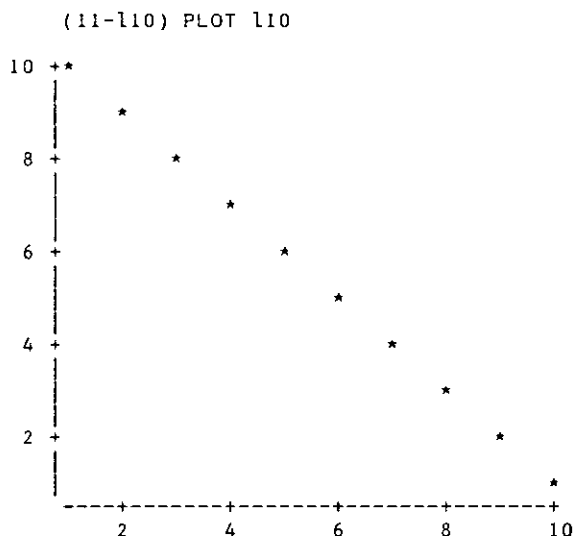
      ▽ R←P ΔCONV X;D
[1]  A CONVERTS RAW DATA TO INTEGER DATA ACCORDING TO
      SPECIFICATION FROM P

[2]  D←[X÷P[5] ◇ R+X-D+D×P[5]
[3]  D←(D-P[6])÷P[5]
[4]  R←P[1][1+(D×P[2])+P[3]+[0.5+R×P[2]÷P[5]
      ▽

```

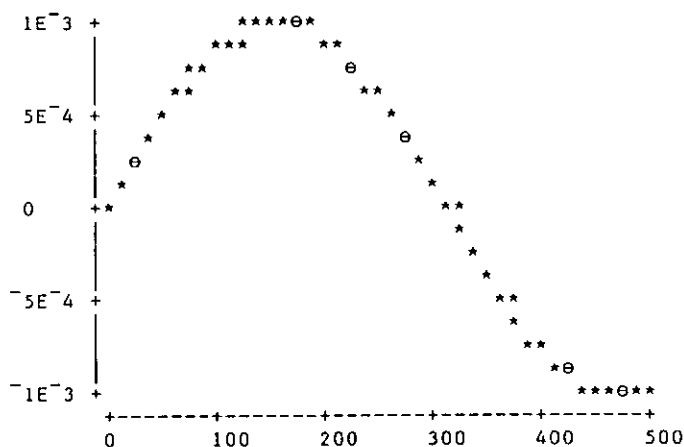
## Some Examples

Set  $\Delta NCHAR$  to 40 18 and  $\Delta NTIC$  to 4 4. (These are quite good default values for general usage. To test the accuracy of the plot try:

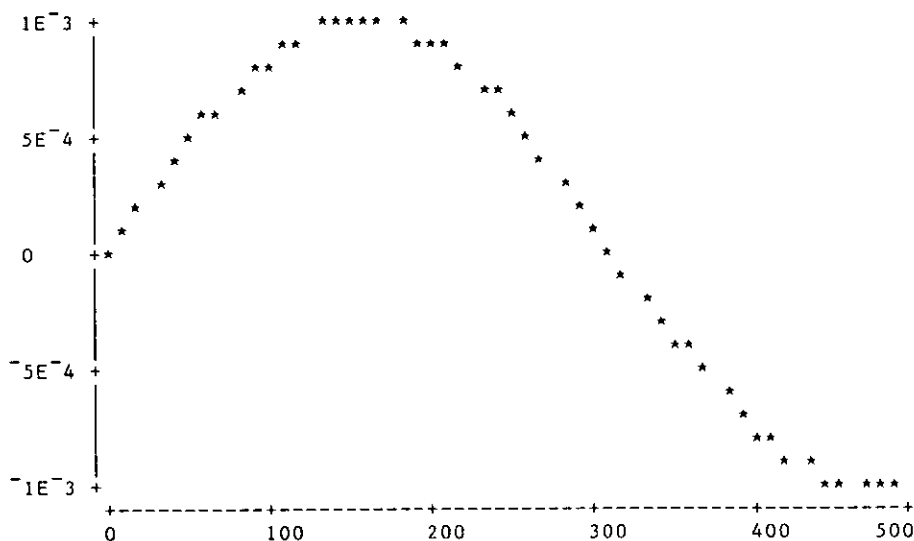


Varying values of  $\Delta NTIC$  (whose components do not have to be equal of course) alter the choice of the interval widths. For this graph, a value of 1 in  $\Delta NTIC$  results in an interval of width 10, 2 gives interval width 5, 3 4 5 and 6 all give width 2, 7 8 9 and 10 all give width 1. Ultimately you run out of space and may either get no plot at all, or perhaps, one without tic-values on the X-axis.

To consider the effect of differing sizes of diagrams, consider a plot of  $Y + .001 \times 10.01 \times X$  against  $X + 10 \times 0, 150$ :



and the effect of crowding is apparent. Changing  $\Delta NCHAR$  to 60 20 gives:



which is a little smoother. Note that the actual size of this is 70 by 23 and not 60 by 20. So the programs appear to produce sensible plots with good choices of tic-values, but we

have to pay for this facility by not having complete control over the number of tic-marks and the total size of the plot.

### Efficiency

PLOT does not produce instantaneous results as there are many involved calculations necessary. (I hope there are no unnecessary ones!) Where efficiency considerations are important is in the calculation of the frequencies to be attached to each point of the plotting region. The obvious APL one-liner (which could replace lines 10, 11 and 12 of PLOT)

```
R+((PARY[1]+1-1PARY[1])°. =Y)+.X°. =1PARX[1]
```

...is rather slow, as it creates two potentially large boolean matrices and then multiplies them together. David Ziemann's recent talk on efficiency (see VECTOR 4.4, April 1988) points to more efficient methods. Lines 10, 11 and 12 of PLOT calculate unique pairs of X,Y values by using  $\uparrow$  on a matrix, hence finding their associated frequencies, assigning these to appropriate values of a vector (initially of zeros) and then reshaping. This technique works well for producing two-way frequency tables when working with character data from a database. But that's another story.

### Editor's Note

I have used Alan's approach very successfully in designing simple barcharts which can be 'popped up' in text windows. A handy feature of the PC character set is the presence of half-width and half-height bars, so with a simple modification to his code you can get a screen resolution of 50 by 160, and yet still be in text mode.

*,

**British APL Association Software Library  
Software Submission Form - Page 1 of 2**

Please copy and fill in this form for EACH disk you submit.

Details corresponding to items flagged (*) will NOT be made publically available, but are for our records only. Please Use BLOCK CAPITALS for all items except number 18.

0. Submission date: _____
1. Name: _____ 2. Daytime phone(*): _____
3. FULL address(*): _____  
_____  
_____
4. Electronic mail addresses(*): _____
5. Short disk title: _____
6. Brief description of disk: _____  
_____  
_____
7. List target machine: _____
8. List additional software required: _____
9. Indicate special hardware requirements: _____
10. Is user documentation provided on the disk?(Y/N): _____
11. List titles of any paper documentation included with your submission: _____  
_____
12. Is this documentation, or any other, available to users upon application to you? (Y/N): _____ Please give details: _____  
_____
13. Does the disk include any form of payment request from users of the software?(Y/N): _____ Please give details: _____  
_____
14. Does your submission constitute a 'demonstration disk' in that it demonstrates software that is available for purchase?(Y/N): _____

**British APL Association Software Library**  
**Software Submission Form - Page 2 of 2**

15. If the TARGET machine is NOT a DOS-based PC, does the disk include instructions for transfer to the target machine?(Y/N):_____
16. File names and descriptions. This information will be made publically available in the software library catalogue. Please save us some work by including these details on a file named <CAT> (no file extension) on your submitted disk. Please enter these details for all files on the disk, except <CAT> itself. You can affix a print of <CAT> below.

For APL workspaces, please use the space below to document functions.

[illegible]

17. Use any extra space above for comments you wish to appear in the software library catalogue.
18. Your signature is necessary. It declares your legal right to make the disk freely available for copying and use, and grants that right to the British APL Association.

Signature: _____

Mail your submission to: The BAA Software Library, c/o Alison Chatterton, 9 Oak Grove,  
Hertford, Herts SG13 8AT, ENGLAND

## BAPLA Software Library: Order Form

To: Alison Chatterton  
 9 Oak Grove,  
 Hertford,  
 Herts SG13 8AT, ENGLAND

Please supply the disks circled below:

0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17
18	19	20	21	22	23	24	25	26	27	28	29	30	31	32	33	34	35
36	37	38	39	40	41	42	43	44	45	46	47	48	49	50	51	52	53
54	55	56	57	58	59	60	61	62	63	64	65	66	67	68	69	70	71
72	73	74	75	76	77	78	79	80	81	82	83	84	85	86	87	88	89
90	91	92	93	94	95	96	97	98	99	100							

Total number of disks (please check): _____

at 2 Pounds per disk: BAA Member's rate _____

OR at 3 Pounds per disk: Non-member's rate _____

I want to join the BAA, and enclose  
 the 10 Pounds annual membership fee: _____

Postage and handling: 1.00

Add 2 Pounds for orders outside the UK: _____

Total order - remittance included: _____

Your signature: _____

Your name: _____

Full address: _____  
 _____  
 _____  
 _____

Please make cheques payable to the British APL Association. Payment can be made in USD - \$1.5 for each £. All orders must include payment - allow 30 days for delivery.

Software is accepted by the British APL Association on good faith and we do not vouch for or make any claims regarding donated software. The BAA shall not be held responsible for damage caused by the use of library software, and shall not be liable in any way in the event that submitted material proves to be the subject of third party copyright. Your signature above accordingly indemnifies the BAA from all liability.

**Index to Advertisers**

APL People	111
British Airways	2
Cocking and Drury	16,86
Dyadic Systems Ltd	8
HMW Computing	34
Maddox Ford	6
Mercia Software	12
Metapraxix	54
MicroAPL	90
APL Booklist	50

All queries regarding advertising in VECTOR should be made to the advertising editor, Cathy Dargue, at the following address:

Cathy Dargue,  
60 Downhall Ley,  
Buntingford,  
Herts SG9 9TL

Tel: 0895-31313 ext 3038

## BRITISH APL ASSOCIATION Membership Application Form

Membership of the British APL Association is open to anyone interested in APL. The membership year runs from 1st May to 30th April.

Name: _____  
 Department: _____  
 Organisation: _____  
 Address Line 1: _____  
 Address Line 2: _____  
 Address Line 3: _____  
 Post or zip code: _____  
 Country: _____  
 Telephone Number: _____

Membership category (please ring): .....88/89    89/90

Non-voting student membership (UK only) .....	£5	£5
UK private membership .....	£10	£10
Overseas private membership .....	£18	£18
Airmail supplement (not needed for Europe) .....	£8	£8
Corporate membership .....	£85	£85
Corporate membership overseas .....	£140	£140
Sustaining membership .....	£360	£360

For student applicants:

Name of course: _____  
 Name and title of supervisor: _____  
 Signature of supervisor: _____

If you would like to pay by credit card ...

Card no (Access/Visa): _____

### PAYMENT

Payment should be enclosed with membership applications in the form of a UK Sterling cheque "The British APL Association", or you may quote your Access or Visa number. Please send the completed form to:

BAA Administration,  
 Alison Chatterton, 9 Oak Grove,  
 HERTFORD SG13 8AT  
 England

## The British APL Association

The British APL Association is a Specialist Group of the British Computer Society. It is administered by a Committee of officers who are elected by a postal ballot of Association members prior to the Annual General Meeting. Working groups are also established in areas such as activity planning and journal production. Offers of assistance and involvement with any Association matters are welcomed and should be addressed in the first instance to the Secretary.

### 1988/89 Committee

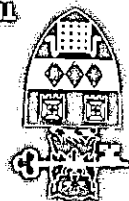
Chairman:	Peter Donnelly 0420-87024	Dyadic Systems Ltd., Park House, The High Street, ALTON, Hants GU34 1EN
Secretary:	Graham Parkhouse 0483-571281 (x2379)	Dept. of Mech. Eng., University of Surrey, GUILDFORD GU2 5XH
Treasurer:	Philip Goacher 03727-21282	27 Downs Way, EPSOM, Surrey
Activities:	Jonathan Martin 01 562-5697	(S499) British Airways, PO Box 10, Heathrow Airport, HOUNSLOW, Middlesex TW6 2JA
Journal Editor:	Adrian Smith 0904-653071 (x3382)	Brook House, Gilling East, YORK.
Education:	Dr Alan Sykes 0792-295296	Dept of Management Science University College of Swansea, Singleton Park, SWANSEA SA2 8PP
Publicity:	Cathy Dargue 0707-325161 (x2418)	60 Downhall Ley, BUNTINGFORD, Herts SG9 9TL
Technical:	Vacant post	
Recruitment:	Val Lusmore 0225-448501	17, Barton Street, BATH, Avon
Projects:	David Eastwood 01-922 8866	MicroAPL Ltd South Bank Technopark, 90 London Road, LONDON SE1 6LN
Administration:	Alison Chatterton 0992-552489	9, Oak Grove, HERTFORD, SG13 8AT

### Journal Working Group

Jonathan Barman	0374-588835
Anthony Camacho	0727-60130
Cathy Dargue	0895-31313
Val Lusmore	0225-448501
Adrian Smith	04393-385
David Ziemann	01-436 9481

### Activities Working Group

Phil Goacher	03727-21282
Maurice Jordan	01-562 3090
David Parker	01-834 2333



The British Computer Society, 13 Mansfield Street, London W1M 0BD.

Peter C. Alex Systems  
215 Goddard Terrace  
LONDON NW6 6ER  
Tel 01-624 7016  
Mobile 0860 377 968

MicroPL Ltd  
South Bank Technology  
30 London Road  
LONDON SE1 6LN  
Tel 01-922 8856

Inner Product Ltd  
Ragb House  
76 Clapham Common Southside  
LONDON SW4 9DG  
Tel 01-673 3654

HMW Computing Ltd  
Hamilton House  
1 Temple Avenue  
LONDON EC4A 3HA  
Tel 01-339 4212  
Fax 01-339 3825

Cocking & Dury Ltd  
189 Tottenham Court Rd  
LONDON, W1P 9LE  
Tel 01-436 9481

AFL People  
The Old Mailhouse  
Cleare St Bath, BA1 5NS  
Tel 0225 462602

HP Sharp Associates  
10 Dean Lane St  
LONDON SW1H 0DX  
Tel 01-222 7038

Meta Systems Ltd  
856 West Barnes Lane  
Mortspur Park, Surrey  
Tel 01-871 1111

AFL Impulse Ltd  
Rusper Sandy Lane  
Ipswich, SEVENOKS  
Kent TN15 0PD  
Tel 0732 885126

Dyadic Systems Ltd  
Park House, The High Street  
ALTON HANTS  
Tel 0420 87024

AFL Software Tech  
14 Rosewood Avenue  
Aveson, BRISTOL BS2 2PP  
Tel 0454 415797

## SUSTAINING MEMBERS

VECTOR is the quarterly journal of the British APL Association and is distributed to Association members in the UK and overseas. The British APL Association is a specialist group of the British Computer Society, APL stands for 'A Programming Language' - an interactive computer language noted for its elegance, conciseness and fast development speed. It is supported on many timesharing bureaux and on most mainframe, mini and micro computers.

The Committee of the British APL Association wish to acknowledge the generous financial support of the following Association Sustaining Members. In many cases these organisations also provide manpower and administrative assistance to the Association at their own cost.